

Framework/Facilitator/STFStationConfig.prw

2383 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "STFStationConfig.ch"
2 #INCLUDE "Protheus.ch"
3 #INCLUDE "AP5MAIL.CH"
4 #INCLUDE "STFSTCFG.CH"
5
6 Static cFil3      := NIL
7 Static cFil2      := NIL
8 Static cFil1      := NIL
9 Static cLG_CODIGO := NIL
10 Static cIdAmb     := NIL
11 Static cNodeAnt   := NIL
12 Static cEsta      := NIL
13 Static oCfgTef    := NIL
14
15 //-----
16 /*{Protheus.doc} STFStatCon
17 Configurador de Estação
18 @return NIL
19 @obs
20 @sample
21 */
22 //-----
23 Function STFStatCon()
24     STFStationConfig()
25 Return
26
27 //-----
28 /*{Protheus.doc} STFStationConfig
29 Configurador de Estação
30 @return NIL
31 @obs
32 @sample
33 */
34 //-----
35 Function STFStationConfig()
36 Local aSize := MsAdvSize( .F. ) //Tamanho da Janela
37 Local aSections := {}           //Seções da Janela
38 Local aPosObj   := {}           //Posições do Objeto
39 Local nTamX := aSize[5] //LARGURA
40 Local nTamY := aSize[6] - aSize[7] //ALTURA
41 Local oDlg := NIL //Janela
42 Local aArea := {} //Area Anterior
43 Local aPosObjD2 := {} //Objeto2
44 Local nTamSx3 := MF7->(TamSx3("MF7_NODE")[1]) //To do - Carregar o código do nó principal
45 Local aDados := {} //No pai, idnode, descricao, ordem, ambiente, gravado, validado, editado, IdInterno
46 Local cIdAmbiente := IIF(nModulo == 12, "1", "3") //Ambiente dos nós
47 Local oPanel := NIL //Painel
48 Local nTamNode := MF7->(TamSx3("MF7_DESCR")[1]) //to do - descrição do nó
49 Local cSearchNode := space(nTamNode) //
50 Local nTamCol := 0 //Tamanho da coluna
51 Local aList := { STR0001, STR0002, STR0003, STR0004 } //ambientes //"1=Retaguarda"###"2=Frete de Lojas"###"3=POS"###"4=Central de PDV"
52 Local cLog := "" //Log

```

```

53 Local oBtn1 := NIL           //Botões
54 Local oBtnLog := NIL        //botão 2
55 Local oBtn3 := NIL          //Botão 3
56 Local nColFin := 0          //Coluna Finaç
57 Local aCoord := {}          //Coordenadas da Tree
58 Local oTree := NIL          //Objeto da árvore
59
60
61   IniStaticVar()
62
63   InsertData()
64
65   cNodeAnt      := ""
66
67   LoadData(@aDados, cIdAmbiente)
68
69   DEFINE MSDIALOG oDlg TITLE STR0005 FROM aSize[7],0 TO aSize[6],aSize[5] PIXEL //"Configurador Loja"
70
71
72   aAdd(aSections, {100, 5, .T., .T. }) // Paine1 1 - 05%
73   aAdd(aSections, {100, 87, .T., .T. }) //Paine1 2 - 85%
74   aAdd(aSections, {100, 8, .T., .T. }) //Paine1 2 - 8%
75
76
77   aArea := { aSize[1], aSize[2], aSize[3], aSize[4], 3, 3, 3, 3}
78
79   aPosObj := MsObjSize( aArea, aSections, .T., .F.)
80
81   //Cria os painéis verticais - PAINEL 1
82   nTamCol := CalcFieldSize("C",nTamNode,0,"@!")
83
84   @aPosObj[01,01],aPosObj[01,02] TO aPosObj[01,03], aPosObj[01,04 ] OF oDlg PIXEL
85
86   @aPosObj[01,01]+3,aPosObj[01,02]+5 MsGet cSearchNode Size nTamCol,10 OF oDlg PIXEL
87   nColFin := aPosObj[01,02]+5+nTamCol+1
88
89   SButton():Create( oDlg, aPosObj[01,01]+3, nColFin , 15, {|| SearchNode(cSearchNode, oTree, @aDados, cIdAmbiente, @oPanel, @oDlg, @cLog)} , .F., STR0006, { ||
!Empty(cSearchNode) } )// --> oObjeto //"Localizar o Nó"
90   //Falta criar o tlist programar o bchange para re-montar a tree
91   nColFin += 40
92   nTamCol := CalcFieldSize("C",Len(STR0007)) //"Estação"
93
94   @aPosObj[01,01]+5,nColFin SAY STR0008 SIZE nTamCol, 10 PIXEL OF oDlg //"Estacao"
95   nTamCol := CalcFieldSize("C",SLG->(TamSx3("LG_CODIGO"))[1],0,"@!")
96
97   @aPosObj[01,01]+3,nColFin MsGet cLG_CODIGO Size nTamCol,10 PIXEL OF oDlg F3 "SLG" PIXEL Picture "@!" VALID BuildList(@oTree, @aDados, cIdAmbiente, nTamSX3, oDlg,
@oPanel, @cLog, cLG_CODIGO, aCoord, nTamSX3)
98   nColFin += nTamCol+1
99   nTamCol := CalcFieldSize("C",20,0,"@!")
100
101   TComboBox():New(aPosObj[01,01]+3, aPosObj[01,04 ] - (nTamCol+10), {||u| if( Pcount( )>0, cIdAmbiente := u,cIdAmbiente ) } ,aList, nTamCol,10, oDlg, ,, { ||
BuildList(@oTree, @aDados, cIdAmbiente, nTamSX3, oDlg, @oPanel, @cLog, cLG_CODIGO, aCoord, nTamSX3) } ,,,.T.,,.F.,.T.,,,,"cIdAmbiente")

```

```

102
103 //PAINEL 2
104 @aPosObj[02,01],aPosObj[02,02] TO aPosObj[02,03], aPosObj[02,04 ] OF oDlg PIXEL
105
106 //Cria a divisão do objeto Tree - 2 Painels horizontais
107 aSections := {}
108 aAdd(aSections, {100, 30, .T., .T., .F.}) //30% Tree
109 aAdd(aSections, {100, 70, .T., .T., .T.}) //70% PaineL - Retorna X, y ao invés de linha/coluna finaç
110
111 aArea := { aPosObj[02,01], aPosObj[02,02], aPosObj[02,03], aPosObj[02,04], 0, 0}
112 aPosObjD2 := MsObjSize( aArea, aSections, .T., .T.)
113 aPosObjD2[02,01] := aPosObj[02,01]
114 //l //h
115 aPosObjD2[02,02] := aPosObjD2[01,04]+3
116 oPanel := TPanel():New ( aPosObjD2[02,01], aPosObjD2[02,02], "" , oDlg, /*[ oFont]*/, /*[ lCentered]*/, /*[ uParam7]*/, /*[ nClrText]*/, /*[ nClrBack]*/,
aPosObj[02,04]-aPosObjD2[02,02], aPosObj[02,03]-aPosObjD2[02,01], /*[ lLowered]*/, /*lRaised*/ )
116 aCoord := aClone(aPosObj[02])
117 aCoord[4] := aPosObjD2[01,04]
118
119 BuildTree(@oTree, @aDados, cIdAmbiente, @oPanel, oDlg, @cLog, aCoord, nTamSX3)
120
121
122 //PAINEL 3
123 @aPosObj[03,01],aPosObj[02,02] TO aPosObj[03,03], aPosObj[03,04 ] OF oDlg PIXEL
124
125 oBtn1 := TButton():New( aPosObj[03,01]+5, aPosObj[02,02]+05, STR0009,oDlg,{|| cLog := ValidAllNodes(cIdAmbiente, oTree:GetCargo(), oTree,
aDados)},55,15,,,F.,.T.,.F.,.F.,{ || .T. },,.F. ) //"Validar Tudo"
126
127 oBtnLog := TButton():New( aPosObj[03,01]+5, aPosObj[02,02]+05+60, STR0010,oDlg,{|| ShowMainLog(oTree:GetCargo(), oTree, oPanel, cIdAmbiente, @cLog, aDados)
},55,15,,,F.,.T.,.F.,.F.,{ || !Empty(cLog) },,.F. ) //"Visualizar Log"
128 If Empty(cLog)
129 oBtnLog:Disable()
130 EndIf
131
132 oBtn3 := TButton():New( aPosObj[03,01]+5, aPosObj[02,02]+05+120, STR0011,oDlg,{||oBtnLog:Free(), oBtnLog := NIL, oDlg:End() },55,15,,,F.,.T.,.F.,.F.,{ || .T.
},,.F. ) //"Fechar"
133
134
135 ACTIVATE DIALOG oDlg CENTERED
136
137 ReleaseStaticVar()
138
139 Return
140
141
142 //-----
143 /*{Protheus.doc} ActionClick
144 Evento do clique do nó da arvore
145 @param cCargo Nó
146 @param oTree Arvore
147 @param aDados Array dos nós
148 @param cIdAmb Código do Ambiente

```

```

149 @param oPanel  Painei
150 @param oDlg    Janela
151 @param cLog     Log
152 @return  NIL
153 @obs
154 @sample
155 /*/
156 //-----
157 Static Function ActionClick( cCargo, oTree, aDados, cIdAmb, oPanel, oDlg, cLog)
158 Local nPos      := 0      //Posicao do Array aDados
159 Local lContinua := .T.    //Continua o processamento
160 Local cFatherNode := ""    //Nó pai
161
162 DEFAULT cCargo := ""
163 DEFAULT oTree  := NIL
164 DEFAULT aDados := {}
165 DEFAULT cIdAmb := ""
166 DEFAULT oPanel := NIL
167 DEFAULT oDlg   := NIL
168 DEFAULT cLog   := ""
169
170 If !Empty(cNodeAnt) .AND.;
171     ( nPos := aScan(aDados, { |x| x[_CODE_NODE] == cNodeAnt}) ) > 0
172
173     cFatherNode := aDados[nPos, _FATHER_NODE]
174     If aDados[nPos, _EDITED_NODE]
175         lContinua := MsgYesNo(STR0012 + AllTrim(aDados[nPos, _DESCRIPTION_NODE]) + " ? ") //"Sair sem salvar as alterações do processo "
176     Else
177
178         aEval(aDados[nPos, _COMPS_NODE], { |c| IIF(ValType(c[_COMP_OBJECT]) == "0", c[_COMP_OBJECT] := FreeObj(c[_COMP_OBJECT]), )})
179
180         If !( cFatherNode <> cNodeAnt .OR. ;
181             ((nPos := aScan(aDados, { |x| x[_CODE_NODE] == cCargo}) ) > 0 .AND. ;
182             (cFatherNode == aDados[nPos, _FATHER_NODE] .OR. ;
183             cNodeAnt == aDados[nPos, _FATHER_NODE] .OR. ;
184             cFatherNode == cCargo ) ) )
185
186             ReleaseChildren(cIdAmb, cNodeAnt, @aDados, @oTree)
187
188         EndIf
189     EndIf
190
191
192 If lContinua
193     If cNodeAnt <> aDados[nPos, _CODE_NODE]
194         nPos := aScan(aDados, { |x| x[_CODE_NODE] == cNodeAnt})
195     EndIf
196     aEval(aDados[nPos, _COMPS_NODE], { |c| IIF(ValType(c[_COMP_OBJECT]) == "0" , c[_COMP_OBJECT] := FreeObj(c[_COMP_OBJECT]), )})
197
198     //Mudou o foco, então libera os objetos criados pelos componentes
199     If cFatherNode == cNodeAnt .OR. ;
200         ((nPos := aScan(aDados, { |x| x[_CODE_NODE] == cCargo}) ) > 0 .AND. ;

```

```

201         (cFatherNode == aDados[nPos,_FATHER_NODE] .OR. ;
202         cNodeAnt == aDados[nPos,_FATHER_NODE] .OR. ;
203         cFatherNode == cCargo ) )
204     //Seta como editado o nó pai
205     If aDados[nPos,_CODE_NODE] == cFatherNode .OR. ;
206         (nPos := aScan(aDados, { |x| x[_CODE_NODE] == cFatherNode} ) ) > 0
207
208         STFSetNodeEdited( lContinua, @aDados[nPos])
209     EndIf
210 Else
211     If !Empty(cNodeAnt)
212         ReleaseChildren(cIdAmb, cNodeAnt, @aDados, @oTree)
213     EndIf
214 EndIf
215
216 Else
217     oTree:TreeSeek(cNodeAnt)
218 EndIf
219
220
221 EndIf
222 If lContinua .AND. ;
223     ( (nPos > 0 .AND. aDados[nPos,_CODE_NODE] == cCargo) .OR. (nPos := aScan(aDados, { |x| x[_CODE_NODE] == cCargo} ) ) > 0 )
224
225     //Carrega os componentes do nó
226     If cNodeAnt <> cCargo
227         cLog := ""
228     EndIf
229     cNodeAnt := cCargo
230     If Len(aDados[nPos,_COMPS_NODE]) == 0
231         aDados[nPos,_COMPS_NODE] := LoadComp(cCargo, cIdAmb)
232     EndIf
233
234     oPanel:Hide()
235     MsFreeObj(@oPanel, .T.)
236
237     MontaPanel(cCargo, oTree, @aDados[nPos], oPanel, cIdAmb,, aDados, @cLog)
238     cNodeAnt := cCargo
239     oPanel:Show()
240
241     If Len( aDados[nPos, _COMPS_NODE]) > 0 .AND. ValType( aDados[nPos, _COMPS_NODE][1, _COMP_OBJECT] ) == "0"
242         aDados[nPos, _COMPS_NODE][1, _COMP_OBJECT]:SetFocus()
243     EndIf
244
245
246 ElseIf nPos = 0
247
248     oPanel:Hide()
249     MsFreeObj(@oPanel, .T.)
250
251     MontaPanel(cCargo, oTree, {}, oPanel, cIdAmb, ,aDados, @cLog)
252     cNodeAnt := cCargo

```

```

253         oPanel:Show()
254
255     EndIf
256
257
258 Return
259
260 //-----
261 /*{Protheus.doc} LoadTree
262 Carrega o nó da árvore
263 @param oDlg      Janela
264 @param oTree      Arvore
265 @param cIdAmb     Código do Ambiente
266 @param cMainNode  Nó principal
267 @param aDados     Array dos nós
268 @return  NIL
269 @obs
270 @sample
271 */
272 //-----
273 Static Function LoadTree(oDlg, oTree, cIdAmb, cMainNode, aDados)
274
275 //Carregar/Nontar nós principais
276 Local nPos := 0      //Posição do nó
277 Local lPai  := .F.   //No Pai
278 Local nTam  := MF7->(TamSX3("MF7_DESCR")[1]) //To do: amanho da descrição do nó
279
280 DEFAULT oDlg := NIL
281 DEFAULT oTree := NIL
282 DEFAULT cIdAmb := ""
283 DEFAULT cMainNode := ""
284 DEFAULT aDados := {}
285
286
287 If (nPos := aScan(aDados, { |x| x[_FATHER_NODE] == cMainNode }) ) > 0 //substituir por um setfilter
288     Do While nPos <= Len(aDados) .AND. aDados[nPos, _FATHER_NODE] == cMainNode
289         If Empty(aDados[nPos, _ENVIRONMENT_NODE]) .OR. cIdAmb $ aDados[nPos, _ENVIRONMENT_NODE] //Está no ambiente, então monta o nó
290             //Verifica se existe filhos para criar um ícone de folder ou de item
291             lPai := ExistNodes(cIdAmb, aDados[nPos, _CODE_NODE], aDados)
292             If !lPai
293                 oTree:AddTreeItem(aDados[nPos, _DESCRIPTION_NODE], "SUMARIO", , aDados[nPos, _CODE_NODE])
294             Else
295                 oTree:AddTree (aDados[nPos, _DESCRIPTION_NODE] , , "FOLDER5" , "FOLDER6" , , , aDados[nPos, _CODE_NODE] )
296                 LoadTree(oDlg, oTree, cIdAmb, aDados[nPos, _CODE_NODE], aDados)
297                 oTree:EndTree()
298             EndIf
299             aDados[nPos, _IS_FATHER] := lPai
300         EndIf
301         nPos ++
302     EndDo
303 EndIf
304

```

```

305
306 Return
307
308 //-----
309 /*{Protheus.doc} LoadComp
310 Carrega os componentes do Node
311 @param  cCargo    Nó
312 @param  cIdAmb    Código do Ambiente
313 @return  aRetorno - Componentes do Nó
314 @obs
315 @sample
316 */
317 //-----
318 Static Function LoadComp(cCargo, cIdAmb ) //Carrega os componentes do Node
319 Local aComps := {} //Lista dos componentes
320 Local nPos   := 0
321 Local aRetorno := {}
322 Local aArea := GetArea()
323 Local aAreaMF8 := MF8->(GetArea())
324
325 DEFAULT cCargo := ""
326 DEFAULT cIdAmb := ""
327
328 MF8->(DbSetFilter({ || MF8_FILIAL == xFilial() .AND. MF8_NODE == cCargo .AND. (Empty(MF8_ENVIRO) .OR. cIdAmb $ MF8_ENVIRO) .AND. (Empty(MF8_PAISES) .OR. cPaisLoc $
MF8_PAISES )}, ;
329 "MF8_FILIAL == xFilial() .AND. MF8_NODE == cCargo .AND. (Empty(MF8_ENVIRO) .OR. cIdAmb $ MF8_ENVIRO ) .AND. (Empty(MF8_PAISES) .OR. cPaisLoc $ MF8_PAISES )"))
330
331 MF8->(DbGoTop())
332
333 Do While MF8->(!Eof())
334   MF8->(aAdd(aRetorno, { MF8_NODE,;/_COMP_NODE 01 //codigo do nó
335     MF8_ID,; // _COMP_ID 02 //id do componentes
336     MF8_ENVIRO,; // _COMP_ENVIRONMENT 03 //ambiente do componente
337     MF8_TYPE,; // _COMP_TYPE 04 //tipo do componente
338     RTrim(MF8_NAME),; // _COMP_NAME 05 //Nome do componente
339     MF8_TITLE,; // _COMP_TITLE 06 //titulo do componente
340     RTrim(MF8_F3),; // _COMP_F3 07 //f3 do componente
341     IIF(!Empty(MF8_DEFAULT),&(AllTrim(MF8_DEFAULT)),NIL),; // _COMP_DEFAULT 08 //valor default do componente
342     RTrim(MF8_TABLE),; // _COMP_TABLE_FILE 09 //NOME DA TABELA OU ARQUIVIO CONFIG //tipo de arquivo de configuração
343     MF8_INDEX,; // _COMP_INDEX 10 //indice da tabela
344     MF8_KEY,; // _COMP_KEY 11 //SE FOI CAMPO DÁ UM EVAL //chave da tabela/arquivo de configuração
345     MF8_INTYPE,; // _COMP_INTYPE 12 */ //tipo de arquivo.ini
346     IIF(!Empty(MF8_VALID),&(AllTrim(MF8_VALID)),NIL),; // _COMP_VALID 13 //expressão de validação*/
347     MF8_LEN,; // _COMP_LEN 14
348     RTrim(MF8_LIST),; // _COMP_LIST 15 // lista de valores //
349     MF8_FTYPE,; // _COMP_FIELD_TYPE 16
350     ,; // _COMP_CTRL_TYPE 17
351     ,; // _COMP_OLDVALUE 18
352     MF8_POSICA,;
353   } ) )
354
355   nPos++

```

```

356     LoadValue(@aRetorno[nPos])
357     MF8->(DbSkip())
358 EndDo
359
360 MF8->(DbClearFilter())
361
362 RestArea(aAreaMF8)
363 RestArea(aArea)
364 Return aRetorno
365
366 //-----
367 /*{Protheus.doc} ExistNodes
368 Verifica se o nó existe
369 @param cIdAmb    Código do Ambiente
370 @param cMainNode Nó principal
371 @param aDados    Array dos nós
372 @return lAchou   Se componente existe, retorna True.
373 @obs
374 @sample
375 */
376 //-----
377 Static Function ExistNodes(cIdAmb, cMainNode, aDados)
378 Local lAchou := .F. //Achou o nó
379 Local nPos := 0     //Posicao
380
381 DEFAULT cIdAmb := ""
382 DEFAULT cMainNode := ""
383 DEFAULT aDados := {}
384
385 If (nPos := aScan(aDados, { |x| x[_FATHER_NODE] == cMainNode }) ) > 0 //substituir por um setfilter
386 Do While !lAchou .AND. nPos <= Len(aDados) .AND. aDados[nPos, _FATHER_NODE] == cMainNode
387     If Empty(aDados[nPos, _ENVIRONMENT_NODE]) .OR. cIdAmb $ aDados[nPos, _ENVIRONMENT_NODE] //Está no ambiente, então monta o nó
388         lAchou := .T.
389     EndIf
390     nPos++
391 EndDo
392 EndIf
393
394
395 Return lAchou
396
397 //-----
398 /*{Protheus.doc} MontaPanel
399 Função para Montar o Painel
400 @param cCargo    Nó
401 @param oTree     Arvore
402 @param aDadosL   Nó de Posicionamento
403 @param oPanel    Painel
404 @param cIdAmb    Código do Ambiente
405 @param aControls Controles da Janela
406 @param aDados    Array dos nós
407 @param cLog      Log

```

```

408 @return NIL
409 @obs
410 @sample
411 /*/
412 //-----
413 Static Function MontaPainel(cCargo, oTree, aDadosL, oPanel, cIdAmb, aControls, aDados, cLog)
414 Local nRow      := 0                //Linha do Painel
415 Local nCol      := 0                //Coluna do Painel
416 Local aObjects := {}               //Objetos do painel
417 Local aPosObj2 := {}               //Coordenadas do painel
418 Local cTitulo  := ""               //titulo do Painel
419 Local nRow1    := 0                //Linhas da seção 2
420 Local lPai     := .F.              //Eh no pai?
421 Local nC       := 0                //contador de controles
422 Local nComps   := 0                //Contador de componentes
423 Local nInitCBox := 0                //Inicializador da Lista
424 Local nTam     := 0                //Tamanho do título
425 Local aList    := {}               //Lista de Valores
426 Local nPos     := 0                //Posicao dos dados
427 Local oPanel2  := NIL              //painel 2
428 Local oBtn1    := NIL              //Botão1
429 Local oBtn2    := NIL              //Botão2
430 Local oBtn3    := NIL              //botão3
431 Local cVariable := NIL              //Variável do bloco
432 Local cGet     := NIL              //Comando do Get
433 Local cNomeRot  := ""              //Nome da rotina
434
435 DEFAULT cCargo   := ""
436 DEFAULT oTree    := NIL
437 DEFAULT aDadosL  := {}
438 DEFAULT oPanel   := NIL
439 DEFAULT cIdAmb   := ""
440 DEFAULT aControls := {}
441 DEFAULT aDados   := {}
442 DEFAULT cLog     := ""
443
444 If ValType(oPanel) == "0"
445     nRow      := Int((oPanel:nHeight ) / 2) //Linha do Painel
446     nCol      := Int((oPanel:nWidth  ) / 2) //Coluna do Painel
447     nRow1    := nRow-29                    //Linhas da seção 2
448 EndIf
449
450 If Len(aDadosL) >= _DESCRIPTION_NODE
451     cTitulo := AllTrim(aDadosL[_DESCRIPTION_NODE]) //titulo do Painel
452 EndIf
453
454 //ÚÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁ ¿
455 //³Parte 01 da tela³
456 //ÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁ Û
457 If Len(aDadosL) > 0
458     lPai     := aDadosL[_IS_FATHER]
459     nPos     := Len(aDadosL[_COMPS_NODE])

```

```

460 EndIf
461
462 For nC := 1 to nPos
463
464   If !aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE] $ "C/R/X"
465     aAdd(aObjects, { CalcFieldSize("C",aDadosL[_COMPS_NODE][nC,_COMP_LEN],0,"@!",aDadosL[_COMPS_NODE][nC,_COMP_TITLE]), 6, .F., .F.}) //Impar Say
466     aAdd(aControls, { aDadosL[_COMPS_NODE][nC,_COMP_TITLE], , , "S", } )
467     nComps++
468     //Verifica a criação da variável
469
470     If aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE] == "G"
471
472       aAdd(aObjects, {CalcFieldSize("C",aDadosL[_COMPS_NODE][nC,_COMP_LEN],0,"@!",aDadosL[_COMPS_NODE][nC,_COMP_TITLE]), 10, .F., .F.} ) //Impar Say
473
474
475       If aDadosL[_COMPS_NODE][nC,_COMP_FIELD_TYPE]=="N" .AND. !Empty(aDadosL[_COMPS_NODE][nC,_COMP_LEN])
476         aAdd(aControls, {aDadosL[_COMPS_NODE][nC,_COMP_DEFAULT] , ;
477           IIF( Empty(aDadosL[_COMPS_NODE][nC,_COMP_INITYPE]),;
478             Replicate("9",aDadosL[_COMPS_NODE][nC,_COMP_LEN]),;
479             Replicate("9",aDadosL[_COMPS_NODE][nC,_COMP_LEN] - Val(aDadosL[_COMPS_NODE][nC,_COMP_INITYPE])+1) + "." +
480             Replicate("9",Val(aDadosL[_COMPS_NODE][nC,_COMP_INITYPE])) ),;
481           aDadosL[_COMPS_NODE][nC,_COMP_F3],;
482           aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE], nC} )
483
484       Else
485         aAdd(aControls, {aDadosL[_COMPS_NODE][nC,_COMP_DEFAULT] , ;
486           "" ,;
487           aDadosL[_COMPS_NODE][nC,_COMP_F3],;
488           aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE], nC} )
489
490       EndIf
491
492       nComps++
493     ElseIf aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE] == "L"
494       If ! "" $ aDadosL[_COMPS_NODE][nC,_COMP_LIST] .AND. ! "" $ aDadosL[_COMPS_NODE][nC,_COMP_LIST]
495         aList := RetSx3Box(aDadosL[_COMPS_NODE][nC,_COMP_LIST],@nInitCBox,@nTam,aDadosL[_COMPS_NODE][nC,_COMP_LEN],AllTrim(aDadosL[_COMPS_NODE][nC,_COMP_DEFAULT]))
496         aList := StrTokArr(aDadosL[_COMPS_NODE][nC,_COMP_LIST], ";")
497
498       Else
499         aList := Eval(&(aDadosL[_COMPS_NODE][nC,_COMP_LIST]) )
500       EndIf
501
502       aAdd(aObjects,{CalcFieldSize("C",nTam,0,"@!",aDadosL[_COMPS_NODE][nC,_COMP_TITLE]), 10, .F., .F.} ) //Impar Say
503       aAdd(aControls,{NIL , NIL, aClone(aList), aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE], nC})
504
505       nComps++
506
507     ElseIf aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE] == "M"
508       aAdd(aObjects,{CalcFieldSize("C",110,0,"@!",aDadosL[_COMPS_NODE][nC,_COMP_TITLE]),(Int(aDadosL[_COMPS_NODE][nC,_COMP_LEN]/110)+1)*10 , .F., .F.} )//Impar Say
509       aAdd(aControls, {aDadosL[_COMPS_NODE][nC,_COMP_DEFAULT] , NIL, ,aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE], nC} )
510       nComps++
511     EndIf
512   ElseIf aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE] == "C"

```

```

511      aAdd(aObjects, {CalcFieldSize("C",Len(aDadosL[_COMPS_NODE][nC,_COMP_TITLE])+5,0,"",aDadosL[_COMPS_NODE][nC,_COMP_TITLE]),10 , .F., .F.}) //Impar Say
512      aAdd(aControls, {AllTrim(aDadosL[_COMPS_NODE][nC,_COMP_DEFAULT]) ,NIL , aDadosL[_COMPS_NODE][nC,_COMP_TITLE], aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE], nC} )
513      nComps++
514      ElseIf aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE] == "R"
515          aAdd(aObjects,{1,1 , .F., .F.}) //Impar Say
516          aAdd(aControls, {aDadosL[_COMPS_NODE][nC,_COMP_DEFAULT] , NIL , ,aDadosL[_COMPS_NODE][nC,_COMP_CTRL_TYPE], nC} )
517          nComps++
518
519      EndIf
520  Next
521
522
523  If nComps > 0
524
525      @ 0, 0 TO nRow1, nCol OF oPanel PIXEL
526      oPanel2 := TScrollBar():New( oPanel, 0,0,nRow1,nCol)
527
528
529      aPosObj2 := MsObjSize( { 0 , 15 , nRow1 ,nCol, 2, 3,3,3}, aObjects, .T., .F.)
530      nTam := CalcFieldSize("C",Len(cTitulo), 0, "@!")
531
532      @05, ( nCol/2 - (nTam/2) ) SAY Upper(cTitulo) PIXEL SIZE nTam, 10*2 OF oPanel2
533
534
535      @nRow1+3, 0 TO nRow, nCol OF oPanel PIXEL
536      oBtn1 := TButton():New( nRow1+8, 05, STR0013,oPanel,{|| WriteControls(@aDadosL,@cLog, cTitulo, oTree, aDados )},55,15,,,F...T...F...F.,{ || aDadosL[_EDITED_NODE]
537      },,.F. ) //"Gravar"
538      If !aDadosL[_EDITED_NODE]
539          oBtn1:Disable()
540      EndIf
541      oBtn2 := TButton():New( nRow1+8, 05+60, STR0014,oPanel,{|| ValidNode(@aDadosL, @cLog, cTitulo, oTree, aDados)},55,15,,,F...T...F...F.,{ || aDadosL[_EDITED_NODE]
542      },,.F. ) //"Validar"
543      If !aDadosL[_EDITED_NODE]
544          oBtn2:Disable()
545      EndIf
546      oBtn3 := TButton():New( nRow1+8, 05+120, STR0015,oPanel,{|| ExibeLog(cCargo, oTree, aDadosL, oPanel, cIdAmb, aControls, @cLog, aDados) },55,15,,,F...T...F...F.,{ ||
547      !aDadosL[_VALID_NODE] },,.F. ) //"Log"
548      If aDadosL[_VALID_NODE]
549          oBtn3:Disable()
550      EndIf
551
552      //Detalhes dos campos
553      For nC := 1 to nComps
554          cVariable := "aDadosL["+Str(_COMPS_NODE)+"]aControls[" + Str(nC) + ", 05] , "+Str(_COMP_DEFAULT)+ " ]"
555
556          cGet := "{ |U| Iif(PCOUNT() > 0, (STFSetNodeEdited(.T., @aDadosL)," +cVariable+" := U), "+cVariable+" )} "
557
558      Do Case
559          Case aControls[nC, 04] == "S"

```

```

560         tSay():New( aPosObj2[nC, 1], aPosObj2[nC, 2] , &("{ || '" + StrTran(aControls[nC, 01], "", "") + "' }"),oPanel2,,,,,T,,, , )
561
562     Case aControls[nC, 04] == "G"
563         cVariable := "" + cVariable + ""
564         aDadosL[_COMPS_NODE][aControls[nC, 05], _COMP_OBJECT] := TGet():New( aPosObj2[nC, 1],aPosObj2[nC, 2],&cGet,oPanel2,aObjects[nC, 01],aControls[nC,
565 02], /*&(cBlkVld)*,,,, .T., .T., .T., , .F., .F., .F., .F. ,aControls[nC, 03],&cVariable,,,,.T.,Empty(aControls[nC, 03]) )
566
567     Case aControls[nC, 04] == "L"
568
569         cVariable := "" + cVariable + ""
570         aDadosL[_COMPS_NODE][aControls[nC, 05], _COMP_OBJECT] := TComboBox():New( aPosObj2[nC, 1],aPosObj2[nC, 2], &cGet,aControls[nC, 03], aObjects[nC, 01],
aObjects[nC, 02], oPanel2, ,, ,,,.T.,,.F.,.T.,,,&cVariable)
571
572     Case aControls[nC, 04] == "C"
573         cGet := "{ |U| Iif(PCOUNT() > 0, "+cVariable+" := U, "+cVariable+" )} "
574         cVariable := "" + cVariable + ""
575         aDadosL[_COMPS_NODE][aControls[nC, 05], _COMP_OBJECT] := TCheckBox():New(aPosObj2[nC, 1],aPosObj2[nC, 2],aControls[nC, 03],&cGet,oPanel2,
aObjects[nC, 01],aObjects[nC, 02],,{|| STFSetNodeEdited(.T., @aDadosL)},,,,,.T.,,)
576
577     Case aControls[nC, 04] == "M"
578         aDadosL[_COMPS_NODE][aControls[nC, 05], _COMP_OBJECT] := TMultiGet():New(aPosObj2[nC, 1],aPosObj2[nC, 2],&cGet,oPanel2,;
579         aObjects[nC, 01],aObjects[nC, 02],/*oFont*/,.F.,;
580         /*nClrFore*/,/*nClrBack*/,/*oCursor*/,.T.,;
581         /*cMg*/,.T.,/*lCenter*/,;
582         /*lRight*/,.F.,/*bChange*/,;
583         ,.F., .T.)
584
585     Case aControls[nC, 04] == "R"
586
587         cNomeRot := Substr(aDadosL[_COMPS_NODE][aControls[nC, 05], _COMP_NAME], 1, At("(", aDadosL[_COMPS_NODE][aControls[nC, 05], _COMP_NAME]) -1)
588
589         If FindFunction(AllTrim(cNomeRot := cNomeRot))
590             STFSetNodeEdited(.T., @aDadosL)
591             //cVariable := "aDadosL["+Str(_COMPS_NODE)+"] [aControls[" + Str(nC) + ", 05] , "+Str(_COMP_NAME)+ " ]"
592
593             Eval(&("{ || "+aDadosL[_COMPS_NODE][aControls[nC, 05], _COMP_NAME]+ " }"))
594             oBtn2:Refresh()
595         Else
596             MsgStop(STR0016 + cNomeRot) //"Função nao encontrada no repositório "
597         EndIf
598     EndCase
599 Next
600
601 eLSE
602     @ 0, 0 TO nRow, nCol OF oPanel PIXEL
603     EndIf
604 Return
605
606 //-----

```

```

609 /*/{Protheus.doc} LoadValue
610 Função para carregar o valor do componente
611 @param aComp      Definição do Componente
612 @return aComp      Componente com o valor carregado
613 @obs
614 @sample
615 */
616 //-----
617 Static Function LoadValue(aComp)
618 Local aAreaX := NIL      //WorkArea da Tabela
619 Local cChave := ""       //Chave da Tabela
620 Local cComp1 := ""       //tipo do Componente + Nome
621
622   DEFAULT aComp := {}
623
624   If Len(aComp) > 0
625     cComp1 := RetCompType(aComp[_COMP_TYPE]) + space(1) + aComp[_COMP_NAME] //tipo do Componente + Nome
626
627     aComp[_COMP_CTRL_TYPE] := "G" //Get
628     Do Case
629       Case aComp[_COMP_TYPE] == _PARAMETRO
630
631         cChave := PadR( AllTrim(aComp[_COMP_NAME]), Len(SX6->X6_VAR))
632         aAreaX := SX6->(GetArea())
633
634
635         If !SX6->(DbSeek(cFilAnt + cChave)) //4
636           If !SX6->(DbSeek(cFil3 + cChave)) //3
637             If !SX6->(DbSeek(cFil2 + cChave)) //2
638               SX6->(DbSeek(cFil1 + cChave)) //1
639             EndIf
640           EndIf
641         EndIf
642       EndIf
643
644
645       If SX6->(Found())
646         If Empty(aComp[_COMP_TITLE])
647           aComp[_COMP_TITLE] := SX6->(AllTrim(AllTrim(X6Descric()) + " " + X6Desc1() + X6Desc2()))
648         EndIf
649         If Empty(aComp[_COMP_FIELD_TYPE])
650           aComp[_COMP_FIELD_TYPE] := SX6->X6_TIPO
651         EndIf
652         aComp[_COMP_DEFAULT] := GetMv(aComp[_COMP_NAME])
653       Else
654         aComp[_COMP_DEFAULT] := SuperGetMv(aComp[_COMP_NAME], .t., aComp[_COMP_DEFAULT])
655       EndIf
656
657       RestArea(aAreaX)
658
659     If Empty(aComp[_COMP_LEN])
660

```

```

661         aComp[_COMP_LEN] := Len(SX6->X6_VAR)
662     EndIf
663
664     Case aComp[_COMP_TYPE] == _ARQUIVO_INI
665         aComp[_COMP_KEY] := AllTrim(aComp[_COMP_KEY])
666         aComp[_COMP_NAME] := AllTrim(aComp[_COMP_NAME])
667         aComp[_COMP_DEFAULT] := GetPvProfString(aComp[_COMP_KEY], aComp[_COMP_NAME], aComp[_COMP_DEFAULT], IIF( aComp[_COMP_INITYPE] == "1" , GetAdv97(),
GetClientDir()+ aComp[_COMP_TABLE_FILE]))
668
669         If Empty(aComp[_COMP_LEN])
670             aComp[_COMP_LEN] := Max(60, Len(aComp[_COMP_DEFAULT]))
671         EndIf
672         aComp[_COMP_DEFAULT] := PadR( aComp[_COMP_DEFAULT], aComp[_COMP_LEN])
673
674         If Empty(aComp[_COMP_FIELD_TYPE])
675             aComp[_COMP_FIELD_TYPE] := "C"
676         EndIf
677
678     Case aComp[_COMP_TYPE] == _CAMPO
679         If Empty(aComp[_COMP_LEN]) .OR. ;
680             Empty(aComp[_COMP_DEFAULT]) .OR. ;
681             Empty(aComp[_COMP_TITLE]) .OR. ;
682             Empty(aComp[_COMP_F3]) .OR. ;
683             Empty(aComp[_COMP_FIELD_TYPE]) .OR. ;
684             Empty(aComp[_COMP_VALID])
685
686         aAreaX := SX3->(GetArea())
687         SX3->(DbSetOrder(02))
688         cChave := PadR(aComp[_COMP_NAME], Len(SX3->X3_CAMPO))
689         If SX3->(DbSeek(cChave))
690             If Empty(aComp[_COMP_TITLE])
691                 aComp[_COMP_TITLE] := SX3->(X3Titulo())
692             EndIf
693             If Empty(aComp[_COMP_DEFAULT])
694                 aComp[_COMP_DEFAULT] := &(SX3->X3_RELACAO)
695             EndIf
696             If Empty(aComp[_COMP_LEN])
697                 aComp[_COMP_LEN] := SX3->X3_TAMANHO
698             EndIf
699             If Empty(aComp[_COMP_F3])
700                 aComp[_COMP_F3] := SX3->X3_F3
701             EndIf
702             If Empty(aComp[_COMP_LIST])
703                 aComp[_COMP_LIST] := SX3->(X3CBox())
704             EndIf
705             If Empty(aComp[_COMP_FIELD_TYPE])
706                 aComp[_COMP_FIELD_TYPE] := SX3->X3_TIPO
707             EndIf
708
709             If AllTrim(SX3->X3_TIPO) == "N" .AND. SX3->X3_DECIMAL > 0
710                 aComp[_COMP_INITYPE] := AllTrim(Str(SX3->X3_DECIMAL))
711             EndIf

```

```

712
713         EndIf
714     EndIf
715     RestArea(aAreaX)
716
717     //Carrega o Valor do Campo
718     If !Empty(aComp[_COMP_TABLE_FILE]) .AND. ;
719         !Empty(aComp[_COMP_INDEX]) .AND. ;
720         !Empty(aComp[_COMP_KEY]) .AND. ;
721         (aComp[_COMP_TABLE_FILE]->(FieldPos(aComp[_COMP_NAME]))) > 0
722
723         aAreaX := (aComp[_COMP_TABLE_FILE]->(GetArea()))
724
725         (aComp[_COMP_TABLE_FILE]->(DbSetOrder(aComp[_COMP_INDEX])))
726
727         If (aComp[_COMP_TABLE_FILE]->(DbSeek(&(aComp[_COMP_KEY])))
728             If Empty(aComp[_COMP_POSICA])
729                 aComp[_COMP_DEFAULT] := (aComp[_COMP_TABLE_FILE]->(FieldGet(FieldPos(aComp[_COMP_NAME]))))
730             ElseIf aComp[_COMP_POSICA] <= Len((aComp[_COMP_TABLE_FILE]->(FieldGet(FieldPos(aComp[_COMP_NAME])))))
731                 aComp[_COMP_DEFAULT] := Substr((aComp[_COMP_TABLE_FILE]->(FieldGet(FieldPos(aComp[_COMP_NAME])))), aComp[_COMP_POSICA], 1)
732             EndIf
733         Else
734             If Empty(aComp[_COMP_DEFAULT])
735                 aComp[_COMP_DEFAULT] := CriaVar(aComp[_COMP_NAME])
736             EndIf
737         EndIf
738
739         Restarea(aAreaX)
740     EndIf
741
742     If (aComp[_COMP_TABLE_FILE]->(FieldPos(aComp[_COMP_NAME]))) = 0
743         aComp[_COMP_CTRL_TYPE] := "X"
744     EndIf
745
746 EndCase
747
748 If aComp[_COMP_TYPE] <> _ROTINA //rotina somente macro-executa
749
750     If Empty(aComp[_COMP_FIELD_TYPE])
751         aComp[_COMP_FIELD_TYPE] := ValType(aComp[_COMP_DEFAULT])
752     EndIf
753
754     If !Empty(aComp[_COMP_LIST])
755
756         aComp[_COMP_CTRL_TYPE] := "L"
757
758     ElseIf aComp[_COMP_FIELD_TYPE] = "L"
759         aComp[_COMP_CTRL_TYPE] := "C" //CheckBox
760     ElseIf aComp[_COMP_LEN] > 60
761
762         aComp[_COMP_CTRL_TYPE] := "M"
763     EndIf

```

```

764
765         If !Empty(aComp[_COMP_F3]) .AND. Empty(aComp[_COMP_DEFAULT])
766
767             aComp[_COMP_DEFAULT] := space(aComp[_COMP_LEN])
768         EndIf
769
770     Else
771
772         aComp[_COMP_CTRL_TYPE] := "R"
773     EndIf
774
775     If Empty(aComp[_COMP_TITLE])
776
777         aComp[_COMP_TITLE] := "[" + AllTrim(cCompl)+"]"
778     Else
779
780         aComp[_COMP_TITLE] := AllTrim(aComp[_COMP_TITLE]) + space(1) + "[" + AllTrim(cCompl)+"]"
781     EndIf
782
783 EndIf
784
785 Return aComp
786
787
788 //-----
789 /*/{Protheus.doc} STFSetNodeEdited
790 Muda o estado do nó para editado
791 @param lEdit      Nó Editado?
792 @param aDadosL    Nó de Posicionamento
793 @return lValid    Se Valido, retorna True
794 @obs
795 @sample
796 */
797 //-----
798 Function STFSetNodeEdited(lEdit, aDadosL)
799     DEFAULT aDadosL := {}
800     DEFAULT lEdit := .T.    //No editado?
801
802     If Len(aDadosL) >= _EDITED_NODE
803         aDadosL[_EDITED_NODE] := lEdit
804     EndIf
805
806 Return lEdit
807
808
809 //-----
810 /*/{Protheus.doc} ValidNode
811 Muda o estado do nó para editado
812 @param aDadosL    Nó de Posicionamento
813 @param cLog        Resultado da Validação
814 @param cTitulo     Título do No
815 @param oTree       Objeto da arvore

```

```

816 @param aDados   Array com os nós
817 @param lRetFatherNode Retorna os nós pais?
818 @return lValid   Se Valido, retorna True
819 @obs
820 @sample
821 /**
822 //-----
823 Static Function ValidNode(aDadosL, cLog, cTitulo, oTree, aDados, lRetFatherNode)
824 Local lValid := .T. //Nó valido
825 Local nC     := 0           //Variável contadora
826 Local nCampos := Len(aDadosL[_COMPS_NODE]) //Quantidade de linhas
827 Local bValid := NIL        //Bloco de validação
828 Local cLog2  := ""         //Log
829 Local nNivel := 0           //Nível do nó
830 Local cEst   := cLG_CODIGO
831 Local cLgTmp := "" //Variável temporária de Log
832
833
834 DEFAULT aDadosL := {}
835 DEFAULT cLog    := ""
836 DEFAULT cTitulo := ""
837 DEFAULT oTree   := NIL
838 DEFAULT aDados  := {}
839 DEFAULT lRetFatherNode := .T. //Retorna os nós pais
840
841 If ValType(oTree) == "0"
842     nNivel := oTree:Nivel()           //Nível do nó
843 EndIf
844
845 If nCampos > 0
846
847     If aDadosL[_EDITED_NODE] //No alterado, então valida
848         //cRIA TODAS AS VARIÁVEIS
849         aEval( aDadosL[_COMPS_NODE], { |aComp| IIF(aComp[_COMP_TYPE] <> _ROTINA, &("M->" + aComp[_COMP_NAME]) := aComp[_COMP_DEFAULT],) } )
850
851         For nC := 1 to nCampos
852             If !Empty(bValid := aDadosL[_COMPS_NODE][nC, _COMP_VALID]) .AND. ;
853                 ValType(bValid) == "B"
854                 cLgTmp := ""
855                 If !Eval(bValid, aDadosL[_COMPS_NODE][nC, _COMP_DEFAULT])
856                     cLog += CRLF + space(nNivel*5) + "- " + aDadosL[_COMPS_NODE][nC, _COMP_TITLE] +;
857                         STR0017 + cValToChar(aDadosL[_COMPS_NODE][nC, _COMP_DEFAULT]) + "]" + cLgTmp //" Valor incorreto ["
858                     lValid := .F.
859                 Else
860                     cLog += CRLF + space(nNivel*5) + "- " + aDadosL[_COMPS_NODE][nC, _COMP_TITLE] +;
861                         STR0018 + cValToChar(aDadosL[_COMPS_NODE][nC, _COMP_DEFAULT]) + "]" + cLgTmp //" Valor correto ["
862                 EndIf
863             Else
864                 cLog += CRLF + space(nNivel*5) + "- " + aDadosL[_COMPS_NODE][nC, _COMP_TITLE] +;
865                     STR0019 + cValToChar(aDadosL[_COMPS_NODE][nC, _COMP_DEFAULT]) + "]" //" Não existe validação para este componente.
866             Conteudo["

```

```

867
868         EndIf
869
870     Next
871
872     aEval( aDadosL[_COMPS_NODE], { |aComp| IIF(aComp[_COMP_TYPE] <> _ROTINA, &("M->" + aComp[_COMP_NAME]) := NIL,) } )
873
874     If !Empty(aDadosL[_VALID_FUNCTION])
875         bValid := TranslateParameters(aDadosL[_VALID_FUNCTION])
876         lValid := Eval( &("{ || " + bValid + "}") ) .AND. lValid
877         cLog := cLog + CRLF + cLog2
878     EndIf
879 Else
880     cLog := CRLF + space(nNivel*5) + STR0020 //"- Componentes deste nó não foram editados, portanto não serão validados"
881
882 EndIf
883
884 If lRetFatherNode
885     cLog := RetFatherNodes(aDadosL[_CODE_NODE], aDados, nNivel) + cLog
886 Else
887     cLog := space(5*(nNivel-1)) + cTitulo + cLog
888 EndIf
889
890 If aDadosL[_EDITED_NODE]
891     If !lValid
892         oTree:ChangeBmp(IF(!aDadosL[_IS_FATHER], "CANCEL", "FOLDER7"), IF(!aDadosL[_IS_FATHER], "CANCEL", "FOLDER8"),,, aDadosL[_CODE_NODE] )
893     Else
894         //Muda o controle para ok
895         oTree:ChangeBmp(IF(!aDadosL[_IS_FATHER], "PCOFX0K", "FOLDER10") , IF(!aDadosL[_IS_FATHER], "PCOFX0K", "FOLDER11"),,, aDadosL[_CODE_NODE])
896     EndIf
897     aDadosL[_VALID_NODE] := lValid
898 EndIf
899 //REALIZAR A VALIDAÇÃO DE NÓS PREDECESSORES
900
901 EndIf
902
903 Return lValid
904
905
906 //-----
907 /*{Protheus.doc} RetCompType
908 Função para retornar a descrição do tipo do componente
909 @param cType      Tipo do componente
910 @return cType      Descrição do Componente
911 @obs
912 @sample
913 */
914 //-----
915 Static Function RetCompType(cType)
916 Local nPos := 0      //Posicao do array
917 Local aTypes := { { _PARAMETRO, STR0021},; //"Parâmetro"
918                  { _ARQUIVO_INI, STR0022},; //"Arquivo de Configurações"

```

```

919         {_CAMPO, STR0023},; //"Campo"
920         {_ROTINA, STR0024}} //"Rotina"
921
922     DEFAULT cType := ""
923     If (nPos := aScan(aTypes, {|c| c[1] == cType})) > 0
924         cType := aTypes[nPos, 2]
925     EndIf
926
927 Return cType
928
929 //-----
930 /*{Protheus.doc} TransLateParameters
931 Função para traduzir os parâmetros
932 @param cFunction    Função a ser traduzida
933 @return cFunction    Função traduzida
934 @obs
935 @sample
936 */
937 //-----
938 Static Function TransLateParameters(cFunction)
939
940     DEFAULT cFunction := ""
941
942     cFunction := StrTran(cFunction, "__NODE_CONTROLS","aControls")
943     cFunction := StrTran(cFunction, "__NODE_DATA","aDadosL")
944     cFunction := StrTran(cFunction, "__NODE_LOG","@cLog2")
945
946 Return cFunction
947
948
949 //-----
950 /*{Protheus.doc} ReturnKey
951 Função para retornar as chaves da tabela
952 @param uCompKey    Valor da Chave
953 @param cKey        Composição da chave
954 @return aRet       Array bidimensional contendo o nome e conteúdo do campo
955 @obs
956 @sample
957 */
958 //-----
959 Static Function ReturnKey(uCompKey, cKey)
960     Local aCampos := {}
961     Local nC := 0
962     Local nPos := 0
963     Local aRet := {}
964     Local nCampos := 0
965
966     DEFAULT uCompKey := NIL
967     DEFAULT cKey := ""
968
969     aCampos := StrTokArr(cKey, "+")
970     nCampos := Len(aCampos)

```

```

971     For nC := 1 To nCampos
972
973         If (nPos := At(aCampos[nC], "(") > 0 //Existe função?
974             aCampos[nC] := AllTrim(Left(aCampos[nC], nPos-1))
975             If (nPos := At(aCampos[nC], ",") > 0 .OR. (nPos := At(aCampos[nC], ")") > 0
976                 aCampos[nC] := AllTrim(Left(aCampos[nC], nPos-1)))
977
978             EndIf
979         EndIf
980     If FieldPos(aCampos[nC]) == 0
981         aRet := NIL
982         Exit
983     EndIf
984
985     //DÁ um sustrs baseado na chave
986     If ValType(uCompKey) <> "C" .OR. nC == nCampos
987         aAdd(aRet, {aCampos[nC], uCompKey})
988     Else
989         nPos := TamSx3(aCampos[nC])[1]
990         aAdd(aRet, {aCampos[nC], Left(uCompKey, nPos)})
991         uCompKey := Substr(uCompKey, nPos+1)
992     EndIf
993 Next
994
995 Return aRet
996
997 //-----
998 /*{Protheus.doc} ReleaseChildren
999 Função para Liberar os componentes do nó, incluindo seus filhos
1000 @param cIdAmb      Código do Ambiente
1001 @param cCodeNode   Nó
1002 @param aDados      Array dos nós
1003 @param oTree       Arvore
1004 @param lSorted     Ordenação
1005 @return NIL
1006 @obs
1007 @sample
1008 */
1009 //-----
1010 Static Function ReleaseChildren(cIdAmb, cCodeNode, aDados, oTree, lSorted)
1011     Local nPos := 0 //Posicao do nó
1012     Local nTamDados := 0 //Tamanho do array
1013
1014     Local lSortedMain := .F. //ordenado na principal
1015
1016     DEFAULT cIdAmb := ""
1017     DEFAULT cCodeNode := ""
1018     DEFAULT aDados := {}
1019     DEFAULT oTree := NIL
1020     DEFAULT lSorted := .F. //Array não ordenado
1021
1022

```

```

1023     nPos := aScan(aDados, {|x| x[_CODE_NODE] == cCodeNode}) //Posicao do nó
1024     nTamDados := Len(aDados) //Tamanho do array
1025
1026     If nPos > 0
1027         If !aDados[nPos, _IS_FATHER]
1028             STFSetNodeEdited( .F., @aDados[nPos])
1029             aDados[nPos, _VALID_NODE] := .T.
1030             oTree:ChangeBmp("SUMARIO", "SUMARIO",,,cCodeNode )
1031             // aEval( aDados[nPos, COMPS_NODE], { |aDadosL| IIF(aDadosL[_COMP_TYPE] <> _ROTINA, &("M->" + AllTrim(aDadosL[_COMP_NAME])) := NIL ,)})
1032             aDados[nPos, _COMPS_NODE] := {}
1033
1034         Else
1035             //Ordena pelo pai
1036             If !lSorted
1037                 aSort( aDados,,, {|x, y| x[_FATHER_NODE] + Str(x[_ORDER_NODE]) + x[_CODE_NODE] < y[_FATHER_NODE] + Str(y[_ORDER_NODE]) + y[_CODE_NODE]})
1038                 lSorted := .T.
1039                 lSortedMain := .T.
1040             EndIf
1041             nPos := aScan(aDados, {|x| x[_FATHER_NODE] == cCodeNode })
1042             While nPos > 0 .AND. nPos <= nTamDados .AND. aDados[nPos, _FATHER_NODE] == cCodeNode
1043                 If Empty(aDados[nPos, _ENVIRONMENT_NODE]) .OR. cIdAmb $ aDados[nPos, _ENVIRONMENT_NODE]
1044                     ReleaseChildren(cIdAmb, aDados[nPos, _CODE_NODE], @aDados, @oTree, lSorted)
1045                 EndIf
1046                 nPos++
1047             EndDo
1048             //Volta a ordem para os filhos
1049             If lSortedMain
1050                 aSort( aDados,,, {|x, y| x[_CODE_NODE] < y[_CODE_NODE]}) //Volta a ordem para os filhos
1051                 nPos := aScan(aDados, {|x| x[_CODE_NODE] == cCodeNode})
1052                 oTree:ChangeBmp("FOLDER5", "FOLDER6",,,cCodeNode )
1053                 aDados[nPos, _COMPS_NODE] := {}
1054             EndIf
1055         EndIf
1056     EndIf
1057 Return
1058
1059 //-----
1060 /*{Protheus.doc} SearchNode
1061 Função para busca do Nó
1062
1063 @param cSearchNode Código do Ambiente
1064 @param oTree Árvore
1065 @param aDados Array dos nós
1066 @param cIdAmbiente Código do Ambiente
1067 @param oPanel Objeto Panel
1068 @param oDlg Janela
1069 @param cLog Log
1070 @return NIL
1071 @obs
1072 @sample
1073 */
1074 //-----

```

```

1075 Static Function SearchNode(cSearchNode, oTree, aDados, cIdAmbiente, oPanel, oDlg, cLog)
1076 Local aArea := GetArea()
1077 Local aAreaMF7 := MF7->(GetArea())
1078
1079     DEFAULT cSearchNode := ""
1080     DEFAULT oTree       := NIL
1081     DEFAULT aDados      := {}
1082     DEFAULT cIdAmbiente := ""
1083     DEFAULT oPanel      := NIL
1084     DEFAULT oDlg        := NIL
1085     DEFAULT cLog        := ""
1086
1087
1088     MF7->(DbSetFilter( { || MF7_FILIAL == xFilial() .AND. (Empty(MF7_ENVIRO) .OR. cIdAmbiente $ MF7_ENVIRO) .AND. AllTrim(Upper(cSearchNode)) $
AllTrim(Upper(FwNoAccent(MF7_DESCR)))}, "MF7_FILIAL == xFilial() .AND. (Empty(MF7_ENVIRO) .OR. cIdAmbiente $ MF7_ENVIRO) .AND. AllTrim(Upper(cSearchNode)) $
AllTrim(Upper(FwNoAccent(MF7_DESCR)))") )
MF7->(DbGoTop())
1089
1090
1091     If MF7->(!Eof()) .AND. oTree:GetCargo() <> MF7->MF7_NODE
1092
1093         oTree:TreeSeek("")
1094         cNode := MF7->MF7_NODE
1095         If oTree:TreeSeek(MF7->MF7_NODE) //Posiciona no topo e procura
1096             ActionClick(oTree:GetCargo(), oTree, @aDados, cIdAmbiente, @oPanel, @oDlg, @cLog)
1097         EndIf
1098     EndIf
1099
1100     MF7->(DbClearFilter())
1101     RestArea(aAreaMF7)
1102     RestArea(aArea)
1103
1104 Return
1105
1106
1107 //-----
1108 /*/{Protheus.doc} BuildList
1109 Função para Reconstrução do nó
1110 @param cSearchNode  Código do Ambiente
1111 @param oTree        Objeto da árvore
1112 @param aDados       Itens da árvore
1113 @param cIdAmbiente  Codigo do Ambiente editado
1114 @param nTamNode     Tamanho do nó
1115 @param oDlg         Janela
1116 @param oPanel       Painei
1117 @param cLog         Variavel de Log
1118 @param cLG_CODIGO   Codigo da estação alterado
1119 @param aCoörd       Array de coordenadas da Tree
1120 @param nTamSX3      Tamanho do Campo descrição do nó
1121 @return  NIL
1122 @obs
1123 @sample
1124 */

```

```

1125 //-----
1126 Static Function BuildList(oTree, aDados, cIdAmbiente, nTamNode, oDlg, oPanel, cLog, cLG_CODIGO, aCoord, nTamSX3)
1127
1128     DEFAULT oTree := NIL
1129     DEFAULT cIdAmbiente := ""
1130     DEFAULT nTamNode := 0
1131     DEFAULT oDlg := NIL
1132     DEFAULT oPanel:= NIL
1133     DEFAULT cLog := ""
1134     DEFAULT cLG_CODIGO := ""
1135     DEFAULT aCoord := {}
1136     DEFAULT nTamSX3 := 0
1137
1138
1139     If cIdAmb <> cIdAmbiente .OR. cEsta <> cLG_CODIGO
1140         aSort( aDados,,, {|x, y| x[_FATHER_NODE] + Str(x[_ORDER_NODE]) + x[_CODE_NODE] < y[_FATHER_NODE] + Str(y[_ORDER_NODE]) + y[_CODE_NODE]})
1141
1142         If cIdAmb <> cIdAmbiente
1143             aDados := {}
1144             LoadData(@aDados, cIdAmbiente)
1145         EndIf
1146         BuildTree(@oTree, @aDados, cIdAmbiente, @oPanel, oDlg, @cLog, aCoord, nTamSX3)
1147
1148         If Len(aDados) > 1 .AND. oTree:TreeSeek(aDados[01, _CODE_NODE] )
1149
1150             If cEsta == cLG_CODIGO
1151                 cIdAmb := cIdAmbiente
1152                 ActionClick(oTree:GetCargo(), oTree, @aDados, cIdAmbiente, @oPanel, @oDlg, @cLog)
1153             Else
1154                 cEsta := cLG_CODIGO
1155                 ActionClick(oTree:GetCargo(), oTree, @aDados, cIdAmbiente, @oPanel, @oDlg, @cLog)
1156             EndIf
1157
1158         EndIf
1159     EndIf
1160
1161     Return
1162
1163
1164 //-----
1165 /*{Protheus.doc} ExibeLog
1166 Função para Exibição de LOG
1167 @param cCargo Id do No
1168 @param oTree Objeto da Arvore
1169 @param aDadosL //Nó atual
1170 @param oPanel Objeto Panel
1171 @param cIdAmb Código do Ambiente
1172 @param aControls Array de Controles
1173 @param cLog Log
1174 @param aDados Nos da arvore
1175 @return NIL
1176 @obs

```

```

1177 @sample
1178 /*/
1179 //-----
1180 Static Function ExibeLog(cCargo, oTree, aDadosL, oPanel, cIdAmb, aControls, cLog, aDados)
1181 Local nRow      := 0           //Linha do Log
1182 Local nCol      := 0           //coluna do Log
1183 Local aObjects  := {}          //Objetos
1184 Local aPosObj2  := {}          //Coordenadas do Objeto
1185 Local nTam      := 0           //Tamanho
1186 Local cTitulo   := ""          //titulo da Janela
1187 Local nRow1     := 0           //Linha do painel 2
1188 Local oPanel2   := NIL         //Painel 2
1189 Local oBtn1     := NIL         //botão 1
1190 Local oBtn2     := NIL         //Botão 2
1191 Local oBtn3     := NIL         //botão 3
1192 Local cFile     := ""          //Arquivo de Gravação
1193
1194 DEFAULT cCargo := ""
1195 DEFAULT oTree  := NIL
1196 DEFAULT aDadosL := {}
1197 DEFAULT oPanel := NIL
1198 DEFAULT cIdAmb := ""
1199 DEFAULT aControls := {}
1200 DEFAULT cLog     := ""
1201 DEFAULT aDados   := {}
1202
1203 If Valtype(oPanel) == "0"
1204     nRow      := Int((oPanel:nHeight ) / 2) //Linha do Log
1205     nCol      := Int((oPanel:nWidth  ) / 2) //coluna do Log
1206     nRow1     := nRow-29                    //Linha do painel 2
1207
1208 EndIf
1209
1210 If Len(aDadosL) >= _DESCRIPTION_NODE
1211     cTitulo := STR0025 +aDadosL[_DESCRIPTION_NODE] //"Log de Validação - "
1212 EndIf
1213
1214
1215 @ 0, 0 TO nRow1, nCol OF oPanel PIXEL
1216 oPanel2 := TScrollBox():New( oPanel, 0,0,nRow1,nCol)
1217 nTam := CalcFieldSize("C",Len(cTitulo),0, "@!")
1218
1219 @05, ( nCol/2 - (nTam/2) ) SAY Upper(cTitulo) PIXEL SIZE nTam, 10 OF oPanel2
1220
1221
1222 aObjects := { {CalcFieldSize("C",110,0),nRow1-5, .F., .F.} }
1223 aPosObj2 := MsObjSize( { 0 , 15 , nRow1 ,nCol, 2, 3,3,3}, aObjects, .T., .F.)
1224
1225
1226 TMultiGet():New(aPosObj2[1, 1],aPosObj2[1, 2],{|| Iif(PCOUNT() > 0, cLog := U,cLog)} ,oPanel2,,
1227     aObjects[1, 01],aObjects[1, 02],/*oFont*/,.T.,;
1228     /*nClrFore*/,/*nClrBack*/,/*oCursor*/,.T.,;

```

```

1229             /*cMg*/,.T.,,/*lCenter*/;;
1230             /*lRight*/,.T.,,/*bChange*/;;
1231             ,.F.,.T.)
1232
1233
1234     @nRow1+3, 0 TO nRow, nCol 0F oPanel PIXEL
1235
1236     oBtn1 := TButton():New( nRow1+8, 05, STR0013,oPanel,{|| (cFile:=cGetFile(STR0026,""),If(cFile="",.t.,MemoWrite(cFile,cLog))) },55,15,,,F.,.T.,.F.,,F.,{ ||
aDadosL[_EDITED_NODE] },,.F. ) //"Gravar"###"Arquivos Texto (*.TXT) |*.txt|"
1237     If !aDadosL[_EDITED_NODE]
1238         oBtn1:Disable()
1239     EndIf
1240     oBtn2 := TButton():New( nRow1+8, 05+60, STR0027,oPanel,{|| SendMail(cTitulo, cLog) },55,15,,,F.,.T.,.F.,,F.,{ || aDadosL[_EDITED_NODE] },,.F. ) //"Enviar e-mail"
1241     oBtn3 := TButton():New( nRow1+8, 05+120, STR0011,oPanel,{|| MontaPanel(cCargo, oTree, aDadosL, oPanel, cIdAmb, {}, aDados, cLog) },55,15,,,F.,.T.,.F.,,F.,{ ||
!aDadosL[_VALID_NODE] },,.F. ) //"Fechar"
1242
1243 Return
1244
1245 //-----
1246 /*{Protheus.doc} RetFatherNodes
1247 Função para retornar os nós Pais
1248 @param cCargo Código do No
1249 @param aDados Nos da arvore
1250 @param nNivel Nivel da Arvore
1251 @return cRetorno Código do No Pai
1252 @obs
1253 @sample
1254 */
1255 //-----
1256 Static Function RetFatherNodes(cCargo,aDados, nNivel)
1257     Local nPos := 0 //Posicao do nó pai
1258     Local cRetorno := ""
1259                                     //Retorno da função
1260     DEFAULT cCargo := ""
1261     DEFAULT aDados := {}
1262     DEFAULT nNivel := 1
1263
1264     nPos := aScan(aDados, {|l| l[_CODE_NODE] == cCargo})
1265
1266     If nPos > 0
1267         nNivel := nNivel -1
1268
1269         cRetorno := space(nNivel*5) + aDados[nPos,_DESCRIPTION_NODE]
1270
1271         If !Empty(aDados[nPos,_FATHER_NODE])
1272             cRetorno := RetFatherNodes(aDados[nPos,_FATHER_NODE], aDados, nNivel) + CRLF + cRetorno
1273         EndIf
1274     EndIf
1275
1276 Return cRetorno
1277
1278

```

```

1279 //-----
1280 /*{Protheus.doc} ShowMainLog
1281 Função para retornar os Log da Janela Principal
1282 @param cCargo Código do No
1283 @param oTree Objeto Arvore
1284 @param oPanel Objeto Panel
1285 @param cIdAmb Código do Ambiente
1286 @param cLog Código do Log
1287 @param aDados Array de Dados
1288 @return NIL
1289 @obs
1290 @sample
1291 */
1292 //-----
1293 Static Function ShowMainLog(cCargo, oTree, oPanel, cIdAmb, cLog, aDados)
1294
1295     Local nPos := 0 //Posicao do Log
1296
1297     DEFAULT cCargo := ""
1298     DEFAULT oTree := NIL
1299     DEFAULT oPanel := NIL
1300     DEFAULT cIdAmb := ""
1301     DEFAULT cLog := ""
1302     DEFAULT aDados := {}
1303
1304     If (nPos := aScan(aDados, {|l| l[_CODE_NODE] == cCargo})) > 0
1305
1306         ExibeLog(cCargo, oTree, aDados[nPos], oPanel, cIdAmb, {}, @cLog, aDados)
1307     EndIf
1308
1309 Return
1310
1311 //-----
1312 /*{Protheus.doc} SendMail
1313 Função para enviar e-mail do log de validação
1314 @param cAssunto Assunto e-mail
1315 @param cMsg Mensagem do e-mail
1316 @return lRet E-mail enviado com sucesso
1317 @obs
1318 @sample
1319 */
1320 //-----
1321 Static Function SendMail(cAssunto, cMsg)
1322 Local oMail := NIL //Objeto do Service de e-mail
1323 Local oMessage := NIL //Mensagem
1324 Local nErro := 0 //Erro
1325 Local lRet := .T. //Retorno de execução
1326 Local cSMTPServer := GetMV("MV_RELSESV", "smtp.servername.com.br") //SMTP Server
1327 Local cSMTPUser := GetMV("MV_RELAUSR", "minhaconta@servername.com.br") //Usuário de autenticação
1328 Local cSMTPPass := GetMV("MV_RELAPSW", "minhasenha") //Senha
1329 Local cMailFrom := GetMV("MV_RELFROM", "minhaconta@servername.com.br") //Remetente
1330 Local nPort := GetMV("MV_GCPPORT", 25) //Porta

```

```

1331 Local lUseAuth      := GetMV("MV_RELAUTH",.T.)           //Autentica?
1332 Local cPara         := GetMV("MV_LJEMLAD",,"minhaconta@servername.com.br") //Destinatário
1333 Local lTLS          := GetMV("MV_RELTLS ",.F.,.F.)       //LTS?
1334 Local nTOUIT        := GetMV("MV_RELTIME",.F.,120)       //Time-Out
1335
1336 DEFAULT cAssunto := ""
1337 DEFAULT cMsg := ""
1338
1339 cMsg := StrTran(StrTran(StrTran(StrTran(StrTran(StrTran(cMsg, ";"), "&", "&"), " ", "&nbsp;"), ">", ">"), "<", "<"), CRLF, "<BR>")
1340
1341 If !lTLS
1342     //MailSMTPOn
1343     CONNECT SMTP SERVER cSMTPServer ACCOUNT cSMTPUser PASSWORD cSMTPPass RESULT lRet
1344     If lRet
1345         SEND MAIL FROM cMailFrom ;
1346         TO cPara ;
1347         SUBJECT cAssunto ;
1348         BODY cMsg;
1349         ATTACHMENT ;
1350         RESULT lRet
1351     If !lRet
1352         GET MAIL ERROR cMailError
1353         ConOut(STR0028 + RTrim(cMailError)) //"Erro no envio do e-mail "
1354     EndIf
1355     DISCONNECT SMTP SERVER
1356 Else
1357     GET MAIL ERROR cMailError
1358     ConOut(STR0029 + RTrim(cErro)) //"Erro na conexão:"
1359 EndIf
1360
1361 Else
1362
1363     oMail := TMailManager():New()
1364     oMail:SetUseTLS(.T.)
1365     oMail:Init( '', cSMTPServer , cSMTPUser, cSMTPPass, 0, nPort )
1366     oMail:SetSmtpTimeOut( nTOUIT )
1367     nErro := oMail:SmtpConnect()
1368
1369     conout(STR0045+str(nErro,6)) //"Status de Retorno = "
1370
1371 If lUseAuth
1372     nErro := oMail:SmtpAuth(cSMTPUser ,cSMTPPass)
1373     If nErro <> 0
1374         // Recupera erro ...
1375         cMailError := oMail:GetErrorString(nErro)
1376         DEFAULT cMailError := '***UNKNOWN***'
1377         Conout(STR0030+str(nErro,4)+' ('+cMailError+')') //"Erro de Autenticacao "
1378         lRet := .F.
1379     Endif
1380 Endif
1381
1382 if nErro <> 0

```

```

1383
1384         // Recupera erro
1385         cMailError := oMail:GetErrorString(nErro)
1386         DEFAULT cMailError := '***UNKNOW***'
1387         conout(cMailError)
1388
1389         Conout(STR0031+str(nErro,4)) //"Erro de Conexão SMTP "
1390         oMail:SMTPDisconnect()
1391
1392         lRet := .F.
1393
1394     Endif
1395
1396     If lRet
1397         oMessage := TMailMessage():New()
1398         oMessage:Clear()
1399         oMessage:cFrom := cMailFrom
1400         oMessage:cTo := cPara
1401         oMessage:cSubject := cAssunto
1402         oMessage:cBody := cMsg
1403         nErro := oMessage:Send( oMail )
1404
1405         if nErro <> 0
1406             xError := oMail:GetErrorString(nErro)
1407             Conout(STR0032+str(nErro,4)+" (" +xError+")") //"Erro de Envio SMTP "
1408             lRet := .F.
1409         Endif
1410
1411         oMail:SMTPDisconnect()
1412         FreeObj(oMessage)
1413     Endif
1414
1415     FreeObj(oMail)
1416
1417 EndIf
1418
1419 Return lRet
1420
1421
1422 //-----
1423 /*{Protheus.doc} ValidAllNodes
1424 Função para validar todos os nós
1425 @param cIdAmb Código do Ambiente
1426 @param cCargo Código do No
1427 @param oTree Objeto Arvore
1428 @param aDados Array dos nós
1429 @param lRetFather Retorna a descrição dos nós pais
1430 @param aDadosF Array dos nós ordenados pelo pai
1431 @param lValid No valido?
1432 @return cRet Retorno da descrição do NO
1433 @obs
1434 @sample

```

```

1435 /*/
1436 //-----
1437 Static Function ValidAllNodes(cIdAmb, cCargo, oTree, aDados, lRetFather, aDadosF, lValid)
1438     Local cTitulo := ""           //título do no
1439     Local cRetorno := ""          //Retorno da funcao
1440     Local nPos2 := 0              //Posicao dos filhos
1441     Local nTamDados := Len(aDados) //Tamanho dos Dados
1442     Local lValChild := .T.        //Filhos Válidos
1443     Local nNivel := 0             //Nível do No
1444     Local cRet := ""              //Retorno
1445     Local nPos := 0               //Posico
1446
1447     DEFAULT cIdAmb := ""
1448     DEFAULT cCargo := ""
1449     DEFAULT oTree := NIL
1450     DEFAULT aDados := {}
1451     DEFAULT lRetFather := .T.
1452     DEFAULT aDadosF := {}
1453     DEFAULT lValid := .T.
1454
1455     nPos := aScan(aDados, { | x| x[_CODE_NODE] == cCargo })
1456     cTitulo := aDados[nPos, _DESCRIPTION_NODE]
1457
1458     If Len(aDados[nPos, _COMPS_NODE]) = 0
1459         //Carrega os componentes
1460         aDados[nPos, _COMPS_NODE] := LoadComp(cCargo, cIdAmb)
1461     EndIf
1462
1463     If aDados[nPos, _IS_FATHER]
1464         //Retorna os filhos
1465
1466         If lRetFather
1467             aDadosF := aClone(aDados)
1468             aSort( aDadosF,,, {|x, y| x[_FATHER_NODE] + Str(x[_ORDER_NODE]) + x[_CODE_NODE] < y[_FATHER_NODE] + Str(y[_ORDER_NODE]) + y[_CODE_NODE]})
1469         EndIf
1470
1471
1472         If (nPos2 := aScan(aDadosF,{ |x| x[_FATHER_NODE] == cCargo})) > 0
1473             Do While nPos2 <= nTamDados .AND. aDadosF[nPos2, _FATHER_NODE] == cCargo
1474                 If Empty(aDadosF[nPos2, _ENVIRONMENT_NODE]) .OR. cIdAmb $ aDadosF[nPos2, _ENVIRONMENT_NODE]
1475                     cRetorno := ValidAllNodes( cIdAmb, aDadosF[nPos2, _CODE_NODE], oTree, @aDados, .F., aDadosF, @lValid) + CRLF + cRetorno
1476
1477                     lValChild := lValChild .AND. lValid
1478                 EndIf
1479                 nPos2++
1480             EndDo
1481
1482         EndIf
1483         oTree:TreeSeek(cCargo)
1484         cTitulo := oTree:GetPrompt(.T.)
1485
1486         ValidNode(@aDados[nPos] , @cRet, cTitulo, oTree, aDados, lRetFather)

```

```

1487
1488     If !lValChild
1489         aDados[nPos,_VALID_NODE] := .F.
1490     EndIf
1491     cRetorno := cRet + CRLF + cRetorno
1492
1493 Else
1494
1495     oTree:TreeSeek(cCargo)
1496     cTitulo := oTree:GetPrompt(.T.)
1497     ValidNode(@aDados[nPos], @cRet, cTitulo, oTree, aDados, lRetFather)
1498     cRetorno := cRet + cRetorno
1499 EndIf
1500
1501 nNivel := oTree:Nivel()
1502
1503 lValid := aDados[nPos,_VALID_NODE]
1504 cRet := cRetorno
1505
1506 Return cRet
1507
1508 //-----
1509 /*{Protheus.doc} IniStaticVar
1510 Função para iniciar as variáveis estáticas
1511 @return NIL
1512 @obs
1513 @sample
1514 */
1515 //-----
1516 Static Function IniStaticVar()
1517
1518     cFil3 := FWxFilial("SX6",cFilAnt, "C", "E", "E") //Nivel 3
1519     cFil2 := FWxFilial("SX6",cFilAnt, "C", "C", "E")// Nivel 2
1520     cFil1 := FWxFilial("SX6",cFilant, "C", "C", "C") //Nivel 1
1521     cLG_CODIGO := SLG->(CriaVar("LG_CODIGO") )
1522     cLG_CODIGO := cEstacao := SLG->(DbGotop(), LG_CODIGO) //Alterar
1523     cIdAmb := IIF(nModulo == 12, "1", "3")
1524
1525     cNodeAnt := ""
1526     oCfgTef := NIL
1527
1528 Return
1529
1530
1531 //-----
1532 /*{Protheus.doc} ReleaseStaticVar
1533 Função para retornar os nós Pais
1534 @return NIL
1535 @obs
1536 @sample
1537 */
1538 //-----

```

```

1539 Static Function ReleaseStaticVar()
1540 //Libera as variáveis de memória
1541     cFil3     := NIL
1542     cFil2     := NIL
1543     cFil1     := NIL
1544     cLG_CODIGO := NIL
1545     nIdAmb     := NIL
1546     cNodeAnt   := NIL
1547     cEsta     := NIL
1548     If oCfgTef <> NIL
1549         oCfgTef := FreeObj(oCfgTef)
1550     EndIf
1551 Return
1552
1553 //-----
1554 /*{Protoheus.doc} LoadData
1555 Função para retornar os nós
1556 @param aData     Array dos nós
1557 @param cIdAmbiente Codigo do Ambiente
1558 @return NIL
1559 @obs
1560 @sample
1561 */
1562 //-----
1563 Static Function LoadData(aData, cIdAmbiente)
1564 Local aArea      := GetArea()
1565 Local aAreaMF7   := MF7->(GetArea())
1566 Local nTamSx3 := MF7->(TamSx3("MF7_NODE")[1]) //To do tamanho do nó
1567
1568     DEFAULT aData := {}
1569     DEFAULT cIdAmbiente := ""
1570
1571
1572     MF7->(DbSetFilter({|| MF7_FILIAL == xFilial() .AND. (Empty(MF7_ENVIRO) .OR. cIdAmbiente $ MF7_ENVIRO) }, "MF7_FILIAL == xFilial() .AND. (Empty(MF7_ENVIRO) .OR. cIdAmbiente
    $ MF7_ENVIRO)" ))
1573     MF7->(DbGoTop())
1574     MF7->(DbSetOrder(2)) //MF7_FILIAL+MF7_FATHER+STR(MF7_ORDER,4)+ MF7_NODE
1575
1576
1577 Do While MF7->(!Eof())
1578     aAdd(aData, {MF7->MF7_FATHER,; //1
1579                 MF7->MF7_NODE,; //2
1580                 MF7->MF7_DESCR,; //3
1581                 MF7->MF7_ORDER,; //4
1582                 MF7->MF7_ENVIRO,; //5
1583                 .T.,; //6
1584                 .F.,; //7
1585                 MF7->MF7_VALID,; //8
1586                 ,; //9
1587                 ,; //10
1588                 MF7->MF7_TABREC,; //11
1589                 MF7->MF7_TABINS,; //12

```

```

1590             MF7->MF7_POSREC,; //13
1591             MF7->MF7_FUNREC,; //14
1592             {} } ) //15
1593         MF7->(DbSkip())
1594     EndDo
1595
1596     MF7->(DbClearFilter())
1597
1598     RestArea(aAreaMF7)
1599     RestArea(aArea)
1600
1601 Return
1602
1603 //-----
1604 /*{Protheus.doc} WriteControls
1605 Função para gravar os controls do NO
1606 @param aDadosL Dados do No
1607 @param cLog      Log
1608 @param cTitulo  Titulo do No
1609 @param oTree    Objeto da arvore
1610 @param aDados   Array dos nos
1611 @return  NIL
1612 @obs
1613 @sample
1614 */
1615 //-----
1616 Static Function WriteControls(aDadosL, cLog, cTitulo, oTree, aDados)
1617     Local lValidNode := .T.      //Nó valido?
1618     Local nC := 0                //Variável
1619     Local nTam := 0              //Tamano dos controles
1620     Local lWrite := .T.          //Gravou o controle
1621     Local aArea := GetArea()     //Workarea anterior
1622     Local cIdAmb := cIdAmb        //Codigo do Ambiente - Variável estática não e visualizada pelo eval
1623     Local cLG_CODIGO := cLG_CODIGO //Codigo da Estação - Variável estática não e visualizada pelo eval
1624
1625     DEFAULT aDadosL := {}
1626     DEFAULT cLog := ""
1627     DEFAULT cTitulo := ""
1628     DEFAULT oTree := NIL
1629     DEFAULT aDados := {}
1630
1631     If aDadosL[_EDITED_NODE]
1632
1633         nTam := Len(aDadosL[_COMPS_NODE] )
1634         If lValidNode := ValidNode(aDadosL, @cLog, cTitulo, oTree, aDados)
1635
1636
1637             If !Empty(aDadosL[_WRITE_FUNCTION])
1638
1639                 lWrite := Eval("&({ || " + aDadosL[_WRITE_FUNCTION]+ "}")") //Delega a função de gravação para o nó
1640                                     //Caso dê errado é responsavel pelo rollback
1641             Else

```

```

1642         BeginTran()
1643         If aDadosL[_RECORD_TABLE_ONCE] //Grava todos os controles de um nó de uma vez
1644             lWrite := WriteAllCt(@aDadosL[_COMPS_NODE],oTree,aDadosL[_INSERT_TABLE], _COMP_DEFAULT) //Insere o registro de uma tabela, caso o registro ainda não
exista
1645             Else
1646                 For nC := 1 to nTam
1647                     lWrite := lWrite .AND. WriteCtrl(aDadosL[_COMPS_NODE, nC],aDadosL[_INSERT_TABLE], _COMP_DEFAULT)
1648                     If !lWrite
1649                         Exit
1650                     EndIf
1651                 Next
1652             EndIf
1653         EndIf
1654
1655         If Empty(aDadosL[_WRITE_FUNCTION])
1656             If lWrite
1657                 EndTran()
1658             Else
1659                 DisarmTransaction()
1660             EndIf
1661         EndIf
1662
1663         If lWrite
1664
1665             aDadosL[_EDITED_NODE] := .F.
1666             nTam := nC
1667             //Grava nos dados antigos os valores editados
1668
1669
1670             aEval(aDadosL[_COMPS_NODE], {|x| x[_COMP_OLDVALUE] := x[_COMP_DEFAULT]})
1671             If !Empty(aDadosL[_POS_RECORDED])
1672                 &(aDadosL[_POS_RECORDED])
1673             EndIf
1674
1675             oTree:ChangeBmp(IF(!aDadosL[_IS_FATHER], "SUMARIO", "FOLDER5"),IF(!aDadosL[_IS_FATHER], "SUMARIO", "FOLDER6"),,, aDadosL[_CODE_NODE] )
1676
1677
1678         Else
1679             If !aDadosL[_RECORD_TABLE_ONCE]
1680                 For nC := 1 to nTam
1681                     lWrite := lWrite .AND. WriteCtrl(aDadosL[_COMPS_NODE, nC],aDadosL[_INSERT_TABLE], _COMP_OLDVALUE)
1682                     If !lWrite
1683                         Exit
1684                     EndIf
1685                 Next
1686             Else
1687                 WriteAllCt(@aDadosL[_COMPS_NODE],oTree,aDadosL[_INSERT_TABLE], _COMP_OLDVALUE)
1688             EndIf
1689             MsgStop(STR0033) //"Problemas na gravação do controle"
1690         EndIf
1691     Else
1692         MsgStop(STR0034) //"Nó inválido.Gravação não permitida."

```

```

1693         EndIf
1694     Else
1695         MsgStop(STR0035) //"Nó não editado.Gravação não realizada"
1696     EndIf
1697
1698     RestArea(aArea)
1699
1700     Return
1701
1702 //-----
1703 /*{Protheus.doc} WriteCtrl
1704 Função para gravar o controle
1705 @param aComp  Dados do Componente
1706 @param lInsert Insere novo registro na tabela?
1707 @param nCol    Coluna ser gravada [default/old - rollback]
1708 @return lRecorded - Componente gravado com sucesso
1709 @obs
1710 @sample
1711 */
1712 //-----
1713 Static Function WriteCtrl(aComp, lInsert, nCol)
1714     Local lRecorded := .T.    //Gravado
1715     Local aArea := GetArea() //Area anterior
1716
1717     DEFAULT aComp := {}
1718     DEFAULT lInsert := .F.
1719     DEFAULT nCol    := _COMP_DEFAULT
1720
1721
1722     Do Case
1723         Case aComp[_COMP_TYPE] == _PARAMETRO
1724             PutMv( AllTrim(aComp[_COMP_NAME]), aComp[nCol])
1725
1726
1727         Case aComp[_COMP_TYPE] == _ARQUIVO_INI
1728
1729             WritePProString(aComp[_COMP_KEY], aComp[_COMP_NAME], aComp[nCol], IIF( aComp[_COMP_INITYPE] == "1" , GetAdv97(), GetClientDir()+ aComp[_COMP_TABLE_FILE]) )
1730
1731         Case aComp[_COMP_TYPE] == _CAMPO
1732
1733             lRecorded := WriteTable(aComp[_COMP_TABLE_FILE], aComp[_COMP_INDEX], lInsert, aComp[_COMP_KEY], {{aComp[_COMP_NAME], aComp[nCol], aComp[_COMP_POSICA]}} )
1734         EndCase
1735
1736
1737     RestArea(aArea)
1738
1739     If lRecorded
1740         aComp[_COMP_OLDVALUE] := aComp[nCol]
1741     EndIf
1742
1743
1744     Return lRecorded

```

```

1745
1746 //-----
1747 /*{Protheus.doc} WriteAllCt
1748 Função para gravar todos os campos de uma mesma tabela, conforme configuração do nó
1749 @param aComps Componentes do No
1750 @param oTree Objeto Arvore
1751 @param lInsert Insere um registro, caso a chave não exista?
1752 @param nCol Coluna da informação a ser gravada
1753 @return lRecorded
1754 @obs
1755 @sample
1756 */
1757 //-----
1758 Static Function WriteAllCt(aComps,oTree,lInsert,nCol)
1759     Local nC := "" //Contador do Laço
1760     Local cTable := "" //Tabela
1761     Local nTam := 0 //Tamanho do Campo
1762     Local lRecorded := .T. //Gravado
1763     Local aCamposK := {} //Campos a serem gravados
1764     Local nIndex := 0 //Indice
1765     Local cKey := "" //Chave
1766     Local cCampo := "" //Campo
1767     Local nPos := 0 //Posicao do array
1768
1769     DEFAULT aComps := {}
1770     DEFAULT oTree := NIL
1771     DEFAULT lInsert := .F.
1772     DEFAULT nCol := _COMP_DEFAULT
1773
1774
1775     aSort(aComps, ,, { | x, y | x[_COMP_TYPE] + x[_COMP_TABLE_FILE] + x[_COMP_ID] + x[_COMP_NAME] + Str(x[_COMP_POSICA]) < y[_COMP_TYPE] + y[_COMP_TABLE_FILE] +
1776     y[_COMP_ID] + y[_COMP_NAME] + Str(y[_COMP_POSICA]) })
1777     nTam := Len(aComps)
1778
1779     For nC := 1 to nTam
1780         If aComps[nC, _COMP_TYPE] == _CAMPO
1781             //Bufferiza os campos da tabela
1782             If aComps[nC, _COMP_TABLE_FILE] <> cTable
1783                 If !Empty(cTable)
1784                     lRecorded := WriteTable(cTable , nIndex , lInsert, cKey, aCamposK )
1785                 EndIf
1786                 aCamposK := {}
1787                 cTable := aComps[nC, _COMP_TABLE_FILE]
1788                 nIndex := aComps[nC, _COMP_INDEX]
1789                 cKey := aComps[nC, _COMP_KEY]
1790                 cCampo := ""
1791
1792                 If cCampo <> aComps[nC, _COMP_NAME] .OR. Empty(aComps[nC, _COMP_POSICA])
1793                     aAdd(aCamposK, { aComps[nC, _COMP_NAME],aComps[nC, nCol], aComps[nC, _COMP_POSICA] } )
1794                     nPos++
1795                     cCampo := aComps[nC, _COMP_NAME]

```

```

1796         Else
1797             aCamposK[nPos, 02] += aComps[nC, nCol]
1798             aCamposK[nPos, 03] := 0
1799         EndIf
1800     Else
1801         If cCampo <> aComps[nC, _COMP_NAME] .OR. Empty(aComps[nC, _COMP_POSICA])
1802             aAdd(aCamposK, { aComps[nC, _COMP_NAME], aComps[nC, nCol], aComps[nC, _COMP_POSICA] } )
1803             nPos++
1804             cCampo := aComps[nC, _COMP_NAME]
1805         Else
1806             aCamposK[nPos, 02] += aComps[nC, nCol]
1807             aCamposK[nPos, 03] := 0
1808         EndIf
1809     EndIf
1810 Else
1811     If aComps[nC, nCol] <> NIL
1812         lRecorded := lRecorded .AND. WriteCtrl(aComps[nC], lInsert, nCol)
1813     EndIf
1814 EndIf
1815
1816 If !lRecorded
1817     Exit
1818 EndIf
1819
1820
1821 Next
1822 //ultima tabela
1823 If !Empty(cTable) .AND. lRecorded
1824     lRecorded := WriteTable(cTable, nIndex, lInsert, cKey, aCamposK )
1825 EndIf
1826
1827
1828 aSort(aComps, , , {|x,y| x[_COMP_NODE] + x[_COMP_ID] <= y[_COMP_NODE] + y[_COMP_ID] })
1829
1830 Return lRecorded
1831
1832 //-----
1833 /*{Protheus.doc} WriteTable
1834 Função para gravar a tabela
1835 @param cTable Tabela
1836 @param nIndex Indice
1837 @param lInsert Insere um registro, caso a chave não exista
1838 @param cKey cChave de busca
1839 @param aCamposK Campos a serem gravados
1840 @return lRecorded Registro gravado
1841 @obs
1842 @sample
1843 */
1844 //-----
1845 Static Function WriteTable(cTable, nIndex, lInsert, cKey, aCamposK )
1846     Local aCampos := {} //Campos a serem inseridos
1847     Local aAreaX := {} //Workarea anterior da tabela

```

```

1848 Local lRecorded := .T. //Registro inserido?
1849 Local nC := 0 //Contador
1850 Local nTam := 0 //Tamanho dos campos
1851 Local cCampo := "" //Conteudo do campo
1852
1853
1854 DEFAULT cTable := ""
1855 DEFAULT nIndex := 0
1856 DEFAULT lInsert := .F.
1857 DEFAULT cKey := ""
1858 DEFAULT aCamposK := {}
1859
1860
1861 aAreaX := (cTable)->(GetArea())
1862
1863 (cTable)->(DbSetOrder(nIndex))
1864
1865 If (cTable)->(DbSeek(&cKey))
1866 lRecorded := RecLock(cTable, .F.)
1867 Else
1868
1869 If lInsert
1870
1871 aCampos := (cTable)->(ReturnKey(&cKey, IndexKey() ) )
1872 If (nTam := Len(aCampos)) > 0
1873 If (lRecorded := RecLock(cTable, .T.))
1874 //To do: Verificar se não existem campos obrigatórios ou valid de campos
1875 For nC := 1 To nTam
1876 (cTable)->(FieldPut(FieldPos(aCampos[nC, 1]), aCampos[nC, 2]))
1877 Next
1878 If cTable == "SLG" .AND. FieldPos("LG_ISPOS") > 0
1879 (cTable)->(FieldPut(FieldPos("LG_ISPOS"), IIF(cIdAmb == "3", "1", "2")))
1880 EndIf
1881 EndIf
1882 Else
1883 lRecorded := .F.
1884 EndIf
1885 Else
1886 lRecorded := .F.
1887 EndIf
1888 EndIf
1889
1890 If lRecorded
1891 //Registro travado, pronto para gravar
1892 nTam := Len(aCamposK)
1893 For nC := 1 To nTam
1894
1895 If Empty(aCamposK[nC, 3] )
1896 (cTable)->(FieldPut(FieldPos(aCamposK[nC, 1]), aCamposK[nC, 2]))
1897 Else
1898 cCampo := (cTable)->(FieldGet(FieldPos(aCamposK[nC, 1])))
1899 If aCamposK[nC, 3] = 1

```

```

1900             cCampo := aCamposK[nC, 2] + Substr(cCampo, aCamposK[nC, 3]+1)
1901         ElseIf aCamposK[nC, 3] == Len(cCampo)
1902             cCampo := Left(cCampo,aCamposK[nC, 3]-1) + aCamposK[nC, 2]
1903         Else
1904             cCampo := Substr(cCampo, 1, aCamposK[nC, 3]-1) + aCamposK[nC, 2] + Substr(cCampo, aCamposK[nC, 3]+1)
1905         EndIf
1906
1907         (cTable)->(FieldPut(FieldPos(aCamposK[nC, 1]), cCampo))
1908     EndIf
1909
1910     Next
1911     (cTable)->(MsUnLock())
1912
1913 EndIf
1914
1915 Restarea(aAreaX)
1916
1917
1918 Return lRecorded
1919
1920 //-----
1921 /*{Protheus.doc} BuildTree
1922 Função para construir a arvore
1923 @param oTree Objeto da arvore
1924 @param aDados Nos da arvore
1925 @param cIdAmbiente Codigo do ambiente
1926 @param oPanel Objeto Panel
1927 @param oDlg Dialogo
1928 @param cLog Log
1929 @param aCoord Coordenadas do No
1930 @param nTamSX3 Tamanho do No
1931 @return NIL
1932 @obs
1933 @sample
1934 */
1935 //-----
1936 Static Function BuildTree(oTree, aDados, cIdAmbiente, oPanel, oDlg, cLog, aCoord, nTamSX3)
1937
1938     DEFAULT oTree := NIL
1939     DEFAULT aDados := {}
1940     DEFAULT oPanel := NIL
1941     DEFAULT oDlg := NIL
1942     DEFAULT cLog := ""
1943     DEFAULT aCoord := {}
1944     DEFAULT nTamSX3 := 0
1945
1946     If ValType(oTree) == "0"
1947         oTree := FreeObj(oTree)
1948     EndIf
1949
1950     If cIdAmbiente == "3" //POS
1951

```

```

1952     STFFStrategyECF()
1953     aRet := STFFireEvent( ProcName(0) ,;      // Nome do processo
1954                          "STCheckDLL",; // Nome do evento
1955                          {.T.} )
1956 EndIf
1957
1958
1959 oTree := DbTree():New(aCoord[1],aCoord[2],aCoord[3],aCoord[4],oDlg,{|| ActionClick(oTree:GetCargo(), oTree, aDados, cIdAmbiente, @oPanel, oDlg, @cLog) },,.T.)
1960      //Coordenadas do Pannel)
1961 oTree:SetScroll( 1, .t. ) //Horizontal
1962 oTree:SetScroll( 2, .T.)  //Vertical
1963
1964 oTree:BeginUpdate()
1965
1966     LoadTree(oDlg, oTree, cIdAmbiente, space(nTamSx3), aDados)
1967
1968 oTree:EndTree()
1969 oTree:EndUpdate()
1970
1971 //Ordena pelo código
1972 aSort( aDados,, {|x, y| x[_CODE_NODE] < y[_CODE_NODE]})
1973 Return
1974
1975 //-----
1976 /*{Protheus.doc} InsertData
1977 Função para retornar os nós Pais
1978 @return lRecorded Registro inserido
1979 @obs
1980 @sample
1981 */
1982 //-----
1983 Static Function InsertData()
1984     Local aArea := GetArea() //Workarea Anterior
1985     Local aAreaMF7 := MF7->(GetArea()) //WorkArea MF7
1986     Local aAreaMF8 := MF8->(GetArea()) //WorkArea MF8
1987     Local nConta := 0 //Contador
1988     Local aData := {} //Array de processos/nós
1989     Local nC := 0 //Contador
1990     Local cLastCmp := "" //Ultimo nó do usuário
1991     Local lRecorded := .T. //Registros atualizados
1992     Local nTamSX3 := TamSX3("MF7_NODE")[1] //Tamanho do nó
1993     Local aComps := {} //array de componentes
1994     Local nTamID := TamSX3("MF8_ID")[1] //Tamanho do Id
1995     Local nComps := 0 //Tamanho de Componentes
1996     Local nCampos := _TOTAL_CAMPOS //Total de campos MF8
1997     Local aCampos := {} //Array de Campos MF8
1998     Local nC2 := 0 //Contador de Campos
1999     Local aRecs := {} //Array de Registros
2000     Local nTotalNodes := 0 //Total dos nos
2001     Local lRecorded := .T. //Registro inserido
2002
2003 BEGIN TRANSACTION

```

```

2004
2005 aData := STFStComponentes(nTamSX3) //Carrega os nós
2006 nTotalNodes := _TOTAL_NODES //Total dos nos
2007
2008 MF7->( DbEval({ || nConta := nConta+1},{ || MF7_PROPR == "S" .AND. MF7_FILIAL == xFilial() } ) )
2009 If nConta <> nTotalNodes //Se a quantidade total de nós for diferente
2010     lRecorded := .T. //Registro inserido
2011
2012 MF7->(DbSetFilter({ || MF7_PROPR == "S" .AND. MF7_FILIAL == xFilial() }, " MF7_PROPR == 'S' .AND. MF7_FILIAL == xFilial() ") )
2013 MF7->(DbGoTop())
2014 Do While !MF7->(Eof())
2015     RecLock("MF7", .F.)
2016     MF7->(DbDelete())
2017     MF7->(MsUnLock())
2018     MF7->(DbSkip())
2019 EndDo
2020 MF7->(DBCclearFilter())
2021
2022 //Deleta o MF8
2023 MF8->(DbSetFilter({ || MF8_PROPR == "S" .AND. MF8_FILIAL == xFilial() }, " MF8_PROPR == 'S' .AND. MF8_FILIAL == xFilial() ") )
2024 MF8->(DbGoTop())
2025 Do While !MF8->(Eof())
2026     RecLock("MF8", .F.)
2027     MF8->(DbDelete())
2028     MF8->(MsUnLock())
2029     MF8->(DbSkip())
2030 EndDo
2031
2032 aCampos := { "MF8_NODE",; //1
2033             "MF8_ID",; //2
2034             "MF8_ENVIRO",; //3
2035             "MF8_TYPE",; //4
2036             "MF8_NAME",; //5
2037             "MF8_TITLE",; //6
2038             "MF8_F3",; //7
2039             "MF8_DEFAULT",; //8
2040             "MF8_TABLE",; //9
2041             "MF8_INDEX",; //10
2042             "MF8_KEY",; //11
2043             "MF8_INTYPE",; //12
2044             'MF8_VALID',; //13/Aspas duplas + nome ocorre erro no compilador
2045             "MF8_LEN",; //14
2046             "MF8_LIST",; //15
2047             "MF8_FTYPE",; //16
2048             "MF8_PAISES",; //17
2049             "MF8_POSICA"} //18
2050
2051 aComps := STFStItensComponents(nTamSX3, nTamID)
2052
2053 MF7->(DbSetOrder(1)) //MF7_FILIAL + MF7_NODE
2054 //Ordena pelo nó pai
2055 nConta := 0

```

```

2056 MF7->(DbSetFilter({ || MF7_PROPR <> "S" .AND. MF7_FILIAL == xFilial() }, " MF7_PROPR <> 'S' .AND. MF7_FILIAL == xFilial() ") )
2057 MF7->(DbGoTop())
2058 MF7->( DbEval({ || nConta := nConta+1, cLastCmp := MF7_NODE},{ ||!Eof() } ) )
2059 MF7->(DbClearFilter())
2060 aSort( aData,, {||x, y| x[_CODE_NODE] < y[_CODE_NODE]})
2061
2062
2063 If nConta > 0 //Existem registros do usuário , então Deleta
2064   cLastCmp := IIF(cLastCmp < aData[_TOTAL_NODES, 2],aData[_TOTAL_NODES, 2], cLastCmp)
2065   For nC := 1 to nTotalNodes
2066     If MF7->(DbSeek(xFilial() + aData[nC, _CODE_NODE])) //
2067       cLastCmp := Soma1(cLastCmp)
2068       RecLock("MF7", .F.)
2069       MF7->MF7_NODE := cLastCmp
2070       MF7->(MsUnlock())
2071       //atualiza os componentes
2072       aRecs := {}
2073       MF8->(DbSetFilter({ || MF8_NODE == aData[nC, _CODE_NODE] .AND. MF8_FILIAL == xFilial() } , " MF8_NODE == aData[nC, " + Str(_CODE_NODE) + "]" .AND.
MF8_FILIAL == xFilial() ") )
2074       MF8->(DbGotop())
2075
2076       MF8->( DbEval({ || aAdd(aRecs, Recno()) },{ || !Eof() } ) )
2077
2078       MF8->( aEval(aRecs, { | r| DbGoTo(r), RecLock("MF8", .F.), MF8_NODE := cLastCmp, MsUnlock()}))
2079       MF8->(DbClearFilter())
2080
2081       MF7->(DbSetFilter({ || MF7_FATHER == aData[nC, _CODE_NODE] .AND. MF7_FILIAL == xFilial() }, " MF7_FATHER == aData[nC, " + Str(_CODE_NODE) + "]" .AND.
MF7_FILIAL == xFilial()") )
2082       MF7->(DbGotop())
2083       MF7->( DbEval({ || RecLock("MF7", .F.), MF7_FATHER := cLastCmp, MsUnLock() }, { || !Eof() } ))
2084       MF7->(DbClearFilter())
2085     EndIf
2086   Next
2087 EndIf
2088 For nC := 1 to nTotalNodes
2089   If !MF7->(DbSeek(xFilial() + aData[nC, _CODE_NODE] ) )
2090     RecLock("MF7", .T.)
2091     MF7->MF7_FILIAL := xFilial("MF7")
2092     MF7->MF7_FATHER = aData[nC, _FATHER_NODE]
2093     MF7->MF7_NODE := aData[nC, _CODE_NODE]
2094     MF7->MF7_DESCR := aData[nC, _DESCRIPTION_NODE]
2095     MF7->MF7_ORDER := aData[nC, _ORDER_NODE]
2096     MF7->MF7_VALID := aData[nC, _VALID_FUNCTION]
2097     MF7->MF7_ENVIRO := aData[nC, _ENVIRONMENT_NODE]
2098     MF7->MF7_TABREC := aData[nC, _RECORD_TABLE_ONCE]
2099     MF7->MF7_TABINS := aData[nC, _INSERT_TABLE]
2100     MF7->MF7_PROPR := "S"
2101     MF7->MF7_POSREC := aData[nC, _POS_RECORDED]
2102     MF7->MF7_FUNREC := aData[nC, _WRITE_FUNCTION]
2103     MF7->(MsUnLock())
2104
2105   Else

```

```

2106         lRecorded := .F.
2107         Exit
2108     EndIf
2109 Next
2110 If lRecorded
2111     //Insere os componentes
2112     nComps := Len(aComps)
2113     For nC := 1 to nComps
2114
2115         If MF8->( RecLock("MF8", .T.) )
2116
2117             MF8->(FieldPut(FieldPos("MF8_FILIAL"), xFilial()))
2118             MF8->(FieldPut(FieldPos("MF8_PROPR"), "S"))
2119             For nC2 := 1 to nCampos
2120                 If !Empty(aComps[nC, nC2])
2121                     MF8->(FieldPut(FieldPos(aCampos[nC2]), aComps[nC, nC2]))
2122                 EndIf
2123             Next
2124             MF8->(MsUnLock())
2125
2126         Else
2127
2128             lRecorded := .F.
2129             Exit
2130         EndIf
2131     Next
2132 EndIf
2133 EndIf
2134 EndIf
2135
2136 END TRANSACTION
2137
2138 RestArea(aAreaMF8)
2139 RestArea(aAreaMF7)
2140 RestArea(aArea)
2141 Return lRecorded
2142
2143 //-----
2144 /*{Protheus.doc} STFStTEF20
2145 Função para invocar a Janela TEF 20
2146 @param lPOS Ambiente POS?
2147 @param oPanel Objeto Panel
2148 @return NIL
2149 @obs
2150 @sample
2151 */
2152 //-----
2153 Function STFStTEF20( oPanel, cCodigo)
2154     Local aArea      := GetArea()
2155     Local aAreaSLG := SLG->(GetArea())
2156     Local nOpc := 3
2157     Local cEstacao

```

```

2158     Local lDestroy := .F.
2159
2160     DEFAULT oPanel := NIL
2161     DEFAULT cCodigo := ""
2162
2163
2164     cEstacao := cCodigo
2165
2166
2167     If type("cCadastro") <> "C"
2168         Private cCadastro := STR0036 //"Cadastro de Estação"
2169     EndIf
2170
2171
2172     If cCodigo == "" .AND. Type("cLG_CODIGO") == "C"
2173         cCodigo := cLG_CODIGO
2174     EndIf
2175
2176     If ValType(oCfgTef) <> "0"
2177         oCfgTef := LJCCfgTef():New()
2178     EndIf
2179
2180     oCfgTef:Carregar(cCodigo)
2181
2182     If SLG->(DbSeek(xFilial() + cCodigo))
2183
2184         n0pc := 4
2185
2186     EndIf
2187
2188     oCfgTef:Show(n0pc, oPanel)
2189
2190
2191     cEstacao := cEstAnt
2192
2193
2194     cCadastro := NIL
2195
2196     RestArea(aAreaSLG)
2197     RestArea(aArea)
2198
2199     Return oCfgTef
2200
2201 //-----
2202 /*{Protheus.doc} ValidaTEF20
2203 Função para retornar os nós Pais
2204 @param cRetorno Retorno da função
2205 @return lRet TEF Valido?
2206 @obs
2207 @sample
2208 */
2209 //-----

```

```

2210 Static Function ValidaTEF20(cRetorno)
2211     Local lRet := .T.
2212
2213     If cIdAmb == "3"
2214         If !oCfgTef:TefVl(oCfgTef, @lRet)
2215             cRetorno := oCfgTef:cMensagem
2216         EndIf
2217     EndIf
2218
2219 Return lRet
2220
2221 //-----
2222 /*/{Protheus.doc} GravaTEF20
2223 Função para gravar os dados do TEF20
2224 @param cIdAmb
2225 @return NIL
2226 @obs
2227 @sample
2228 */
2229 //-----
2230 Static Function GravaTEF20(cIdAmb)      //colocar o loja121 na pasta pos_final
2231
2232     If cIdAmb == "3"
2233         oCfgTef:Salvar()
2234     EndIf
2235
2236 Return
2237
2238 //-----
2239 /*/{Protheus.doc} LjGrvJob
2240 Função para gravar no arquivo de config do server os dados do job
2241 @param cIdAmb    Código do ambiente
2242 @param aDadosL   Linha do no
2243 @param cCodigo   Código da Estação
2244 @param cKey      Chave da Seção
2245 @param cRot      Nome da rotina (main)
2246 @return NIL
2247 @obs
2248 @sample
2249 */
2250 //-----
2251 Static Function LjGrvJob(cIdAmb, aDadosL, cCodigo, cKey,;
2252                        cRot)
2253     Local lWrite := .T.      //Gravação com sucesso
2254     Local nTam := Len(aDadosL[_COMPS_NODE]) //Dados
2255     Local lGrvIni := .F.     //GravaIni ?
2256     Local cJobs := ""        //Jobs
2257     Local nI := 0            //Contador
2258     Local lOnStart := .F.    //Grava on start
2259     Local lAltConfig := .F.  //Alteração configuração
2260     Local cAmb := ""         ///Ambiente
2261     Local cEmp := ""         //Empresa

```

```

2262 Local cFil := "" //Filial
2263 Local nC := 0 //Variável contadora
2264 Local lNovo := .F. //Job Novo
2265
2266
2267 DEFAULT cKey := "APFrontLoja"
2268 DEFAULT cRot := "FRTA020"
2269 BeginTran()
2270
2271 For nC := 1 to nTam
2272 If aDadosL[_COMPS_NODE, nC, _COMP_TYPE] <> ARQUIVO_INI .OR. cIdAmb == "3"
2273 lWrite := lWrite .AND. WriteCrLf(aDadosL[_COMPS_NODE, nC], aDadosL[_INSERT_TABLE], _COMP_DEFAULT)
2274 ElseIf aDadosL[_COMPS_NODE, nC, _COMP_TYPE] == ARQUIVO_INI .AND. cIdAmb <> "3"
2275 If RTrim(aDadosL[_COMPS_NODE, nC, _COMP_NAME]) == "Environment" .AND. RTrim(aDadosL[_COMPS_NODE, nC, _COMP_KEY]) == cKey
2276 lGrvIni := !Empty(aDadosL[_COMPS_NODE, nC, _COMP_DEFAULT])
2277 cAmb := aDadosL[_COMPS_NODE, nC, _COMP_DEFAULT]
2278 ElseIf RTrim(aDadosL[_COMPS_NODE, nC, _COMP_NAME]) == "Parm1" .AND. RTrim(aDadosL[_COMPS_NODE, nC, _COMP_KEY]) == cKey
2279 cEmp := aDadosL[_COMPS_NODE, nC, _COMP_DEFAULT]
2280 ElseIf RTrim(aDadosL[_COMPS_NODE, nC, _COMP_NAME]) == "Parm2" .AND. RTrim(aDadosL[_COMPS_NODE, nC, _COMP_KEY]) == cKey
2281 cFil := aDadosL[_COMPS_NODE, nC, _COMP_DEFAULT]
2282 EndIf
2283 EndIf
2284 If !lWrite
2285 Exit
2286 EndIf
2287 Next
2288
2289 If lGrvIni .AND. lWrite
2290 cJobs := AllTrim(GetPvProfString("OnStart", "Jobs", "", GetAdv97()))
2291 lOnStart := If(Len(cJobs) == 0, .F., .T.) // Verifica a existencia desta secão
2292 lNovo := At(cKey, cJobs) == 0
2293 If At(cKey, cJobs) > 0 // Elimina a Chamada do Job APFrontLoja
2294 cJobs := Stuff(cJobs, At(cKey, cJobs), 12, "")
2295 EndIf
2296
2297 cJobs := If(Left(cJobs, 1) == ",", SubStr(cJobs, 2), cJobs) // Elimina "," (virgulas) excedentes
2298 cJobs := If(Right(cJobs, 1) == ",", SubStr(cJobs, 1, Len(cJobs) - 1), cJobs)
2299 For nI := 1 To Len(cJobs)
2300 If SubStr(cJobs, nI, 2) == ",,"
2301 cJobs := Stuff(cJobs, nI, 2, ",")
2302 lAltConfig := .T.
2303 EndIf
2304 Next nI
2305
2306
2307 cJobs := If(Empty(cAmb), "", cKey) + If(Len(cJobs) > 0, ",", "") + cJobs
2308
2309
2310 If lAltConfig .Or. lNovo
2311 WritePProString("OnStart", "Jobs", cJobs, GetAdv97())
2312 EndIf
2313

```

```

2314      WritePProString(cKey, "Main", cRot, GetAdv97())
2315      WritePProString(cKey, "Environment", cAmb, GetAdv97())
2316      WritePProString(cKey, "nParms", IIF(!Empty(cCodigo), "3", "2"), GetAdv97())
2317      WritePProString(cKey, "Parm1", cEmp, GetAdv97())
2318      WritePProString(cKey, "Parm2", cFil, Getadv97())
2319      IF !Empty(cCodigo)
2320          WritePProString(cKey, "Parm3", cCodigo, GetAdv97())
2321      EndIf
2322  EndIf
2323
2324
2325  If lWrite
2326      EndTran()
2327
2328  Else
2329      DisarmTransaction()
2330  EndIf
2331
2332  Return lWrite
2333
2334  //-----
2335  /*{Protheus.doc} Lj120Alt
2336  Função para invocar o cadastro de caixa
2337  @return NIL
2338  @obs
2339  @sample
2340  */
2341  //-----
2342  Static Function Lj120Alt()
2343
2344      Local aCampos := {}
2345      Local aRet := {}
2346      Local aArea := GetArea()
2347      Local aAreaSLF := SLF->(GetArea())
2348
2349
2350      aCampos := { { 1,STR0046, __cUserID,"@!","'','','',,,.T. }; //"Caixa"
2351                  }
2352
2353      If ParamBox(aCampos, STR0047, @aRet) //"Informe o Código do Caixa"
2354          DbSelectArea("SLF")
2355          DbSetOrder(1) //LF_FILIAL + LF_COD
2356          If DbSeek( xFilial() + aRet[1])
2357
2358              a120CFG(.F.)
2359          EndIf
2360          RestArea(aAreaSLF)
2361      EndIf
2362
2363      RestArea(aArea)
2364  Return
2365  //-----

```

```
2366 /*/{Protheus.doc} RetLTPPed
2367 Função para retornar o tipo de quebra do pedido
2368 @return aRet - Combo do pedido
2369 @obs
2370 @sample
2371 */
2372 //-------------------------------------
2373 Static Function RetLTPPed()
2374     Local aRet := { STR0037;; //"1= Loja Reserva + Tipo de Entrega"
2375                     STR0038;; //"2= Loja Reserva + Tipo de Entrega + Código do Contato"
2376                     STR0039;; //"3= Loja Reserva + Tipo de Entrega + Data Entrega"
2377                     STR0040;; //"4= Loja Reserva + Tipo de Entrega + Código do Contato + Data de Entrega"
2378                     STR0041;; //"5= Loja Reserva + Tipo de Entrega + Data da Montagem"
2379                     STR0042;; //"6= Loja Reserva + Tipo de Entrega + Data de Montagem + Código do Contato"
2380                     STR0043;; //"7= Loja Reserva + Tipo de Entrega + Codigo do Contato + Data de entrega + Data de Montagem"
2381                     STR0044} //"8= Loja Reserva + Tipo de Entrega + Data de Entrega + o Codigo do Contato + Turno"
2382
2383 Return aRet
```

Framework/File/STFWCDateLine.PRW

61 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "MSOBJECT.CH"
2
3 User Function STFWCDateLine ; Return  // "dummy" function - Internal Use
4
5 //-----
6 /*{Protheus.doc} STFWCDateLine
7 Classe responsavel em armazenar cada linha do arquivo.
8
9 @param
10 @see
11 @return Self
12 @todo
13 @obs
14 @sample
15 */
16 //-----
17
18 Class STFWCDateLine
19
20     Data cLine                      //Linha de texto do arquivo
21
22     Method STFWCDateLine()          //Metodo construtor
23     Method Line()                   //Metodo que ira retornar cada linha do arquivo
24
25 EndClass
26
27 //-----
28 /*{Protheus.doc} STFWCDateLine
29 Construtor da classe STFWCDateLine
30
31 @param
32 @see
33 @return Self
34 @todo
35 @obs
36 @sample
37 */
38 //-----
39
40 Method STFWCDateLine() Class STFWCDateLine
41
42     Self:cLine := ""
43
44 Return Self
45
46 //-----
47 /*{Protheus.doc} STFWCDateLine
48 Responsavel em retornar a linha de texto do arquivo.
49
50 @param
51 @see
52 @return Linha

```

```
53 @todo
54 @obs
55 @sample
56 /*/
57 //-----
58
59 Method Line() Class STFWCDateLine
60
61 Return Self:cLine
```

Framework/File/STFWCFile.PRW

412 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "MSOBJECT.CH"
2 #INCLUDE "AUTODEF.CH"
3
4 #DEFINE FINALTEXT Chr(13) + Chr(10)
5
6 User Function STFWCFile() ; Return // "dummy" function - Internal Use
7
8 //-----
9 /*/{Protheus.doc} STFWCFile
10 Classe responsavel em manipular um arquivo.
11
12 @param
13 @see
14 @return Self
15 @todo
16 @obs
17 @sample
18 */
19 //-----
20
21 Class STFWCFile
22
23     Data cFileName           //Nome do arquivo
24     Data cPath               //Caminho do arquivo
25     Data nHandle             //Handle do arquivo
26     Data cFile               //cPath + cFileName
27     Data oData               //Objeto do tipo Dados com as linhas de texto do arquivo
28     Data cBar                //barra "/" para Linux "\" para windows
29
30     Method STFWCFile(cFile)   //Metodo construtor
31     Method File(cPath, cFileName) //Metodo que ira receber o Path e Nome do Arquivo
32     Method Open(nModo)        //Metodo que ira abrir o arquivo
33     Method Close()            //Metodo que ira fechar o arquivo
34     Method Write(cTexto)      //Metodo que ira escrever no final arquivo
35     Method Read()             //Metodo que ira ler o arquivo
36     Method Create()           //Metodo que ira criar o arquivo
37     Method Exist()            //Metodo que ira verificar se o arquivo existe
38     Method Erase()            //Metodo que ira apagar um arquivo
39     Method Data()             //Metodo que ira retornar uma colecao com o conteudo do arquivo
40     Method WriteXML(cTexto)    //Metodo que ira escrever no inicio do arquivo (PROVISORIO)
41     Method Rename(cFileName)  //Metodo que ira renomear o arquivo
42
43 EndClass
44
45 //-----
46 /*/{Protheus.doc} STFWCFile
47 Classe responsavel em manipular um arquivo.
48
49 @param EXPC1 (1 - cFile) - Caminho do arquivo e Nome do Arquivo.
50 @see
51 @return Self
52 @todo

```

```

53 @obs
54 @sample
55 */
56 //-----
57
58 Method STFWCFile(cFile) Class STFWCFile
59
60     Self:cFile      := cFile
61
62     Self:nHandle     := 0
63
64     Self:oData       := STFWCHashTable():STFWCHashTable()
65
66 Return Self
67
68 //-----
69 /*{Protheus.doc} File
70 Classe responsavel em manipular um arquivo atraves de um caminho especifico.
71
72 @param    EXPC1 (1 - cPath)      - Caminho do arquivo.
73 @param    EXPC2 (2 - cFileName) - Nome do arquivo.
74 @see
75 @return   Self
76 @todo
77 @obs
78 @sample
79 */
80 //-----
81
82 Method File(cPath, cFileName) Class STFWCFile
83
84     Default cFileName := ""
85     Default cPath      := ""
86
87     Self:cFileName := cFileName
88     Self:cPath     := cPath
89     Self:nHandle   := 0
90
91     //Verifica se o path esta vindo com barra no final
92     If GetRemoteType() <> REMOTE_LINUX
93
94         Self:cBar := "\"
95
96         If Substr(Self:cPath, Len(Self:cPath), 1) == Self:cBar
97             Self:cFile := Self:cPath + Self:cFileName
98         Else
99             Self:cFile := Self:cPath + Self:cBar + Self:cFileName
100         EndIf
101     Else
102
103         Self:cBar := "/"
104

```

```

105      If Substr(Self:cPath, Len(Self:cPath), 1) == Self:cBar
106          Self:cFile := Self:cPath + Self:cFileName
107      Else
108          Self:cFile := Self:cPath + Self:cBar + Self:cFileName
109      EndIf
110  EndIf
111
112  Self:oData := STFWCHashTable():STFWCHashTable()
113
114  Return Self
115
116  //-----
117  /*{Protheus.doc} Open
118  Responsavel em abrir um arquivo.
119
120  @param    ExpN1 (1 - nModo) - Modo de abertura do arquivo. 0 - Abrir para ler | 1 - abrir para escrever | 2 - abri para ler e escrever(default)
121  @see
122  @return   Logico
123  @todo
124  @obs
125  @sample
126  */
127  //-----
128
129  Method Open(nMode) Class STFWCFile
130
131      Local lReturn := .T.                //Variavel de retorno do metodo
132
133      Default nMode := 2
134
135      //Abre o arquivo
136      Self:nHandle := FOPEN(Self:cFile, nMode)
137
138      If Self:nHandle == -1
139          lReturn := .F.
140      EndIf
141
142  Return lReturn
143
144  //-----
145  /*{Protheus.doc} Close
146  Responsavel em fechar um arquivo.
147
148  @param
149  @see
150  @return   Logico
151  @todo
152  @obs
153  @sample
154  */
155  //-----
156

```

```

157 Method Close() Class STFWCFile
158
159     Local lReturn := .F.                //Variavel de retorno do metodo
160
161     //Fecha o arquivo
162     lReturn := FCLOSE(Self:nHandle)
163
164     Self:nHandle := 0
165
166 Return lReturn
167
168 //-----
169 /*{Protheus.doc} Write
170 Responsavel em escrever no final do arquivo.
171
172 @param    ExpC1 (1 - cTexto) - Texto do arquivo.
173 @see
174 @return   Logico
175 @todo
176 @obs
177 @sample
178 */
179 //-----
180
181 Method Write(cText) Class STFWCFile
182
183     Local lReturn    := .T.                //Variavel de retorno do metodo
184     Local nSizeText  := 0                 //Tamanho do texto a ser escrito
185     Local nReturn    := 0                 //Variavel de retorno do metodo FWRITE
186     Local nSize      := 0                 //Tamanho de bytes do arquivo
187
188     cText += FINALTEXT
189
190     //Posiciona no final do arquivo
191     nSize := FSEEK(Self:nHandle, 0, 2)
192
193     //Escreve no Arquivo
194     nReturn := FWRITE(Self:nHandle, cText)
195
196     //Verifica se o comando foi executado
197     nSizeText := Len(cText)
198
199     If nReturn <> nSizeText
200         lReturn := .F.
201     EndIf
202
203 Return lReturn
204
205 //-----
206 /*{Protheus.doc} EscrevXML
207 Responsavel em escrever no final do arquivo.
208

```

```

209 @param    ExpC1 (1 - cTexto) - Texto do arquivo.
210 @see
211 @return    Logico
212 @todo
213 @obs
214 @sample
215 /*/
216 //-----
217
218 Method WriteXML(cText) Class STFWCFile
219
220     Local lReturn    := .T.                //Variavel de retorno do metodo
221     Local nSizeText  := 0                  //Tamanho do texto a ser escrito
222     Local nReturn    := 0                  //Variavel de retorno do metodo FWRITE
223     Local nSize      := 0                  //Tamanho de bytes no arquivo
224     Local cReturn    := ""                 //Retorno do conteudo do arquivo
225
226     cText += FINALTEXT
227
228     //Posiciona no inicio do arquivo para pegar o total de bytes escrito no arquivo
229     nSize := FSEEK(Self:nHandle, 0, 2)
230
231     //Posiciona no inicio do arquivo
232     FSEEK(Self:nHandle, 0, 0)
233
234     //Le o Arquivo
235     FREAD(Self:nHandle, cReturn, nSize)
236
237     //Posiciona no inicio do arquivo
238     FSEEK(Self:nHandle, 0, 0)
239
240     //Escreve no arquivo
241     nReturn := FWRITE(Self:nHandle, cText + cReturn)
242
243     //Verifica se o comando foi executado
244     nSizeText := Len(cText + cReturn)
245
246     If nReturn <> nSizeText
247         lReturn := .F.
248     EndIf
249
250 Return lReturn
251
252 //-----
253 /*/{Protheus.doc} Create
254 Responsavel em criar o arquivo.
255
256 @param
257 @see
258 @return    Logico
259 @todo
260 @obs

```

```

261 @sample
262 /*/
263 //-----
264
265 Method Create() Class STFWCFile
266
267     Local lReturn    := .T.                //Variavel de retorno do metodo
268
269     //Cria o arquivo
270     Self:nHandle := FCREATE(Self:cFile)
271
272     If Self:nHandle == -1
273         lReturn := .F.
274     EndIf
275
276 Return lReturn
277
278 //-----
279 /*/{Protheus.doc} Exist
280 Responsavel em verificar se o arquivo existe.
281
282 @param
283 @see
284 @return Logico
285 @todo
286 @obs
287 @sample
288 /*/
289 //-----
290
291 Method Exist() Class STFWCFile
292
293     Local lReturn    := .F.                //Variavel de retorno do metodo
294
295     //Verifica se o arquivo existe
296     lReturn := File(Self:cFile)
297
298 Return lReturn
299
300 //-----
301 /*/{Protheus.doc} Exist
302 Responsavel em apagar um arquivo.
303
304 @param
305 @see
306 @return Logico
307 @todo
308 @obs
309 @sample
310 /*/
311 //-----
312

```

```

313 Method Erase() Class STFWCFile
314
315     Local lReturn    := .T.           //Variavel de retorno do metodo
316     Local nReturn    := 0             //Retorno da funcao FERASE
317
318     //Apaga o arquivo
319     nReturn := FERASE(Self:cFile)
320
321     If nReturn == -1
322         lReturn := .F.
323     EndIf
324
325 Return lReturn
326
327 //-----
328 /*{Protheus.doc} Read
329 Responsavel em ler o conteudo do arquivo.
330
331 @param
332 @see
333 @return Logico
334 @todo
335 @obs
336 @sample
337 */
338 //-----
339
340 Method Read() Class STFWCFile
341
342     Local cLine       := ""           //Retorno da leitura do arquivo
343     Local nCount      := 0            //Variavel de controle do tipo contador
344     Local oDateLine   := Nil         //Objeto do tipo DadoLinha
345
346     //Estancia o objeto Dados (esta inicializando)
347     Self:oData := STFWCHashTable():STFWCHashTable()
348
349     //Coloca o arquivo em uso
350     FT_FUSE(Self:cFile)
351     //Posiciona na primeira linha
352     FT_FGOTOP()
353
354     //Varre o arquivo
355     While !FT_FEOF()
356
357         nCount++
358
359         //Estancia o objeto DadoLinha
360         oDateLine := STFWCDateLine():STFWCDateLine()
361         //Pega a linha
362         cLine     := FT_FREADLN()
363         //Atribui a linha de texto ao objeto
364         oDateLine:cLine := cLine

```

```

365      //Adiciona o objeto DadoLinha na colecao
366      Self:oData:Add("L" + AllTrim(Str(nCount)), oDateLine)
367      //Pula a linha
368      FT_FSKIP()
369      End
370
371      //Libera o arquivo
372      FT_FUSE()
373
374      Return Self:oData
375
376      //-----
377      /*{Protheus.doc} Data
378      Responsavel em retorna uma colecao com os dados do arquivo.
379
380      @param
381      @see
382      @return Objeto Colecao
383      @todo
384      @obs
385      @sample
386      */
387      //-----
388
389      Method Data() Class STFWCFile
390      Return Self:oData
391
392      //-----
393      /*{Protheus.doc} Rename
394      Responsavel em renomear um arquivo.
395
396      @param      EXPC1 (1 - cFileName) - Nome para qual será renomeado
397      @see
398      @return      Numerico
399      @todo
400      @obs
401      @sample
402      */
403      //-----
404
405      Method Rename(cFileName) Class STFWCFile
406
407      Local nReturn := 0          //Retorno do metodo
408
409      //Renomeia o arquivo
410      nReturn := FRENAME(Self:cFile , cFileName)
411
412      Return nReturn

```

Framework/GetStation/STFGetStation.prw

199 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "STFGETSTATION.CH"
3
4
5 Static cAuxEstacao
6
7 //-----
8 /*{Protheus.doc} STFGetStat
9 Retorna o(s) valor(es) do(s) campo(s) da SLG
10
11 @param      uCampos - Informa o campo ou os campos que deseja retornar o valor da SLG
12 @param      lTef     - Informa se o campo eh da tabela do TEF MDG
13 @param      lUpperChar - Informa se o retorna Caracteres em uppercase
14 @return      uReturn - Retorna o valor do campo ou dos campos que foi solicitado na funcao
15 @obs
16 @sample
17 */
18 //-----
19 Function STFGetStat(uCampos , lTef, lUpperChar )
20
21 Local aArea      := GetArea()           //Salva area
22 Local uReturn    := ""                 //Retorno da funcao
23 Local nI         := 0                 //Variavel de loop
24 Local xRet       := ""                 //Variavel que armazena o nome do campo
25 Local cParam     := ""                 //Armazena um bloco de codigo para uma macro execucaoR
26 Local nTamSerie  := TamSX3("LG_SERIE")[1] //Tamanho do campo LG_SERIE
27 Local lRet       := .T.                 // Retorno caso não haja estação
28
29 Default uCampos   := Nil
30 Default lTef      := .F.
31 Default lUpperChar := .T. //Caracteres em uppercase?
32
33 If cAuxEstacao == NIL .AND. !Empty(cEstacao)
34     cAuxEstacao := cEstacao
35 Endif
36
37 DbSelectArea("SLG")
38 DbSetOrder(1)//LG_FILIAL+LG_CODIGO
39
40 If !DbSeek(xFilial("SLG")+cEstacao)
41     If Empty(cEstacao) .AND. ValType(cAuxEstacao) == "C" .AND. !Empty(cAuxEstacao)
42         cEstacao := cAuxEstacao
43     Endif
44     If Empty(cEstacao)
45         //"0 cadastro de estacao desse operador não foi localizado, por favor, saia do sistema e entre novamente."
46         //"Atenção"
47         MsgStop(STR0002, STR0001)
48         DbSeek(xFilial("SLG"),.T.)
49         lRet := .F.
50     Endif
51 Endif
52

```

```

53 If lRet
54   If ValType(uCampos) == "A"
55
56     uReturn := {}
57
58     For nI := 1 To Len(uCampos)
59
60       xRet := FieldGet( FieldPos( If(! Left(uCampos[nI],3) == "LG_", "LG_", "") + uCampos[nI]))
61
62       If ValType(xRet) == "C"
63         If lUpperChar
64           xRet := Upper(AllTrim(xRet))
65         Else
66           xRet := AllTrim(xRet)
67         EndIf
68       EndIf
69
70       If Upper(uCampos[nI]) == "SERIE"
71
72         cParam := SuperGetMV("MV_LJSERIE")
73
74         // Este parametro permite que seja criada uma regra para o preenchimento
75         // do Numero de Serie do cupom:
76         // Por exemplo:
77         // MV_LJSERIE = "Chr(64+Val(cFilAnt))+StrZero(Val(LJGetStation("PDV")),2)"
78         // Ira gerar o numero "A01", onde "A" e a Filial e "01" e o numero do ECF.
79         If !Empty(cParam)
80           xRet := &(cParam)
81         EndIf
82
83         //Formata a variavel de retorno conforme o tamanho do campo
84         Padr(xRet, nTamSerie)
85
86       EndIf
87
88       AAdd(uReturn,xRet)
89
90     Next nI
91
92 Else
93
94   If lTef
95
96     DbSelectArea("MDG")
97     DbSetOrder(1) //MDG_FILIAL+MDG_CODEST
98     If DbSeek(xFilial("MDG") + cEstacao)
99       uReturn := FieldGet( FieldPos( If(! Left(uCampos,4) == "MDG_", "MDG_", "") + uCampos))
100     EndIf
101
102   Else
103     uReturn := FieldGet( FieldPos( If(! Left(uCampos,3) == "LG_", "LG_", "") + uCampos))
104   EndIf

```

```

105
106     If Valtype(uReturn) == "C"
107         If lUpperChar
108             uReturn := Upper(AllTrim(uReturn))
109         Else
110             uReturn := AllTrim(uReturn)
111         EndIf
112     EndIf
113
114     If Upper(uCampos) == "SERIE"
115
116         cParam := SuperGetMV("MV_LJSERIE")
117
118         // Este parametro permite que seja criada uma regra para o preenchimento
119         // do Numero de Serie do cupom:
120         // Por exemplo:
121         // MV_LJSERIE = "Chr(64+Val(cFilAnt))+StrZero(Val(LJGetStation("PDV")),2)"
122         // Ira gerar o numero "A01", onde "A" e a Filial e "01" e o numero do ECF.
123         If !Empty(cParam)
124             uRet := &(cParam)
125         EndIf
126
127         //Formata a variavel de retorno conforme o tamanho do campo
128         Padr(uReturn, nTamSerie )
129
130     EndIf
131
132 EndIf
133 EndIf
134
135 RestArea(aArea)
136
137 Return uReturn
138
139
140 //-----
141 /*{Protheus.doc} STIsRotina
142 Verifica qual eh a rotina chamadora.
143
144 @param      cRotina - Rotina
145 @return     lRetorno - Se foi a rotina do parametro a chamadora
146 @obs
147 @sample
148 */
149 //-----
150 Function STIsRotina( cRotina )
151
152 Local lRetorno := .F.
153 Local nContador:= 1
154
155 Default cRotina := ""
156

```

```

157 //Verifica qual a rotina chamadora
158 While ( !lRetorno .And. !Empty(ProcName(nContador)) )
159     If ( Upper(ProcName(nContador))==cRotina )
160         lRetorno := .T.
161     EndIf
162     nContador++
163 EndDo
164
165 Return lRetorno
166
167
168 //-----
169 /*/{Protheus.doc} STStrToArr
170 Transforma uma lista separada por "," em array
171
172 @param      cLista - Lista a ser transformada em array
173 @return     aLista - Lista em array
174 @obs
175 @sample
176 */
177 //-----
178 Function STStrToArr(cLista)
179
180 Local nPos := 0      // Variavel de loop
181 Local aLista := {}   // Retorno da funcao
182
183 Default cLista := ""
184
185 While Len(cLista)<>0
186     If (nPos:=At(",",cLista))<>0
187         aAdd(aLista, Subs(cLista, 2, nPos-3))
188         cLista := Subs(cLista, nPos+1 )
189     Else
190         If Left(cLista,1)=''
191             aAdd(aLista, Subs(cLista, 2, len(cLista)-2) )
192         Else
193             aAdd(aLista, cLista )
194         EndIf
195         cLista := ""
196     EndIf
197 End
198
199 Return aLista

```

Framework/GetStation/STFSetStation.prw

132 linhas após remoção de metadados de autoria/data

[illegible]

[illegible]

```
105
106 Local aArea      := GetArea()    //Salva area
107 Local lSet       := .T.         //Retorno da função
108 Local nI         := 0           //Contador
109
110 Default aCampos := {}
111
112 DbSelectArea("MDG")
113 DbSetOrder(1)//MDG_FILIAL+MDG_CODEST
114 If DbSeek(xFilial("MDG") + cEstacao)
115
116     If RecLock("MDG", .F.)
117
118         For nI := 1 To Len(aCampos)
119             FieldPut( FieldPos( If(! Left(aCampos[nI, 1],3) == "MDG_", "MDG_", "") + aCampos[nI, 1]), aCampos[nI, 2])
120         Next nI
121
122         MDG->(MsUnLock())
123
124     EndIf
125
126 Else
127     lSet := .F.
128 EndIf
129
130 RestArea(aArea)
131
132 Return lSet
```

Framework/Host/CFGA050A.PRW

2086 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'FWMVCDEF.CH'
2 #INCLUDE "PROTHEUS.CH"
3 #INCLUDE 'FWBROWSE.CH'
4 #INCLUDE 'FWTicket.ch'
5 #INCLUDE "CFGA050.ch"
6 #INCLUDE "POSCSS.CH"
7
8 #DEFINE CON_HTTP    1
9 #DEFINE CON_RPC 2
10 #DEFINE CON_WS    3
11
12
13 Static lWizard      := .F.
14 Static oHostDAO
15 Static cHostID
16 Static aProfile := {}
17 Static lConnVld := .F.
18 Static aSrvAux  := {}
19
20 //-----
21 /*/{Protheus.doc} CFGA050A
22 Rotina de manutenção de hosts
23 Está rotina é chamada pela rotina CFGA050 do Framework
24 */
25 //-----
26 Function CFGA050A()
27 Local aColumns
28 Local aCoors
29 Local aFields
30 Local aSeek
31 Local nCol
32 Local nX
33 Local oDlg, oColumn, oButton, oBrwLayout, oView, oStruct
34 Local cLocalId
35 Local bCommit
36
37 aColumns := {}
38 aCoors := FWGetDialogSize( oMainWnd )
39 aFields := {}
40 aSeek := {}
41 nCol := 0
42 nX := 0
43 cLocalId := GetPvProfString("FwCommunication","ID","",GetAdv97())
44 bCommit := {} || FWCFGA050Commit() }
45
46
47 CursorWait()
48 FWMsgRun( , { || If(oHostDAO == NIL,oHostDAO := FwHostDao():New(),), cHostID := AllTrim(oHostDAO:GetLocalId()) } , , STR0002+"..." )
49 CursorArrow()
50
51 oView := ViewDef()
52 oStruct := oView:GetModel():GetModel('XXPMaster'):GetStruct()

```

```

53
54 aSeek := CreateSeek(oStruct)
55 If select("XXP") >0
56     If Empty(cLocalId) .or. !XXP->(DbSeek(cLocalId)) .or. GetPvProfString("FwCommunication","Replica","0",GetAdv97()) == "1"
57         CFGA050AWizard()
58     Else
59         Define MsDialog oDlg Title I18N( STR0001 ) FROM aCoors[1], aCoors[2] To aCoors[3], aCoors[4] STYLE nOR( WS_VISIBLE, WS_POPUP ) Pixel
60
61         //-----
62         // Define o Browse
63         //-----
64         oBrwLayout:= FWFormBrowse():New()
65         oBrwLayout:SetDataTable(.T.)
66         oBrwLayout:SetAlias("XXP")
67         oBrwLayout:DisableReport()
68         oBrwLayout:DisableDetails()
69         oBrwLayout:SetSeek(Nil,aSeek)
70         oBrwLayout:SetOwner(oDlg)
71         oBrwLayout:SetDescription( I18N( STR0001 ) )
72         oBrwLayout:AddLegend({|| AllTrim(XXP->XXP_ID) == cHostID},'BLUE',STR0058)
73
74         //-----
75         // Adiciona as colunas do Browse
76         //-----
77         aFields := FWXXPFields()
78
79         For nX := 1 to Len(aFields)
80             If aFields[nX][7]
81                 nCol++
82                 AAdd(aColumns,FWBrwColumn():New())
83                 aColumns[nCol]:SetData( &("{||" + aFields[nX][1] + "}") )
84                 aColumns[nCol]:SetTitle(aFields[nX][2])
85                 aColumns[nCol]:SetSize(aFields[nX][4])
86             EndIf
87         Next nX
88
89         oBrwLayout:SetColumns(aColumns)
90
91         //-----
92         // Adiciona os botoes do Browse
93         //-----
94         oBrwLayout:AddButton(I18N(STR0059) , 'VIEWDEF.CFGA050' , ,MODEL_OPERATION_VIEW , ,.F.)
95         oBrwLayout:AddButton(I18N(STR0060) , 'CFGA050AWizard' , ,MODEL_OPERATION_INSERT , ,.F.)
96         oBrwLayout:AddButton(I18N(STR0061) , 'VIEWDEF.CFGA050' , ,MODEL_OPERATION_UPDATE , ,.F.)
97
98         //-----
99         // Adiciona os botoes do Filtro
100        //-----
101        oBrwLayout:SetUseFilter(.T.)
102
103        //-----
104        // Ativação do Browse

```

```

105      //-----
106      oBrwLayout:Activate()
107
108      Activate MsDialog oDlg Centered
109      EndIf
110 EndIf
111
112 Return
113
114 //-----
115 /*{Protheus.doc} ModelDef
116 Modelo de Dados
117
118 */
119 //-----
120 Static Function ModelDef()
121 Local oModel      := Nil
122 Local oModelXXX
123 Local oStruXXP    := DefStrModel("XXP")
124 Local oStruXXX    := DefStrModel("XXX")
125 Local oStruAUX    := DefStrAuxModel("AUX")
126 Local bPosValid  := {|| RunHostUpdate() }
127 Local bCommit    := {|| FWCFG050Commit() }
128
129 DEFAULT oHostDAO := FwHostDao():New()
130 DEFAULT cHostID  := AllTrim(oHostDAO:GetLocalId())
131
132 FwOpenXXX()
133 FwCheckDic("XXP")
134 FwCheckDic("XXX")
135
136 oModel := MPFormModel():New( 'CFG050', /*bPreValidacao*/, bPosValid, bCommit )
137
138 oModel:AddFields('XXPMaster', /*cOwner*/, oStruXXP, /*bPreValidacao*/, /*bPosValid*/, /*bCarga*/ )
139 oModel:AddGrid('XXXDETAIL' , 'XXPMaster', oStruXXX, /*bLinePre*/, /*bLinePost*/, /*bPre*/, /*bPost*/, {oGrid| XXXLoad(oGrid)})
140 oModel:AddFields('AUXDETAIL' , 'XXPMaster', oStruAUX, /*bPreValidacao*/, /*bPosValid*/, {oField| AuxLoad(oField)})
141
142 oModel:SetRelation('XXXDETAIL' ,{'XXX_ID','XXP_ID'},XXX->(IndexKey(1)))
143
144 oModelXXX := oModel:GetModel('XXXDETAIL')
145 oModelXXX:SetUniqueLine({"XXX_PROTODC"})
146 oModelXXX:SetNoInsertLine(.T.)
147
148 oModel:GetModel("AUXDETAIL"):SetOnlyQuery(.T.)
149
150 oModel:SetDescription( STR0001 )
151 oModel:SetPrimaryKey( {"XXP_ID"} )
152 oModel:SetVldActive( { | oModel | CanUpdate( oModel ) } )
153 Return oModel
154
155 //-----
156 /*{Protheus.doc} ViewDef

```

```

157 Interface
158
159 /*/
160 //-----
161 Static Function ViewDef()
162 Local oModel      := FWLoadModel( 'CFGA050' )
163 Local oStruXXP    := DefStrView("XXP")
164 Local oStruXXX    := DefStrView("XXX")
165 Local oStruAUX    := DefStrAuxView("AUX")
166 Local oView       := Nil
167 Local nOperation := oModel:GetOperation()
168
169 oStruXXP:RemoveField('XXP_UPDATE')
170 oStruXXX:RemoveField('XXX_ID')
171
172 oView := FWFormView():New()
173
174 oView:SetModel( oModel )
175 oView:AddField( 'VIEW_XXP', oStruXXP, 'XXPMaster' )
176 oView:AddGrid( 'VIEW_XXX', oStruXXX, 'XXXDETAIL' )
177 oView:AddField( 'VIEW_AUX', oStruAUX, 'AUXDETAIL' )
178 oView:AddOtherObject('ACTIONS',{oPanel| ActionPanel(oPanel) })
179 oView:AddOtherObject('SPACE',{||})
180 oView:AddOtherObject('SRVAUX',{oPanel| SrvAuxPanel(oPanel) })
181
182 oView:CreateHorizontalBox( 'TOP', 30 )
183 oView:CreateHorizontalBox( 'BOTTOM1', 40 )
184 oView:CreateHorizontalBox( 'SPACE', 03 )
185 oView:CreateHorizontalBox( 'BOTTOM2', 27 )
186
187 oView:CreateVerticalBox( 'BOTTOMLEFT', 90, 'BOTTOM1' )
188 oView:CreateVerticalBox( 'BOTTOMRIGHT', 10, 'BOTTOM1' )
189
190 oView:CreateVerticalBox( 'BOTTOM2LEFT', 70, 'BOTTOM2' )
191 oView:CreateVerticalBox( 'BOTTOM2RIGHT', 30, 'BOTTOM2' )
192
193 oView:SetOwnerView( 'VIEW_XXP', 'TOP' )
194 oView:SetOwnerView( 'VIEW_XXX', 'BOTTOMLEFT' )
195 oView:SetOwnerView( 'ACTIONS', 'BOTTOMRIGHT' )
196 oView:SetOwnerView( 'VIEW_AUX', 'BOTTOM2LEFT' )
197 oView:SetOwnerView( 'SRVAUX', 'BOTTOM2RIGHT' )
198
199 oView:EnableTitleView("VIEW_XXP"      , "Host"      )
200 oView:EnableTitleView("VIEW_XXX"      , STR0003)
201 oView:EnableTitleView("ACTIONS"       , STR0004 )
202 oView:EnableTitleView("VIEW_AUX"      , STR0105 )// "Dados do Servidor Auxiliar"
203 oView:EnableTitleView("SRVAUX"       , STR0106 )// "Testar Conexão"
204
205 oView:SetCloseOnOk({||.T.})
206
207 FWModelActive( oModel )
208

```

```

209 Return oView
210
211
212 //-----
213 /*{Protheus.doc} DefStrModel
214 Monta um Struct para utilização da tabela XXP (Model)
215
216 */
217 //-----
218 Static Function DefStrModel(cTable,aYesFields)
219
220 Local aArea      := GetArea()
221 Local aAux       := {}
222 Local aFields    := {}
223 Local aIndex     := {}
224 Local aTriggers  := {}
225 Local bRelac     := Nil
226 Local bValid     := { ||.T.}
227 Local bWhen      := Nil
228 Local cChave     := ""
229 Local nX         := 0
230 Local nLen
231
232 oStruct := FWFormModelStruct():New()
233
234 //-----
235 // Tabela
236 //-----
237 If cTable == "XXP"
238     oStruct:AddTable("XXP",FWXXPPPrimary(),"Hosts")
239     aIndex := FWXXPIndex()
240     aFields := FWXXPFields()
241 ElseIf cTable == "XXX"
242     oStruct:AddTable("XXX",FWXXXPrimary(),STR0003)
243     aIndex := FWXXXIndex()
244     aFields := FWXXXFields()
245 EndIf
246
247 //-----
248 // Indices
249 //-----
250
251
252 For nX := 1 to Len(aIndex)
253     cChave := ""
254     aEval( aIndex[nX][2], { |cX| cChave+= "+"+cX } )
255     cChave := SubStr(cChave,2,Len(cChave))
256
257     oStruct:AddIndex(      ;
258     nX                    ,; // [01] Ordem do indice
259     "XXP"+StrZero(nX,2)   ,; // [02] ID
260     cChave                ,; // [03] Chave do indice

```

```

261      ""                ,; // [04] Descrição do índice
262      ""                ,; // [05] Expressão de lookUp dos campos de índice
263      ""                ,; // [06] Nickname do índice
264      .T.                ) // [07] Indica se o índice pode ser utilizado pela interface
265
266 Next nX
267
268 //-----
269 // Campos
270 //-----
271
272 For nX := 1 to Len(aFields)
273
274     If aYesFields <> Nil .AND. aScan(aYesFields,AllTrim(aFields[nX][1])) == 0
275         Loop
276     EndIf
277
278     nLen := Len(aFields[nX])
279     cGSC := 'GET'
280     bRelac := If(nLen > 12,aFields[nX][13],Nil)
281     bValid := If(nLen > 13,aFields[nX][14],Nil)
282     bWhen := If(nLen > 15,aFields[nX][16],Nil)
283
284     oStruct:AddField(
285         AllTrim( aFields[nX][2] ) ,; // [01] Título do campo
286         AllTrim( aFields[nX][2] ) ,; // [02] ToolTip do campo
287         AllTrim( aFields[nX][1] ) ,; // [03] Id do Field
288         aFields[nX][3] ,; // [04] Tipo do campo
289         aFields[nX][4] ,; // [05] Tamanho do campo
290         aFields[nX][5] ,; // [06] Decimal do campo
291         bValid ,; // [07] Code-block de validação do campo
292         bWhen ,; // [08] Code-block de validação When do campo
293         {} ,; // [09] Lista de valores permitido do campo
294         bRelac ,; // [11] Code-block de inicializacao do campo
295         NIL ,; // [12] Indica se trata-se de um campo chave
296         NIL ,; // [13] Indica se o campo pode receber valor em uma operação de update.
297         aFields[nX][9] ) // [14] Indica se o campo é virtual
298
299 Next nX
300
301 For nX := 1 To Len(aTriggers)
302     If aYesFields <> Nil .AND. aScan(aYesFields,AllTrim(aTriggers[nX][1])) == 0
303         Loop
304     EndIf
305
306     oStruct:AddTrigger(aTriggers[nX][1],aTriggers[nX][2],aTriggers[nX][3],aTriggers[nX][4])
307 Next
308
309 RestArea( aArea )
310
311 Return oStruct
312

```

```

313 //-----
314 /*{Protheus.doc} DefStrView
315 Monta um Struct para utilização da tabela XXP (View)
316
317 */
318 //-----
319 Static Function DefStrView(cTable,aYesFields)
320
321 Local aFields   := {}
322 Local cGSC      := ''
323 Local nX        := 0
324 Local oStruct   := FWFormViewStruct():New()
325 Local uLookUp
326
327 If cTable == "XXP"
328     aFields := FWXXPFields()
329 ElseIf cTable == "XXX"
330     aFields := FWXXXFields()
331 EndIf
332
333 //-----
334 // Campos
335 //-----
336
337 For nX := 1 to Len(aFields)
338
339     If !aFields[nX][11]
340         Loop
341     EndIf
342
343     If aYesFields <> Nil .AND. aScan(aYesFields,AllTrim(aFields[nX][1])) == 0
344         Loop
345     EndIf
346
347     If ValType(aFields[nX][12]) == "A"
348         cGSC := "COMBO"
349     Else
350         cGSC := 'GET'
351     EndIf
352
353     uLookUp := If(Len(aFields[nX]) > 14,aFields[nX][15],Nil)
354
355     oStruct:AddField( ;
356     AllTrim( aFields[nX][1] ),, // [01] Campo
357     StrZero(nX,2) ,, // [02] Ordem
358     AllTrim( aFields[nX][2] ),, // [03] Titulo
359     AllTrim( aFields[nX][2] ),, // [04] Descricao
360     NIL ,, // [05] Help
361     cGSC ,, // [06] Tipo do campo COMBO, Get ou CHECK
362     aFields[nX][6] ,, // [07] Picture
363     Nil ,, // [08] PictVar
364     uLookUp ,, // [09] F3

```

```

365     !aFields[nX][8]           ,;      // [10] Editavel
366     ""                       ,;      // [11] Folder
367     ""                       ,;      // [12] Group
368     aFields[nX][12]          ,;      // [13] Lista Combo
369     0                         ,;      // [14] Tam Max Combo
370     ""                       ,;      // [15] Inic. Browse
371     aFields[nX][9]           )      // [16] Virtual
372
373 Next nX
374
375 Return oStruct
376
377
378 //-----
379 /*{Protheus.doc} DefStrAuxModel
380 Monta um Struct sem tabela, que será utilizado apenas para visualização (Model)
381 Fields da conexão com o servidor auxiliar
382
383 */
384 //-----
385 Static Function DefStrAuxModel()
386 Local oStruct := FWFormModelStruct():New()
387
388 oStruct:AddTable('AUX',{},STR0087)
389
390 oStruct:AddField( ;
391 'IP', ;           // [01] Título do campo
392 STR0086, ;       // [02] ToolTip do campo
393 'AUX_IP', ;      // [03] Id do Field
394 'C', ;          // [04] Tipo do campo
395 020, ;          // [05] Tamanho do campo
396 0, ;           // [06] Decimal do campo
397 /*bValid*/, ;   // [07] Code-block de validação do campo
398 /*bWhen*/, ;    // [08] Code-block de validação When do campo
399 /*aValues*/, ;  // [09] Lista de valores permitido do campo
400 /*l0brigat*/, ; // [10] Indica se o campo tem preenchimento obrigatório
401 /*bRelac*/, ;   // [11] Code-block de inicializacao do campo
402 /*lKey*/, ;     // [12] Indica se trata-se de um campo chave
403 /*lNoUpd*/, ;   // [13] Indica se o campo pode receber valor em uma operação de update.
404 /*lVirtual*/)   // [14] Indica se o campo é virtual
405
406 oStruct:AddField( ;
407 'Porta', ;       // [01] Título do campo
408 STR0085, ;       // [02] ToolTip do campo
409 'AUX_PORT', ;    // [03] Id do Field
410 'C', ;          // [04] Tipo do campo
411 006, ;          // [05] Tamanho do campo
412 0, ;           // [06] Decimal do campo
413 /*bValid*/, ;   // [07] Code-block de validação do campo
414 /*bWhen*/, ;    // [08] Code-block de validação When do campo
415 /*aValues*/, ;  // [09] Lista de valores permitido do campo
416 /*l0brigat*/, ; // [10] Indica se o campo tem preenchimento obrigatório

```

```

417 /*bRelac*/, ;           // [11] Code-block de inicializacao do campo
418 /*lKey*/, ;             // [12] Indica se trata-se de um campo chave
419 /*lNoUpd*/, ;           // [13] Indica se o campo pode receber valor em uma operação de update.
420 /*lVirtual*/)           // [14] Indica se o campo é virtual
421
422 oStruct:AddField( ;
423 STR0016, ;               // [01] Titulo do campo
424 STR0084, ;               // [02] ToolTip do campo
425 'AUX_ENV', ;             // [03] Id do Field
426 'C',;                   // [04] Tipo do campo
427 020, ;                   // [05] Tamanho do campo
428 0, ;                     // [06] Decimal do campo
429 /*bValid*/, ;           // [07] Code-block de validação do campo
430 /*bWhen*/, ;            // [08] Code-block de validação When do campo
431 /*aValues*/, ;          // [09] Lista de valores permitido do campo
432 /*lObrigat*/, ;         // [10] Indica se o campo tem preenchimento obrigatório
433 /*bRelac*/, ;           // [11] Code-block de inicializacao do campo
434 /*lKey*/, ;             // [12] Indica se trata-se de um campo chave
435 /*lNoUpd*/, ;           // [13] Indica se o campo pode receber valor em uma operação de update.
436 /*lVirtual*/)           // [14] Indica se o campo é virtual
437
438
439 Return oStruct
440
441 //-----
442 /*{Protheus.doc} DefStrAuxView
443 Monta um Struct sem tabela, que será utilizado apenas para visualização (View)
444 Fields da conexão com o servidor auxiliar
445
446 */
447 //-----
448 Static Function DefStrAuxView()
449 Local oStruct := FWFormViewStruct():New()
450
451 oStruct:AddField( ;
452 'AUX_IP',;               // [01] Campo
453 '01',;                   // [02] Ordem
454 'IP',;                   // [03] Título
455 'IP',;                   // [04] Descrição
456 /*aHelp*/,;             // [05] Help
457 'CHECK',;                // [06] Tipo do campo COMBO, Get ou CHECK
458 /*cPicture*/,;          // [07] Picture
459 /*bPictVar*/,;          // [08] PictVar
460 /*cLookUp*/,;           // [09] F3
461 .T.,;                   // [10] Editavel
462 /*cFolder*/,;           // [11] Folder
463 /*cGroup*/,;            // [12] Group
464 /*aComboValues*/,;      // [13] Lista Combo
465 /*nMaxLenCombo*/,;      // [14] Tam Max Combo
466 /*cIniBrow*/)           // [15] Inic. Browse
467
468 oStruct:AddField( ;

```

```

469 'AUX_PORT',;           // [01] Campo
470 '02',;                 // [02] Ordem
471 STR0015+":",;          // [03] Titulo
472 STR0015+":",;          // [04] Descricao
473 /*aHelp*/,;            // [05] Help
474 'GET',;                 // [06] Tipo do campo   COMBO, Get ou CHECK
475 '@! ',;                // [07] Picture
476 /*bPictVar*/,;         // [08] PictVar
477 /*cLookUp*/,;          // [09] F3
478 /*lCanChange*/,;        // [10] Editavel
479 /*cFolder*/,;          // [11] Folder
480 /*cGroup*/,;           // [12] Group
481 /*aComboValues*/,;     // [13] Lista Combo
482 /*nMaxLenCombo*/,;     // [14] Tam Max Combo
483 /*cIniBrow*/)          // [15] Inic. Browse
484
485 oStruct:AddField( ;
486 'AUX_ENV',;             // [01] Campo
487 '03',;                 // [02] Ordem
488 STR0016+":",;          // [03] Titulo
489 STR0016+":",;          // [04] Descricao
490 /*aHelp*/,;            // [05] Help
491 'GET',;                 // [06] Tipo do campo   COMBO, Get ou CHECK
492 '@! ',;                // [07] Picture
493 /*bPictVar*/,;         // [08] PictVar
494 /*cLookUp*/,;          // [09] F3
495 /*lCanChange*/,;        // [10] Editavel
496 /*cFolder*/,;          // [11] Folder
497 /*cGroup*/,;           // [12] Group
498 /*aComboValues*/,;     // [13] Lista Combo
499 /*nMaxLenCombo*/,;     // [14] Tam Max Combo
500 /*cIniBrow*/)          // [15] Inic. Browse
501 Return oStruct
502
503
504 //-----
505 /*/{Protheus.doc} CreateSeek
506 Cria as chaves de pesquisa do Seek no FWFormBrowse
507
508 @param   oStruct Estrutura da tabela XXP associada ao Model
509
510 */
511 //-----
512 Static Function CreateSeek(oStruct)
513
514 Local aSeek := {}
515
516 Add( aSeek,{   oStruct:GetProperty( "XXP_ID", MODEL_FIELD_TITULO ), ;
517               {   "",;
518                   oStruct:GetProperty( "XXP_ID", MODEL_FIELD_TIPO   ), ;
519                   oStruct:GetProperty( "XXP_ID", MODEL_FIELD_TAMANHO ), ;
520                   oStruct:GetProperty( "XXP_ID", MODEL_FIELD_DECIMAL ), ;

```

```

521         oStruct:GetProperty( "XXP_ID", MODEL_FIELD_TITULO  )}}})
522
523 Add( aSeek,{   oStruct:GetProperty( "XXP_FATHER", MODEL_FIELD_TITULO ), ;
524               {   {"",;
525                   oStruct:GetProperty( "XXP_FATHER", MODEL_FIELD_TIPO      ), ;
526                   oStruct:GetProperty( "XXP_FATHER", MODEL_FIELD_TAMANHO  ), ;
527                   oStruct:GetProperty( "XXP_FATHER", MODEL_FIELD_DECIMAL  ), ;
528                   oStruct:GetProperty( "XXP_FATHER", MODEL_FIELD_TITULO  )}}})
529
530 Return aSeek
531
532 //-----
533 /*/{Protheus.doc} CFGA050AWizard
534 Wizard para cadastro de um novo Host, vinculando-o à máquina local
535
536 @param   cAlias Alias da tabela de Hosts
537 @param   nRecno Registro atual da tabela de hosts
538 @param   nOpc Código da opção selecionada no browse
539
540 /*/
541 //-----
542 Function CFGA050AWizard(cAlias,nRecno,nOpc)
543 Local i
544 Local cTitle      := STR0021
545 Local lConfirm := .F.
546 Local cLocalID := PadR(GetPvProfString("FwCommunication","ID","",GetAdv97()),19)
547 Local cDesc      := Space(50)
548 Local aHeadSrv   := {}
549 Local aHeadLoc   := {}
550 Local aColsSrv   := {}
551 Local aColsLoc   := {}
552 Local aBrwFun     := {}
553 Local aParamFun   := {}
554 Local cProfile    := ""
555 Local oWizard     := Nil
556 Local oGetDLoc    := Nil
557 Local oPanel      := Nil
558 Local oEnv        := Nil
559 Local oCmbConLoc  := Nil
560 Local oFatherHost := Nil
561 Local lPosLight := FwPosLight()
562 Local nRdServer := If(lPosLight,2,1)
563 Local bFinish     := {|| lConfirm := .T. }
564 Local lYesNo := .T.
565 Local bAction := {||}
566 Local oBrwPanel
567 Local oBrwFun
568 Local oColumn
569 Local lNext
570
571 Local oIP
572 Local oPort

```

```

573 Local oCmbConSrv
574 Local lOk           := .F.
575 Local aConn         := {"1=HTTP","2=RPC","3=WebService"}
576 Local oGetDSrv
577 Local oProfile
578 Local cIP           := Space(40)
579 Local cPort         := Space(6)
580 Local cEnv          := Space(60)
581 Local cSrvConTp := ""
582 Local nSrvConTp := 0
583 Local aProtocols
584 Local lReplica := GetPvProfString("FwCommunication","Replica","0",GetAdv97()) == "1"
585
586 Local oIPLocal
587 Local oIPMain
588 Local oPortMain
589 Local oEnvMain
590 Local cIPMain      := PAdr(GetServerIP(),15)
591 Local cIPSec       := Space(15)
592 Local cPortMain := AllTrim(GetPvProfString("TCP","PORT","0",GetAdv97()))
593 Local cEnvMain := GetEnvServer()
594 Local oIPAux
595 Local oPortAux
596 Local oEnvAux
597 Local cPortAux     := Space(6)
598 Local cEnvAux      := Space(30)
599 Local cConnStatus := Space(200)
600 Local lAux := .F.
601 Local oFont
602 Local aBrowse := {}
603 Local oSchedule
604 Local oFWUISchedulePersist
605 Local aEmpFil := {}
606 Local nIPType := 1
607 Local lMobile := FindFunction("STFIsMobile") .AND. STFIsMobile() //Valida versao mobile
608 Local oButPesq := Nil //Botão Pesquisar
609 Local lEdtIp   := .T. //Habilitado edição do IP?
610
611 Local oUser := Nil //Objeto para criação do edit do usuario
612 Local cUser := Space(20) //Variavel para receber o usuario
613 Local oPwd  := Nil //Objeto para criação do edit da senha
614 Local cPwd  := Space(20) //Variavel para receber a senha
615
616 If lMobile //Mobile somente WS
617     aConn := {"3=WebService"}
618 EndIf
619
620 //Valida a execucao do Wizard
621 If !Empty(cLocalID)
622     XXP->(DbSetOrder(1))
623     If XXP->(DbSeek(cLocalID))
624         If lReplica

```

```

625         cProfile := XXP->XXP_PROFIL
626         oFatherHost := FwHost():New()
627         oFatherHost:load(XXP->XXP_FATHER)
628         cIP := Padr(oFatherHost:getIP(),40)
629
630         aProtocols:= oFatherHost:getAllProtocols()
631         For i := 1 To Len(aProtocols)
632             If aProtocols[i]:isEnabled()
633                 nSrvConTp := aProtocols[i]:getProtocolId()
634                 cPort := Padr(cValToChar(aProtocols[i]:getPort()),6)
635                 cEnv := Padr(aProtocols[i]:getEnvironment(),60)
636                 Exit
637             EndIf
638         Next
639     Else
640         Help( , , 'HELP', , I18n(STR0019,{cLocalID}), 1, 0)
641         Return .F.
642     EndIf
643 EndIf
644 EndIf
645
646 PrepareGrids(@aHeadSrv,@aHeadLoc,@aColsSrv,@aColsLoc,@aBrwFun)
647
648 oWizard := ApWizard():New(cTitle,STR0020,STR0021,STR0022,{|| .T. },bFinish,.T.,/*cResHead*/,/*bExecute*/,.F.,{0,0,490,710})
649
650 oWizard:newPanel( cTitle, STR0023, {||.T.}, ;
651     {|| lNext := VldLocal(@oGetDLoc,cDesc,cIPMain,Val(cPortMain),cEnvMain)}, bFinish, .F., {||.T.} )
652
653 oWizard:newPanel( cTitle, STR0024, {||.T.}, {|| lNext := .T., aParamFun := Aclone(aBrwFun),.T.}, bFinish, .F., ;
654     {|| If(nRdServer == 1,oWizard:SetPanel(If(lNext,4,2))), .T.} )
655
656 oWizard:newPanel( cTitle, STR0025, {|| lNext := .F.,.T.}, ;
657     {|| VldParent(cIP, lOk, cProfile)}, bFinish, .F., {||If(nRdServer == 1,oWizard:SetPanel(If(lNext,7,2))),.T.} )
658
659 oWizard:newPanel( cTitle, STR0063, {|| .T.}, ;
660     {|| .T. }, bFinish, .F., {||.T.} )
661
662 oWizard:newPanel( cTitle, STR0026, {||.T.}, {||VldSched(oFWUISchedulePersist,@oSchedul), bFinish, .F., ;
663     {|| If(nRdServer == 1,oWizard:SetPanel(3),(If(oFWUISchedulePersist == NIL,oFWUISchedulePersist := ConfigSched( ;
664         @oSchedul,FUNCTIONUPDATE, SCHEDMODULE,oWizard:GetPanel(6),AllTrim(UPPER(cEnvAux))),)))} )
665
666 oWizard:newPanel( cTitle, STR0096, {||.T.}, {|| .T. }, {|| lConfirm := VldEmpFil(aBrowse,aEmpFil) }, .F., ;
667     {|| .T. } )
668
669
670 lEdtIp := nIPType=2
671
672 //Painel para selecionar o servidor e com informacoes do host local
673 oPanel := oWizard:GetPanel(2)
674
675 @ 005,007 TO 65,345 PROMPT STR0027 PIXEL OF oPanel
676

```

```

677 @ 015,015 RADIO nRdServer ITEMS STR0028, STR0029 OF oPanel SIZE 100,100 PIXEL WHEN !lPosLight
678
679 @ 037,015 SAY STR0009 PIXEL OF oPanel
680 @ 047,015 MSGET cDesc OF oPanel SIZE 150,10 PIXEL
681
682 @ 015,175 RADIO nIPType ITEMS STR0102, STR0103 ON CHANGE (if(lEdtIp := (nIPType=2),oIPLocal:Enable(),oIPLocal:Disable())) OF oPanel SIZE 100,100 PIXEL // "IP dinâmico" ###
    "IP estático"
683
684 @ 037,175 SAY STR0099 OF oPanel PIXEL SIZE 70,10 // "Endereço IP"
685 @ 047,175 MSGET oIPLocal VAR cIPMain OF oPanel SIZE 70,10 PIXEL WHEN lEdtIp
686
687 oIPLocal:bF3 := {|| cIPMain := Padr(GetServerIP(),Len(cIPMain)),oIPLocal:Refresh(),.T.)
688
689 @ 037,255 SAY STR0104 OF oPanel PIXEL SIZE 70,10 // "Endereço IP Secundário"
690 @ 047,255 MSGET cIPSec OF oPanel SIZE 70,10 PIXEL
691
692 @ 067,007 TO 152,345 PROMPT STR0030 PIXEL OF oPanel
693 @ 079,015 SAY STR0031 OF oPanel PIXEL SIZE 200,9
694
695 oGetDLoc := MsNewGetDados():New(089,015,145,340,GD_UPDATE,/*cLinhaOk*/,/*cTudoOk*/,/*cIniCpos*/,;
696     {"XXX_UP","XXX_DOWN","XXX_ENABLE","XXX_PORT","XXX_ENV"},/*nFreeze*/,/*nMax*/,/*cFieldOk*/,/*cSuperDel*/,/*cDelOk*/,;
697     oPanel,aHeadLoc,aColsLoc,/*uChange*/,/*cTela*/)
698
699 oGetDLoc:AddAction("XXX_UP",{||TrocaLinha(oGetDLoc,-1),"UP3"})
700 oGetDLoc:AddAction("XXX_DOWN",{||TrocaLinha(oGetDLoc,1),"DOWN3"})
701
702
703 //Painel para selecionar as funcionalidades
704 oPanel := oWizard:GetPanel(3)
705
706 @ 005,007 TO 150,345 PROMPT STR0032 PIXEL OF oPanel
707
708 @ 18,15 MSPANEL oBrwPanel SIZE 325,125 LOWERED OF oPanel
709
710 DEFINE FWBROWSE oBrwFun DATA ARRAY ARRAY aBrwFun NO LOCATE NO REPORT NO CONFIG OF oBrwPanel
711
712 ADD MARKCOLUMN oColumn DATA {|oBrowse| If(aBrwFun[oBrwFun:At()][1],'LBOK','LBNO')} DOUBLECLICK {|| aBrwFun[oBrwFun:At()][1] := !aBrwFun[oBrwFun:At()][1]} HEADERCLICK {
    |oBrowse| MarkAllFunc(oBrowse,aBrwFun) } OF oBrwFun
713 ADD COLUMN oColumn DATA {|| aBrwFun[oBrwFun:At()][2]} TITLE 'ID' SIZE 10 OF oBrwFun
714 ADD COLUMN oColumn DATA {|| aBrwFun[oBrwFun:At()][3]} TITLE STR0009 SIZE 100 OF oBrwFun
715
716 oBrwFun:Activate()
717
718
719 //Painel com informacoes do host pai
720 oPanel := oWizard:GetPanel(4)
721
722 @ 005,007 TO 170,345 PROMPT STR0033 OF oPanel PIXEL
723
724 @ 015,015 SAY STR0034 OF oPanel PIXEL SIZE 90,9
725 @ 025,015 MSGET oIP VAR cIP OF oPanel SIZE 90,10 PIXEL
726

```

```

727 // A porta sera informada diretamente na URL do Webservice
728 @ 015,110 SAY STR0015 OF oPanel PIXEL SIZE 50,10 //"Porta
729 @ 025,110 MSGET oPort VAR cPort OF oPanel SIZE 50,10 PIXEL WHEN cSrvConTp <> cValToChar(CON_WS)
730
731 @ 015,165 SAY STR0035 OF oPanel PIXEL SIZE 50,9 //"Protocolo inicial"
732 @ 025,165 COMBOBOX oCmbConSrv VAR cSrvConTp ITEMS aConn SIZE 50,10 OF oPanel PIXEL
733 If nSrvConTp > 0
734     oCmbConSrv:nAt := nSrvConTp
735 EndIf
736
737 @ 015,220 SAY STR0036 OF oPanel PIXEL SIZE 140,10 // "Ambiente (apenas para RPC)"
738 @ 025,220 MSGET oEnv VAR cEnv OF oPanel SIZE 80,10 PIXEL WHEN cSrvConTp == cValToChar(CON_RPC) // Apenas para RPC
739
740 @ 050,620 BTNBMP oButPesq RESOURCE "RELOAD" SIZE 030,030 OF oPanel PIXEL;
741     ACTION FwMsgRun( Nil, {|| SaveUser( AllTrim(cUser), AllTrim(cPwd)), lOk := TestConnection(cIP,cPort,cEnv,
742         Val(SubStr(AllTrim(cSrvConTp),1,1)),@oFatherHost,@oGetDSrv)},/*'cHeader'*/,STR0038)
742
743 oButPesq:cToolTip := STR0037
744
745 If lMobile
746     oButPesq:SetCSS( POSCSS (GetClassName(oButPesq), CSS_BTN_LUPA ))
747 EndIf
748
749 @ 042,015 SAY "Usuário:" OF oPanel PIXEL SIZE 100,10
750 @ 052,015 MSGET oUser VAR cUser OF oPanel SIZE 90,10 PIXEL
751
752 @ 042,110 SAY "Senha:" OF oPanel PIXEL SIZE 100,10
753 @ 052,110 MSGET oPwd VAR cPwd PASSWORD OF oPanel SIZE 80,10 PIXEL
754
755 //Informação
756 @ 046,195 SAY "Os campos usuário e senha são necessarios apenas a partir da relese 12.1.33, caso" +;
757     " contrário deixe os campos vazio. Saiba mais em: https://tdn.totvs.com/pages/viewpage.action?pageId=640207780" OF oPanel PIXEL SIZE 130,35
758
759 @ 070,015 SAY STR0039 OF oPanel PIXEL SIZE 100,10
760
761 oGetDSrv := MsNewGetDados():New(80,015,135,340,0,/*cLinhaOk*/,/*cTudoOk*/,/*cIniCpos*/, /*aAlter*/,;
762     /*nFreeze*/,/*nMax*/,/*cFieldOk*/,/*cSuperDel*/,/*cDelOk*/,oPanel,aHeadSrv,aColsSrv,/*uChange*/,/*cTela*/)
763
764 @ 140,015 SAY STR0040 OF oPanel PIXEL SIZE 70,10
765 @ 150,015 MSGET oProfile VAR cProfile OF oPanel SIZE 80,10 PIXEL
766 oProfile:lHasButton := .T.
767 oProfile:bF3 := {|| ConsProfile(@cProfile), oProfile:Refresh(), oPanel:Refresh()}
768
769 //Painel com informacoes do host pai
770 oPanel := oWizard:GetPanel(5)
771
772 @ 010,007 TO 100,172 PROMPT STR0065 PIXEL OF oPanel
773
774 @ 020,015 SAY STR0099 OF oPanel PIXEL SIZE 90,9 // "Endereço IP"
775 @ 030,015 MSGET oIPMain VAR cIPMain OF oPanel SIZE 90,10 PIXEL
776
777 // A porta sera informada diretamente na URL do Webservice

```

```

778 @ 045,015 SAY STR0015 OF oPanel PIXEL SIZE 50,10
779 @ 055,015 MSGET oPortMain VAR cPortMain OF oPanel SIZE 50,10 PIXEL //WHEN cSrvConTp <> cValToChar(CON_WS)
780
781 @ 070,015 SAY STR0036 OF oPanel PIXEL SIZE 70,10
782 @ 080,015 MSGET oEnvMain VAR cEnvMain OF oPanel SIZE 80,10 PIXEL// WHEN cSrvConTp == cValToChar(CON_RPC) // Apenas para RPC
783
784 oIPMain:Disable()
785 oPortMain:Disable()
786 oEnvMain:Disable()
787
788 @ 010,180 TO 100,343 PROMPT STR0066 PIXEL OF oPanel
789
790 cPortAux := AllTrim(GetPvProfString("FWCOMMUNICATION","AUXPORT","",GetAdv97()))
791 cEnvAux := AllTrim(GetPvProfString("FWCOMMUNICATION","AUXENV","",GetAdv97()))
792
793 cPortAux := Padr(cPortAux,LEN(XXX->XXX_PORT))
794 cEnvAux := Padr(cEnvAux,LEN(XXX->XXX_ENV))
795
796 @ 020,188 SAY STR0099 OF oPanel PIXEL SIZE 90,9 // "Endereço IP"
797 @ 030,188 MSGET oIPAux VAR cIPMain OF oPanel SIZE 90,10 PIXEL
798 oIPAux:Disable()
799
800 // A porta sera informada diretamente na URL do Webservice
801 @ 045,188 SAY STR0015 OF oPanel PIXEL SIZE 50,10
802 @ 055,188 MSGET oPortAux VAR cPortAux OF oPanel SIZE 50,10 PICTURE "@9999" PIXEL //WHEN cSrvConTp <> cValToChar(CON_WS)
803
804 @ 070,188 SAY STR0036 OF oPanel PIXEL SIZE 70,10
805 @ 080,188 MSGET oEnvAux VAR cEnvAux OF oPanel SIZE 80,10 PIXEL// WHEN cSrvConTp == cValToChar(CON_RPC) // Apenas para RPC
806
807 @ 110,007 TO 150,343 PROMPT STR0067 PIXEL OF oPanel
808
809 @ 125,015 BUTTON STR0068      SIZE 50, 12 ACTION FWMsgRun(Nil,{|| lAux := VldServers( cIPMain, cPortAux, cEnvAux, oConnSrv, @cConnStatus, .F. )},/*'cHeader'*/,STR0069) PIXEL OF
      oPanel
810 oFont := TFont():New('Arial',,-11,,.T.)
811 oConnSrv := TSay():New(125,080,{||cConnStatus},oPanel,,oFont,,,,.T.,CLR_GREEN,CLR_WHITE,255,20,,,,,.T.)
812
813
814 oPanel := oWizard:GetPanel(7)
815 aBrowse := FWSM0_Panel(oPanel, FWEmpLoad(),,"",.F., .f.)
816
817 oWizard:activate( .T., /*bValid*/ , {||oWizard:oNext:SetFocus()} , /*<bWhen>*/ )
818
819 If lConfirm
820     FWMsgRun( Nil, {|| SaveHost(cIPMain, cIPSec, cDesc,nRdServer,oSchedule,oFatherHost,oGetDLoc,aHeadLoc,aParamFun,cProfile,lReplica,aEmpFil,cPortAux,cEnvAux,nIPType)},/*'cHeader'*/ , STR0044 )
821 EndIf
822 Return
823
824 Static Function SaveHost(cIP, cIPSec, cDesc,nRdServer,oSchedule,oFatherHost,oGetDLoc,aHeadLoc,aBrwFun,cProfile,lReplica,aEmpFil,cPortAux,cEnvAux,nIPType)
825 Local nX
826 Local oFatherAux
827 Local oHost

```

```

828 Local oProtocol
829 Local cAuxIP
830 Local cAuxPort
831 Local cAuxEnv
832 Local oHostAux
833 Local oConn
834 Local aFun := {}
835 Local lRet := .F.
836 Local lYesNo := .T.
837
838 If nRdServer == 2 .and. !lReplica
839     XA2->(DbGoTop())
840     If !XA2->(Eof())
841         lYesNo := MsgYesNo(STR0041)
842     EndIf
843     XXY->(DbGoTop())
844     If !XXY->(Eof()) .AND. lYesNo
845         lYesNo := MsgYesNo(STR0042)
846     EndIf
847 EndIf
848
849 If !lYesNo
850     Return .F.
851 EndIf
852
853 lWizard := .T.
854
855 //Cria o host atual
856 oHost := FwHost():New()
857 oHost:setIP(cIP)
858 oHost:setIPSecondary(cIPSec)
859 oHost:setDescription(cDesc)
860 oHost:setEmpFil(aEmpFil)
861
862 If nRdServer == 1
863     //funcionalidades
864     For nX := 1 To Len(aBrwFun)
865         If aBrwFun[nX][1]
866             Add(aFun,aBrwFun[nX][2])
867         EndIf
868     Next
869     oHost:setIsFather(.T.)
870     oHost:setIdProfile(oHostDA0:createMasterProfile(aFun))
871 Else
872     oHost:setIdProfile(cProfile)
873     oHost:setFather(oFatherHost)
874
875     // salva funcionalidade e perfil
876     CleanProfXFunc()
877     oHostDA0:saveFunctionality(oFatherHost:getFunctionality())
878     oHostDA0:saveProfile(oFatherHost:getAllProfiles())
879

```

```

880     SaveNewSched(oSchedule, FUNCTIONUPDATE, "Special" )
881
882     oFatherAux := oFatherHost
883     While oFatherAux <> Nil
884         oFatherAux:setIsFather(.T.)
885         If !Empty(oFatherAux:getId())
886             oHostDAO:save(oFatherAux)
887         EndIf
888         oFatherAux := oFatherAux:getFather()
889     End
890 EndIf
891
892 //Descarga dos protocolos configurados
893 For nX := 1 to Len(oGetDLoc:aCols)
894     oProtocol := FwProtocol():New()
895     oProtocol:setProtocolId( Val(oGetDLoc:aCols[nX][aScan(aHeadLoc,{|x|AllTrim(x[2]) == "XXX_PROTOC" }])) )
896     oProtocol:setPort( oGetDLoc:aCols[nX][aScan(aHeadLoc,{|x|AllTrim(x[2]) == "XXX_PORT" }))] )
897     oProtocol:setEnvironment( oGetDLoc:aCols[nX][aScan(aHeadLoc,{|x|AllTrim(x[2]) == "XXX_ENV" }))] )
898     oProtocol:setEnabled( oGetDLoc:aCols[nX][aScan(aHeadLoc,{|x|AllTrim(x[2]) == "XXX_ENABLE" }))] == "1" )
899     oHost:addProtocol( oProtocol )
900 Next nX
901
902 oHostDAO:setIPTYPE(nIPTYPE)
903
904 //Obtem o numero sequencial e salva no ini do server
905 lYesNo := .T.
906 While lYesNo
907     If !Empty(oHost:registrate())
908         If nRdServer == 2
909
910             VldServers( cIP, cPortAux, cEnvAux, , , .T., nIPTYPE )
911
912             cAuxIP := AllTrim(GetPvProfString("FwCommunication","auxip","",GetAdv97()))
913             cAuxPort := AllTrim(GetPvProfString("FwCommunication","auxport","",GetAdv97()))
914             cAuxEnv := AllTrim(GetPvProfString("FwCommunication","auxenv","",GetAdv97()))
915             If lReplica
916                 WritePProString("FwCommunication","Replica","0",GetAdv97())
917             EndIf
918
919             If !Empty(cAuxIP) .And. !Empty(cAuxPort) .And. !Empty(cAuxEnv)
920                 oHostAux := FwHostCreate(cAuxIP,cAuxPort,cAuxEnv,2)
921                 oConn := FwHostConnection():New( oHostAux )
922                 If oConn:connect()
923                     oConn:oCon:startJob("POSstartSchd",.F.,{cEmpAnt, cFilAnt, PadR(GetPvProfString("FwCommunication","ID","",GetAdv97()),19), __cUserID, cAuxEnv,
924                     PadR(GetPvProfString("FwCommunication","PREPAREIN","",GetAdv97()),19)})
925                     oConn:disconnect()
926                 EndIf
927                 FreeObj(oConn)
928                 FreeObj(oHostAux)
929             EndIf
930         EndIf
931     EndIf

```

```

931         lRet := .T.
932         MsgInfo(STR0043) // "Host configurado com sucesso"
933         lYesNo := .F.
934     Else
935         lYesNo := MsgYesNo(STR0045 + CRLF + "Tentar novamente?")
936     EndIf
937 End
938 lWizard := .F.
939 Return lRet
940
941 //-----
942 /*/{Protheus.doc} VldEmpFil
943 Valida o browse de empresa filial
944
945 @param   oGD Referência do objeto da GetDados
946 @param   nDirection Direção da troca ( < 0, para cima, > 0 para baixo)
947
948 */
949 //-----
950 Static Function VldEmpFil(aBrowse,aEmpFil)
951 Local nI := 0
952 Local lRet := .F.
953
954 For nI := 1 to len(aBrowse)
955     If ( aBrowse[nI][1] == "S" )
956         aADD(aEmpFil,aBrowse[nI][2])
957         aADD(aEmpFil,aBrowse[nI][3])
958     EndIf
959 Next nI
960 If LEN(aEmpFil) > 0
961     lRet := .T.
962 Else
963     Help( ,, 'HELP',, STR0097, 1, 0)
964 EndIf
965 Return lRet
966
967 //-----
968 /*/{Protheus.doc} TrocaLinha
969 Troca a linha de um elemento da GetDados em uma direção
970
971 @param   oGD Referência do objeto da GetDados
972 @param   nDirection Direção da troca ( < 0, para cima, > 0 para baixo)
973
974 */
975 //-----
976 Static Function TrocaLinha(oGD,nDirection)
977
978 Local nLinAtu := oGd:oBrowse:nAt
979 Local nMax    := Len(oGd:aCols)
980 Local aBkp    := Nil
981
982 If nDirection < 0 .AND. nLinAtu > 1

```

```

983 //Move linha para cima
984 aBkp := aClone(oGd:aCols[nLinAtu-1])
985 aCols[nLinAtu-1] := aClone(aCols[nLinAtu])
986 aCols[nLinAtu] := aClone(aBkp)
987 ElseIf nDirection > 0 .AND. nLinAtu < nMax
988 //Move linha para baixo
989 aBkp := aClone(oGd:aCols[nLinAtu+1])
990 aCols[nLinAtu+1] := aClone(aCols[nLinAtu])
991 aCols[nLinAtu] := aClone(aBkp)
992 EndIf
993
994 aBkp := Nil
995 oGd:Refresh()
996
997 Return Nil
998
999 //-----
1000 /*{Protheus.doc} PrepareGrids
1001 Monta os arrays aCols e aHeader utilizados nas duas Getdados do Wizard
1002
1003 @param aHeadSrv aHeader de protocolos do host pai
1004 @param aHeadLoc aHeader de protocolos do host local
1005 @param aColsSrv aCols de protocolos do host pai
1006 @param aColsLoc aCols de protocolos do host local
1007
1008 */
1009 //-----
1010 Static Function PrepareGrids(aHeadSrv, aHeadLoc, aColsSrv, aColsLoc, aBrwFun)
1011
1012 Local aFieldsXXX:= FWXXXFields()
1013 Local nX := 0
1014 Local nCols := 0
1015 Local cCombo := ""
1016 Local cPort := ""
1017 Local lEnable := .F.
1018 Local aBkp := {}
1019 // Criacao do aHeader dos protocolos do superior
1020 For nX := 1 to Len(aFieldsXXX)
1021 If aFieldsXXX[nX][1] <> "XXX_ID"
1022 cCombo := ""
1023 If !Empty(aFieldsXXX[nX][12])
1024 aEval(aFieldsXXX[nX][12],{|c0pc|cCombo += c0pc+";" })
1025 cCombo := Substr(cCombo,1,Len(cCombo)-1)
1026 EndIf
1027 AAdd(aHeadSrv,{ aFieldsXXX[nX][2], aFieldsXXX[nX][1], aFieldsXXX[nX][6], aFieldsXXX[nX][4], aFieldsXXX[nX][5],;
1028 " ", "????????????? ", aFieldsXXX[nX][3], " ", "R", cCombo })
1029 EndIf
1030 Next nX
1031
1032 //Ajusta o nome apresentado em tela
1033 aHeadSrv[aScan(aHeadSrv,{|x|AllTrim(x[2])=="XXX_ENV"})][1] := STR0016 + " / URL(Web Service)"
1034

```

```

1035 //Criacao do aCols que listara os protocolos do no superior
1036 nCols := Len(aHeadSrv)
1037 AAdd(aColsSrv,Array(nCols+1))
1038 For nX := 1 to Len(aHeadSrv)
1039     aColsSrv[1][nX] := Space(aHeadSrv[nX][4])
1040 Next nX
1041 aColsSrv[1][nCols+1] := .F.
1042
1043 //Criacao do aHeader dos protocolos locais
1044 AAdd(aHeadLoc,{"-","XXX_UP"      ,"@BMP",1,0,"","","C","","","","",""})
1045 AAdd(aHeadLoc,{"+","XXX_DOWN"    ,"@BMP",1,0,"","","C","","","","",""})
1046
1047 For nX :=1 to Len(aHeadSrv)
1048     If aHeadSrv[nX][2] <> "XXX_ORDER"
1049         AAdd(aHeadLoc,aClone(aHeadSrv[nX]))
1050     EndIf
1051 Next nX
1052 nCols := Len(aHeadLoc)
1053
1054 //Criacao do aCols que armazenara os protocolos locais
1055 AAdd(aColsLoc,Array(nCols+1))
1056
1057 aColsLoc[1][1] := "UP3"
1058 aColsLoc[1][2] := "DOWN3"
1059 aColsLoc[1][nCols+1] := .F.
1060
1061 For nX := 3 to Len(aHeadLoc)
1062     aColsLoc[1][nX] := Space(aHeadLoc[nX][4])
1063 Next nX
1064
1065 For nX := 1 to 3
1066     If nX > 1
1067         AAdd(aColsLoc,aClone(aColsLoc[1]))
1068     EndIf
1069     aColsLoc[nX][aScan(aHeadLoc,{|x| AllTrim(x[2])="XXX_PROT0C"})] := cValToChar(nX)
1070
1071     Do Case
1072         Case nX == CON_HTTP
1073             lEnable := GetPvProfString("HTTP","ENABLE","",GetAdv97()) == "1"
1074             cPort   := GetPvProfString("HTTP","PORT","",GetAdv97())
1075         Case nX == CON_RPC
1076             lEnable := .T.
1077             cPort   := GetPvProfString(GetPvProfString("DRIVERS","ACTIVE","",GetAdv97()),"PORT","",GetAdv97())
1078             aColsLoc[nX][aScan(aHeadLoc,{|x| AllTrim(x[2])="XXX_ENV"})] := GetEnvServer()
1079         Case nX == CON_WS
1080             lEnable := .F.
1081             cPort   := ""
1082     EndCase
1083
1084     aColsLoc[nX][aScan(aHeadLoc,{|x| AllTrim(x[2])="XXX_ENABLE"})] := Iif(lEnable,"1","2")
1085     aColsLoc[nX][aScan(aHeadLoc,{|x| AllTrim(x[2])="XXX_PORT"})]   := cPort
1086

```

```

1087     Next nX
1088
1089     //Inverte as duas primeiras posições de modo a trazer o RPC como primeira opção
1090     aBkp := aClone(aColsLoc[2])
1091     aColsLoc[2] := aClone(aColsLoc[1])
1092     aColsLoc[1] := aClone(aBkp)
1093     aBkp := Nil
1094
1095     If Select('XA2') == 0
1096         FWOpenXA2()
1097     EndIf
1098     // array com funcionalidades
1099     If Select('XXY') == 0
1100         FWOpenXXY()
1101     EndIf
1102     aBrwFun := {}
1103     XXY->(DbSetOrder(1))
1104     XXY->(DbGoTop())
1105     XXY->(Dbeval({|| Aadd(aBrwFun,{.F.,XXY->XXY_ID,XXY->XXY_DESC})}))
1106
1107 Return Nil
1108
1109 //-----
1110 /*{Protheus.doc} TestConnection
1111 Testa a conexão e recupera o código do host a ser criado.
1112
1113 @param   cIP Ip do host pai
1114 @param   cPort Porta a ser utilizada na conexão
1115 @param   cEnv Environment do RPC utilizado na conexão
1116 @param   nConnType Id do protocolo utilizado (RPC, HTTP ou WS)
1117 @param   oFatherHost Referência do objeto do host pai
1118 @param   oGetDSrv Referência do objeto da getdados de protocolos do host pai
1119
1120 */
1121 //-----
1122 Static Function TestConnection(cIP,cPort,cEnv,nConnType,oFatherHost,oGetDSrv)
1123
1124 Local lRet      := .F.
1125
1126 Local oSerial    := nil
1127
1128 Local aProtocols := {}
1129 Local aRefLine   := {}
1130
1131 Local nX          := 0
1132 Local nPOrder     := aScan(oGetDSrv:aHeader,{|x|AllTrim(x[2]) == "XXX_ORDER" })
1133 Local nPEnabled   := aScan(oGetDSrv:aHeader,{|x|AllTrim(x[2]) == "XXX_ENABLE" })
1134 Local nPProtocol  := aScan(oGetDSrv:aHeader,{|x|AllTrim(x[2]) == "XXX_PROTOC" })
1135 Local nPPort      := aScan(oGetDSrv:aHeader,{|x|AllTrim(x[2]) == "XXX_PORT" })
1136 Local nPEnv       := aScan(oGetDSrv:aHeader,{|x|AllTrim(x[2]) == "XXX_ENV" })
1137
1138 Local oConn       := nil

```

```

1139 Local oHost      := FwHost():New()
1140 Local oProtocol := FwProtocol():New()
1141 Local oErro      := ErrorBlock({|e| ConnError(e)})
1142
1143 Local cSerial
1144
1145 If !Empty(cIP) .and. (nConnType <> CON_RPC .or. !Empty(cEnv))
1146
1147     oProtocol:setEnvironment(AllTrim(cEnv))
1148     oProtocol:setProtocolId(nConnType)
1149     oProtocol:setPort(AllTrim(cPort))
1150
1151     oHost:setIp(AllTrim(cIP))
1152     oHost:addProtocol(oProtocol)
1153
1154     oConn := FwHostConnection():New( oHost )
1155
1156     Begin Sequence
1157
1158         oConn:setRmtCompany(.T.)
1159         If oConn:connect()
1160
1161             oSerial := FwHost():new()
1162             cSerial := oConn:runMethod(oSerial,"getSerial")
1163             oConn:disconnect()
1164             FreeObj(oConn)
1165             oSerial:Deserializer(cSerial)
1166
1167             aProtocols:= oSerial:getAllProtocols()
1168             aProfile := oSerial:getAllProfiles()
1169             If Len(aProtocols) > 0
1170
1171                 aRefLine := aClone(oGetDSrv:aCols[1])
1172                 oGetDSrv:aCols := {}
1173
1174                 For nX := 1 to Len(aProtocols)
1175                     aadd(oGetDSrv:aCols,aClone(aRefLine))
1176                     oGetDSrv:aCols[nX][nPOrder]      := cValToChar(nX)
1177                     oGetDSrv:aCols[nX][nPEnabled]     := iif(aProtocols[nX]:isEnabled(),"1","2")
1178                     oGetDSrv:aCols[nX][nPProtocol]   := cValToChar(aProtocols[nX]:getProtocolId())
1179                     oGetDSrv:aCols[nX][nPPort]       := aProtocols[nX]:getPort()
1180                     oGetDSrv:aCols[nX][nPEnv]        := aProtocols[nX]:getEnvironment()
1181                 Next nX
1182
1183                 oGetDSrv:GoTo(1)
1184                 oGetDSrv:Refresh()
1185
1186             EndIf
1187
1188             // testa novamente a conexão mas com a configuração enviada pelo superior
1189             oConn := FwHostConnection():New(oSerial)
1190             If oConn:connect()

```

```

1191         oConn:disconnect()
1192         oFatherHost := oSerial
1193         lRet := .T.
1194     Else
1195         Help( , , 'HELP',, STR0045+CRLF+STR0101+CRLF+STR0099+ ' '+oSerial:getIP()+CRLF+oSerial:getIPSecondary(), 1, 0) //"Não foi possível estabelecer conexão com o
host" ### "Os dados informados diferem da configurada no host superior" ### "Endereço IP"
1196         FreeObj(oSerial)
1197     EndIf
1198     FreeObj(oConn)
1199
1200     Else
1201         Help( , , 'HELP',, STR0045, 1, 0) //"Não foi possível estabelecer conexão com o host"
1202     EndIf
1203
1204     End Sequence
1205     ErrorBlock(oErro)
1206 EndIf
1207
1208 FreeObj(oHost)
1209 oHost := Nil
1210 Return lRet
1211
1212 //-----
1213 /*/{Protheus.doc} ConnError
1214 Funcao acionada caso a conexao provoque uma exceção
1215
1216 @param   e Objeto do erro
1217
1218 */
1219 //-----
1220 Static Function ConnError(e)
1221
1222 If e:gencode > 0
1223     Help( , , 'HELP',, STR0046 + CRLF + e:Description, 1, 0) // "Erro ao estabelecer a conexão:"
1224     BREAK
1225 EndIf
1226 Return
1227 //-----
1228 /*/{Protheus.doc} VldLocal
1229 Validação da configuracao local
1230
1231 @param oGetDLoc Referência do objeto da GetDados
1232
1233 @return lRet Indica se o wizard foi validado
1234
1235 */
1236 //-----
1237 Static Function VldLocal(oGetDLoc,cDesc,cIP,nPort,cEnv)
1238
1239     Local i
1240     Local nPosPrt      := aScan(oGetDLoc:aHeader,{|x| AllTrim(x[2]) == "XXX_PORT"      })
1241     Local nPosProt     := aScan(oGetDLoc:aHeader,{|x| AllTrim(x[2]) == "XXX_PROT0C"     })

```

```

1242 Local nPosEnv      := aScan(oGetDLoc:aHeader,{|x| AllTrim(x[2]) == "XXX_ENV"      })
1243 Local nPosEnable   := aScan(oGetDLoc:aHeader,{|x| AllTrim(x[2]) == "XXX_ENABLE"   })
1244 Local nX           := 0
1245 Local cMsg         := ''
1246 Local oRPC
1247 Local aRPCInfo
1248 Local aInfo
1249 Local bError
1250 Local nLen:=0
1251
1252 If Empty(cDesc) .or. Empty(cIP)
1253   cMsg := STR0098 // "Informe o Nome e Endereço IP"
1254 Else
1255   // valida IP informado
1256   oRPC := TRPC():New(AllTrim(cEnv))
1257   CursorWait()
1258   If oRPC:Connect(AllTrim(cIP), nPort)
1259     bError := ErrorBlock({|e| RPCHandle(e), cMsg := STR0100}) // "Endereço IP inválido"
1260     BEGIN SEQUENCE
1261     aRPCInfo := oRPC:CallProc("GetSrvInfo")
1262     END SEQUENCE
1263     ErrorBlock(bError)
1264
1265     oRPC:Disconnect()
1266     FreeObj(oRPC)
1267
1268     If Empty(cMsg)
1269       aInfo := &("GetSrvInfo()") //Add Macro execução para evitar erro
1270       nLen:=Len(aInfo)
1271       iif(nLen>8,nLen:=8,)
1272       For i := 1 To nLen
1273         If aInfo[i] != aRPCInfo[i]
1274           cMsg := STR0100 // "Endereço IP inválido"
1275           Exit
1276         EndIf
1277       Next
1278       Asize(aRPCInfo,0)
1279       Asize(aInfo,0)
1280       EndIf
1281       CursorArrow()
1282     Else
1283       CursorArrow()
1284       cMsg := STR0100 // "Endereço IP inválido"
1285     EndIf
1286
1287     If Empty(cMsg)
1288       //Valida se as portas foram informadas
1289       For nX := 1 to Len(oGetDLoc:aCols)

```

```

1294         If oGetDLoc:aCols[nX][nPosEnable] == "1"
1295             If Empty(oGetDLoc:aCols[nX][nPosPrt]) .AND. Val(oGetDLoc:aCols[nX][nPosProt]) <> CON_WS
1296                 cMsg := I18n(STR0047,{cValToChar(nX)})
1297                 Exit
1298             EndIf
1299             If Val(oGetDLoc:aCols[nX][nPosProt]) == CON_RPC .AND. Empty(oGetDLoc:aCols[nX][nPosEnv])
1300                 cMsg := I18n(STR0048,{cValToChar(nX)})
1301                 Exit
1302             EndIf
1303             If Val(oGetDLoc:aCols[nX][nPosProt]) == CON_WS .AND. !("http://" $ Lower(oGetDLoc:aCols[nX][nPosEnv]))
1304                 cMsg := I18n(STR0049 +" http://127.0.0.1/ws",{cValToChar(nX)})
1305                 Exit
1306             EndIf
1307         EndIf
1308     Next nX
1309 EndIf
1310 EndIf
1311
1312 If !Empty(cMsg)
1313     Help(,, 'HELP',, cMsg, 1, 0)
1314     Return .F.
1315 EndIf
1316 Return .T.
1317
1318 Static Function VldParent(cIP, lOk, cProfile)
1319 Local lRet := .T.
1320
1321 If Empty(cIP) .or. !lOk
1322     Help(,, 'HELP',, STR0050, 1, 0)
1323     lRet := .F.
1324 ElseIf Empty(cProfile)
1325     Help(,, 'HELP',, STR0051, 1, 0)
1326     lRet := .F.
1327 EndIf
1328 Return lRet
1329
1330 //-----
1331 /*/{Protheus.doc} VldServers
1332 Função responsável por validar a conexão com o servidor auxiliar
1333 e seus possíveis erros de configuração
1334
1335 @param cIPAux          IP do servidor auxiliar
1336 @param cPortAux        Porta do servidor auxiliar
1337 @param cEnvAux         Ambiente do servidor auxiliar
1338 @param oConnSrv        Objeto TSAY que atualiza o log de status
1339 @param cConnStatus     Variável que recebe o conteúdo do status da conexão
1340 @param lUpdate         Indica se será salvo no .ini
1341
1342 @return clRet          Sucesso ou falha de conexão
1343
1344 */
1345 //-----

```

```

1346 Static Function VldServers( cIPAux, cPortAux, cEnvAux, oConnSrv, cConnStatus, lUpdate , nIPType )
1347 Local oRPC
1348 Local aAuxInfo := {}
1349 Local lRet := .F.
1350 Local cInstallPath := AllTrim(UPPER(GetPvProfString("GENERAL","INSTALLPATH","",GetAdv97())))
1351 Local cService := AllTrim(UPPER(GetPvProfString("SERVICE","NAME","",GetAdv97())))
1352 Local cUpdate := ""
1353 Local oErro
1354 Local cPortMain := AllTrim(GetPvProfString("TCP","PORT","0",GetAdv97()))
1355
1356 Default cIPAux := GetPvProfString("FwCommunication","auxip","",GetAdv97())
1357 Default cPortAux := GetPvProfString("FwCommunication","auxport","",GetAdv97())
1358 Default cEnvAux := GetPvProfString("FwCommunication","auxenv","",GetAdv97())
1359 Default lUpdate := .T.
1360
1361 // cria .bat para para serviços do server
1362 FwHostBatch(.T.)
1363 FwHostBatch(.F.)
1364
1365 lConnVld := .F.
1366 cIPAux := ALLTRIM(UPPER(cIPAux))
1367 cPortAux := ALLTRIM(cPortAux)
1368 cEnvAux := ALLTRIM(cEnvAux)
1369 cConnStatus := ""
1370 If !Empty(cIPAux) .AND. !Empty(cPortAux) .AND. !Empty(cEnvAux)
1371     If ALLTRIM(cPortMain) != ALLTRIM(cPortAux)
1372         If ExistDir(cInstallPath) .AND. !Empty(cService)
1373             oErro := ErrorBlock({|e| RPHandle(e,@cConnStatus,oConnSrv)})
1374             Begin Sequence
1375                 oRPC := TRPC():New( cEnvAux )
1376                 If oRPC:Connect( cIPAux,VAL(cPortAux) )
1377                     aAuxInfo := oRPC:CallProc("__LjVldSrvAux",cInstallPath,cService,nIPType,lUpdate)
1378                     oRPC:Disconnect()
1379                     If Empty(aAuxInfo)
1380                         If Empty(cConnStatus)
1381                             cConnStatus := STR0070 + " [SERVICE] - NAME " + STR0077 + " [GENERAL] - INSTALLPATH "+ STR0071
1382                         EndIf
1383                         If oConnSrv != NIL
1384                             oConnSrv:nClrText := CLR_RED
1385                         Else
1386                             conout(cConnStatus)
1387                         EndIf
1388                     Else
1389                         If Empty(cConnStatus)
1390                             Do Case
1391                                 Case !aAuxInfo[3] .AND. !(File(cInstallPath+"\\appserver\\downserver.bat") .AND. File(cInstallPath+"\\appserver\\upserver.bat"))
1392                                     cConnStatus := STR0072 + ' <font color="#000000">(' + STR0073 + ')</font>'
1393                                 Case !aAuxInfo[3] .AND. File(cInstallPath+"\\appserver\\downserver.bat") .AND. File(cInstallPath+"\\appserver\\upserver.bat")
1394                                     cConnStatus := STR0072 + ' <font color="#000000">(' + STR0074 + ')</font>'
1395                                 Case !(File(cInstallPath+"\\appserver\\downserver.bat") .AND. File(cInstallPath+"\\appserver\\upserver.bat")) .AND. aAuxInfo[3]
1396                                     cConnStatus := STR0072 + ' <font color="#000000">(' + STR0075 + ')</font>'
1397                                 Otherwise

```

```

1398             cConnStatus := STR0080
1399         EndCase
1400     EndIf
1401     If oConnSrv != NIL
1402         oConnSrv:nClrText := CLR_GREEN
1403     Else
1404         conout(cConnStatus)
1405     EndIf
1406     If !Update
1407         WritePProString("FWCOMMUNICATION", "BinPath", ALLTRIM(UPPER(aAuxInfo[1])), GetAdv97() )
1408         WritePProString("FWCOMMUNICATION", "Service", ALLTRIM(UPPER(aAuxInfo[2])), GetAdv97() )
1409         WritePProString("FWCOMMUNICATION", "auxip", ALLTRIM(UPPER(cIPAux)), GetAdv97() )
1410         WritePProString("FWCOMMUNICATION", "auxport", ALLTRIM(UPPER(cPortAux)), GetAdv97() )
1411         WritePProString("FWCOMMUNICATION", "auxenv", ALLTRIM(UPPER(cEnvAux)), GetAdv97() )
1412     else
1413         aSrvAux := {cIPAux,cPortAux,cEnvAux}
1414         lConnVld := .T.
1415     EndIf
1416     lRet := .T.
1417 EndIf
1418 Else
1419     If Empty(cConnStatus)
1420         cConnStatus := STR0076
1421     EndIf
1422     If oConnSrv != NIL
1423         oConnSrv:nClrText := CLR_RED
1424     Else
1425         conout(cConnStatus)
1426     EndIf
1427 EndIf
1428 End Sequence
1429 ErrorBlock(oErro)
1430 Else
1431     cConnStatus := STR0070 + " [SERVICE] - NAME " + STR0077 + " [GENERAL] - INSTALLPATH "+ STR0078
1432     If oConnSrv != NIL
1433         oConnSrv:nClrText := CLR_RED
1434     Else
1435         conout(cConnStatus)
1436     EndIf
1437 EndIf
1438 Else
1439     cConnStatus := STR0079
1440     If oConnSrv != NIL
1441         oConnSrv:nClrText := CLR_RED
1442     Else
1443         conout(cConnStatus)
1444     EndIf
1445 EndIf
1446 Else
1447     cConnStatus := STR0081
1448     If oConnSrv != NIL
1449         oConnSrv:nClrText := CLR_RED

```

```

1450     Else
1451         conout(cConnStatus)
1452     EndIf
1453 EndIf
1454 Return lRet
1455
1456 //-----
1457 /*{Protheus.doc} __LjVldSrvAux
1458 Função chamada por RPC para validar o servidor auxiliar
1459
1460 @param cText cInstallPath+"*"+cService+"*"+cValidBat
1461
1462 @return cSrvInfo cInstallPathAux+"*"+cServiceAux+"*"+cValidBatAux
1463
1464 */
1465 //-----
1466 Function __LjVldSrvAux(cMainPath,cMainService,nIPType,lUpdate)
1467 Local aSrvInfo := {}
1468 Local cInstallPath := AllTrim(UPPER(GetPvProfString("GENERAL","INSTALLPATH","",GetAdv97())))
1469 Local cService := AllTrim(UPPER(GetPvProfString("SERVICE","NAME","",GetAdv97())))
1470 Local cIni
1471
1472 // cria .bat para para serviços do server
1473 FwHostBatch(.T.)
1474 FwHostBatch(.F.)
1475
1476 If ExistDir(cInstallPath,1) .AND. !Empty(cService)
1477     If lUpdate
1478         cIni := GetAdv97()
1479         WritePProString("FWCOMMUNICATION", "BinPath", cMainPath, cIni )
1480         WritePProString("FWCOMMUNICATION", "Service", cMainService, cIni )
1481         If nIPType != NIL
1482             WritePProString("FWCOMMUNICATION", "IPType", cValToChar(nIPType), cIni )
1483         EndIf
1484     EndIf
1485
1486     Asize(aSrvInfo,3)
1487     aSrvInfo[1] := cInstallPath
1488     aSrvInfo[2] := cService
1489     aSrvInfo[3] := File(cInstallPath+"\appserver\downserver.bat",-1) .AND. File(cInstallPath+"\appserver\upserver.bat",-1)
1490 EndIf
1491 Return aSrvInfo
1492
1493 //-----
1494 /*{Protheus.doc} RPCHandle
1495 Desvia os erros de RPC
1496
1497 @param e           Objeto de erro
1498 @param cConnStatus Variável texto que receberá o conteúdo
1499 @param oConnSrv    Objeto do TSay
1500
1501 @return lRet Indica se a alteração pode prosseguir

```

```

1502
1503 /*/
1504 //-----
1505 Static Function RPCHandle(e,cConnStatus,oConnSrv)
1506 Default cConnStatus := ""
1507     If e:gencode > 0
1508         cConnStatus := STR0088
1509         If oConnSrv != NIL
1510             oConnSrv:nClrText := CLR_RED
1511         Else
1512             conout(cConnStatus)
1513         EndIf
1514     EndIf
1515 Return
1516
1517 //-----
1518 /*/{Protheus.doc} CanUpdate
1519 Indica se a alteracao e permitida no host
1520
1521 @param oModel Referencia do Model ativo
1522
1523 @return lRet Indica se a alteração pode prosseguir
1524
1525 /*/
1526 //-----
1527 Static Function CanUpdate(oModel)
1528
1529     Local nOperation      := oModel:GetOperation()
1530     Local lRet            := .T.
1531
1532     If !IsBlind() .AND. !lWizard
1533         If (nOperation == MODEL_OPERATION_UPDATE) .AND. !( AllTrim(XXP->XXP_ID) == cHostID )
1534             lRet := .F.
1535             Help( ,, 'Help',,STR0052, 1, 0 )
1536         EndIf
1537     EndIf
1538
1539 Return lRet
1540
1541 //-----
1542 /*/{Protheus.doc} RunHostUpdate
1543 Dispara a atualização nos hosts remotos quanto às novas configurações
1544
1545 /*/
1546 //-----
1547 Static Function RunHostUpdate()
1548
1549     Local lRet            := .T.
1550     Local nX              := 0
1551     Local aBkp            := {}
1552     Local aProtocols      := {}
1553     Local oModel          := FwModelActive()

```

```

1554 Local oModelXXX      := Nil
1555 Local oModelXXP      := Nil
1556 Local oLocalHost     := Nil
1557 Local oFatherHost    := Nil
1558
1559 aBkp      := FwSaveRows()
1560 oModelXXX := oModel:getModel('XXXDETAIL')
1561 oModelXXP := oModel:getModel('XXPMaster')
1562
1563 //Monta o host atual
1564 oLocalHost := FwHost():New()
1565 oLocalHost:setId(AllTrim(oModelXXP:getValue("XXP_ID")))
1566 oLocalHost:setDescription(AllTrim(oModelXXP:getValue("XXP_DESC")))
1567 oLocalHost:setIP(AllTrim(oModelXXP:getValue("XXP_IP")))
1568 oLocalHost:setIPSecondary(AllTrim(oModelXXP:getValue("XXP_IPSEC")))
1569 oLocalHost:setBackupOf(AllTrim(oModelXXP:getValue("XXP_BKPID")))
1570 oLocalHost:setIdProfile(AllTrim(oModelXXP:getValue("XXP_PROFIL")))
1571
1572 oLocalHost:setLastUpdate(DtoS(Date()) + StrTran(Time(),":"))
1573 If oModelXXP:GetOperation() != MODEL_OPERATION_DELETE
1574     oModelXXP:setValue("XXP_UPDATE",oLocalHost:getLastUpdate())
1575 EndIf
1576 //Define o host pai
1577 If !Empty(oModelXXP:getValue("XXP_FATHER"))
1578     oFatherHost := FwHost():New()
1579     oFatherHost:load(oModelXXP:getValue("XXP_FATHER"))
1580     oLocalHost:setFather(oFatherHost)
1581 EndIf
1582
1583 //Descarga dos protocolos configurados
1584 For nX := 1 to oModelXXX:Length()
1585
1586     oModelXXX:goLine(nX)
1587
1588     AAdd(aProtocols,{ oModelXXX:getValue("XXX_ORDER")           ,;
1589                     Val(oModelXXX:getValue("XXX_PROTOC"))       ,;
1590                     oModelXXX:getValue("XXX_PORT")              ,;
1591                     oModelXXX:getValue("XXX_ENV")               ,;
1592                     oModelXXX:getValue("XXX_ENABLE") == "1"     })
1593
1594 Next nX
1595
1596 //Ordena protocolos por prioridade
1597 aSort(aProtocols,,,{|x,y| x[1] < y[1] })
1598
1599 For nX := 1 to Len(aProtocols)
1600
1601     oProtocol := FwProtocol():New()
1602
1603     oProtocol:setProtocolId( aProtocols[nX][2] )
1604     oProtocol:setPort( aProtocols[nX][3] )
1605     oProtocol:setEnvironment( aProtocols[nX][4] )

```

```

1606         oProtocol:setEnabled( aProtocols[nX][5] )
1607
1608         oLocalHost:addProtocol( oProtocol )
1609
1610     Next nX
1611
1612     //Aciona o mecanismo atualizador
1613     FwHostUpdater(cEmpAnt,cFilAnt,oLocalHost)
1614
1615     If oFatherHost <> Nil
1616         FreeObj(oFatherHost)
1617         oFatherHost := Nil
1618     EndIf
1619
1620     FreeObj(oLocalHost)
1621     oLocalHost := Nil
1622 Return .T.
1623
1624 Static Function ActionPanel(oPanel)
1625
1626 Local nWidth      := (oPanel:nClientWidth/2) - 2
1627
1628 @015,000 BUTTON STR0053      SIZE nWidth, 12 ACTION GridChange(-1)  PIXEL OF oPanel
1629 @030,000 BUTTON STR0054 SIZE nWidth, 12 ACTION GridChange(1)  PIXEL OF oPanel
1630
1631 Return Nil
1632
1633 //-----
1634 /*/{Protheus.doc} SrvAuxPanel
1635 Cria o painel adicional OTHEROBJECT de status do servidor auxiliar
1636
1637 */
1638 //-----
1639 Static Function SrvAuxPanel(oPanel)
1640 Local oFont
1641 Local oConnSrv
1642 Local nWidth      := (oPanel:nClientWidth/2) - 5
1643 Local nHeight     := (oPanel:nClientHeight/2) - 40
1644 Local cConnStatus := ""
1645
1646 @ 020,005 BUTTON STR0068      SIZE 50, 12 ACTION PreVldSrv(oConnSrv,@cConnStatus)/*FwMsgRun(Nil,{|lAux| lAux := VldServers( , , , oConnSrv, @cConnStatus, .F. )}},STR0069)*/
1647 oFont := TFont():New('Arial',-11,,.T.)
1648 oConnSrv := TSay():New(040,005,{|cConnStatus|,oPanel,,oFont,,,,.T.,CLR_GREEN,CLR_WHITE,nWidth,nHeight,,,,.T.)
1649
1650 Return Nil
1651
1652 //-----
1653 /*/{Protheus.doc} PreVldSrv
1654 Valida as informações do servidor auxiliar provenientes do MVC
1655
1656 */

```

```

1657 //-----
1658 Static Function PreVldSrv(oConnSrv,cConnStatus)
1659 Local oModel:=FWModelActive()
1660 Local oFieldSrv := oModel:GetModel("AUXDETAIL")
1661 If oFieldSrv != Nil
1662     FWMsgRun(Nil,{|lAux| lAux := VldServers( oFieldSrv:GetValue("AUX_IP"), oFieldSrv:GetValue("AUX_PORT"), oFieldSrv:GetValue("AUX_ENV"), oConnSrv, @cConnStatus, .F.
        )},/*'cHeader'*/,STR0069)
1663 EndIf
1664 Return Nil
1665
1666 //-----
1667 /*/{Protheus.doc} GridChange
1668
1669 Troca linha do Grid
1670
1671 */
1672 //-----
1673 Static Function GridChange( nDirection )
1674
1675 Local oModel      := FwModelActive()
1676 Local aRows       := Nil
1677 Local oView       := FwViewActive()
1678 Local oGrid       := oModel:GetModel( 'XXXDETAIL' )
1679 Local nLine       := oGrid:GetLine()
1680 Local nTam        := oGrid:Length()
1681 Local nTo         := 0
1682 Local cOrder1     := ""
1683 Local cOrder2     := ""
1684
1685 //Se for para cima e a linha for maior que 1
1686 If nDirection < 0 .AND. nLine > 1
1687     nTo := nLine-1
1688 //Se for para baixo e não for a última linha
1689 ElseIf nDirection > 0 .AND. nLine < nTam
1690     nTo := nLine+1
1691 EndIf
1692
1693 If nTo <> 0
1694     aRows := FwSaveRows()
1695
1696     cOrder1 := oGrid:GetValue( "XXX_ORDER", nTo )
1697     cOrder2 := oGrid:GetValue( "XXX_ORDER", nLine )
1698
1699     oGrid:LineShift( nLine, nTo )
1700
1701     oGrid:GoLine( nTo )
1702     oGrid:LoadValue( "XXX_ORDER", cOrder1 )
1703     oGrid:GoLine( nLine )
1704     oGrid:LoadValue( "XXX_ORDER", cOrder2 )
1705
1706     FwRestRows(aRows)
1707

```

```

1708     oView:Refresh('XXXDETAIL')
1709 EndIf
1710
1711 Return NIL
1712
1713 //-----
1714 /*{Protheus.doc} XXXLoad
1715 Carrega as linhas do grid da tabela XXX
1716
1717 @param oGrid Model do grid
1718
1719 */
1720 //-----
1721 Static Function XXXLoad(oGrid)
1722 Local aArea := GetArea()
1723 Local aRet := {}
1724 Local nOperation := oGrid:GetModel():GetOperation()
1725 Local oModel := oGrid:GetModel()
1726 Local cHost
1727 Local oAux
1728
1729 If nOperation != MODEL_OPERATION_INSERT
1730     oModel:GetIdField('XXP_ID',@oAux)
1731     cHost := oAux:GetValue('XXP_ID')
1732     DbSelectArea('XXX')
1733     DbSetOrder(1)
1734     DbSeek(cHost)
1735     While !Eof() .and. XXX->XXX_ID == cHost
1736         Add(aRet,{Recno(),{XXX->XXX_ID,XXX->XXX_ORDER,XXX->XXX_ENABLE,XXX->XXX_PROTOC,XXX->XXX_PORT,XXX->XXX_ENV}})
1737     DbSkip()
1738     End
1739 EndIf
1740
1741 RestArea(aArea)
1742 Return aRet
1743
1744 //-----
1745 /*{Protheus.doc} XXPValid
1746 Valida os campos do grid da tabela XXP
1747
1748 @param oGrid Model do grid
1749 @param cField Campo a ser validade
1750 @param xValue Novo valor
1751 @param nLine Numero da linha do grid
1752 @param xOldValue Valor anterior
1753
1754 */
1755 //-----
1756 Static Function XXPValid(oModel,cField,xValue,xOldValue)
1757 Local lRet := .T.
1758 Local aIP
1759 Local nLen

```

```

1760 Local i,j
1761
1762 If xValue != xOldValue
1763     If cField == 'XXP_IP'
1764         aIP := StrTokArr(xValue, '.')
1765         nLen := Len(aIP)
1766         If nLen != 4
1767             lRet := .F.
1768         Else
1769             For i := 1 To nLen
1770                 If i == nLen
1771                     aIP[i] := Rtrim(aIP[i])
1772                 EndIf
1773                 If Len(aIP[i]) > 3
1774                     lRet := .F.
1775                     Exit
1776                 Else
1777                     For j := 1 To Len(aIP[i])
1778                         If !IsDigit(Subs(aIP[i],j,1))
1779                             lRet := .F.
1780                             Exit
1781                         EndIf
1782                     Next
1783                 EndIf
1784             EndIf
1785             If !lRet
1786                 Exit
1787             EndIf
1788         Next
1789     EndIf
1790 EndIf
1791 EndIf
1792 Return lRet
1793
1794 //-----
1795 /*{Protheus.doc} XXYLookUp
1796 Consulta da tabela XXY
1797
1798 */
1799 //-----
1800 Static oXA2LookUp
1801 Function LJXA2LookUP()
1802     Local oStruct
1803
1804     If Select('XA2') == 0
1805         FWOpenXA2()
1806     EndIf
1807     If oXA2LookUp == NIL
1808         oXA2LookUp := FWLookUp():New()
1809         oStruct := FWLookUPStruct():New()
1810         oStruct:AddTable("XA2",{ 'XA2_ID' },STR0055)
1811         oStruct:AddIndex(1, "01", 'XA2_ID') //"Código"

```

```

1812      oStruct:AddField(1, 'ID', "XA2_ID", "C" , 06, 0) //"Código"
1813      oStruct:AddField(1, STR0083, "XA2_DESC", "C" , 100, 0) //"Descrição"
1814      oStruct:SetReturnsFields({'XA2->XA2_ID'}) //Array com o numero da coluna que deve ser retornado
1815      oStruct:SetFilter('XA2->XA2_MASTER != 0')
1816      FWMakeLookup('XA2',STR0055,oStruct)
1817  EndIf
1818  Return 'XA2'
1819
1820  //-----
1821  /*/{Protheus.doc} ConsProfile
1822  Cria a consulta padrão do perfil
1823
1824  @param cProfile      ID do Perfil
1825
1826  */
1827  //-----
1828  Static Function ConsProfile(cProfile)
1829  Local oOK
1830  Local oDlg
1831  Local oBrowse
1832  Local oColumn
1833  Local oPanelBottom
1834  Local oPanelGrid
1835  Local nAt      := 0
1836  Local aSeek    := {}
1837  Local aProfiles := {}
1838  Local nI
1839  Local oProfile
1840  Local aBkpProfiles := {}
1841
1842  For nI:=1 TO LEN(aProfile)
1843    If aProfile[nI]:getIsMaster() < 1
1844      aADD(aProfiles,{aProfile[nI]:getId(),aProfile[nI]:getDescription()})
1845    EndIf
1846  Next
1847  If LEN(aProfiles) > 0
1848    aBkpProfiles := aClone(aProfiles)
1849    DEFINE MSDIALOG oDlg TITLE STR0055 FROM 178,0 TO 543,800 PIXEL //"Papel de Trabalho"
1850    oPanelBottom := TPanel():New(0,0,"",oDlg,,,,,0,20)
1851    oPanelBottom:Align := CONTROL_ALIGN_BOTTOM
1852
1853    oPanelGrid := TPanel():New(0,0,"",oDlg,,,,,0,0)
1854    oPanelGrid:Align := CONTROL_ALIGN_ALLCLIENT
1855
1856    //-----
1857    // Chave de pesquisa do Seek
1858    //-----
1859    Add( aSeek, { STR0056 , {{"","C",06,0,STR0056}} } ) //"Código"
1860
1861    DEFINE FWBROWSE oBrowse DATA ARRAY ARRAY aProfiles SEEK ORDER aSeek NO REPORT DOUBLECLICK {|| nAt := oBrowse:At(), oDlg:End() } OF oPanelGrid
1862    ADD COLUMN oColumn DATA { || aProfiles[oBrowse:At(),1] } TITLE STR0056 SIZE 10 OF oBrowse //"Código"
1863    ADD COLUMN oColumn DATA { || aProfiles[oBrowse:At(),2] } TITLE STR0009 SIZE 50 OF oBrowse //"Descrição"

```

```

1864      ACTIVATE FWBROWSE oBrowse
1865
1866      @ 5,09 Button oOK PROMPT "OK" Size 037,012 PIXEL OF oPanelBottom ACTION (nAt := oBrowse:At(),oDlg:End())
1867      @ 5,49 Button STR0057 Size 037,012 PIXEL OF oPanelBottom ACTION oDlg:End() //"Cancelar"
1868      ACTIVATE MSDIALOG oDlg CENTERED
1869      If nAt > 0 .AND. LEN(aBkpProfiles) > 0
1870          cProfile := aBkpProfiles[nAt,1]
1871      EndIf
1872      aBkpProfiles := aSize(aBkpProfiles,0)
1873 EndIf
1874
1875 Return
1876
1877 //-----
1878 /*{Protheus.doc} CleanProfXFunc
1879 Limpa as tabelas de perfil, funcionalidades e relacionamentos
1880
1881 */
1882 //-----
1883 Static Function CleanProfXFunc()
1884 Local nX := 1
1885 Local aAlias := {"XA2","XA5","XXZ","XXY"}
1886 For nX:=1 TO LEN(aAlias)
1887     (aAlias[nX])->(DbGoTop())
1888     While !(aAlias[nX])->(Eof())
1889         If (aAlias[nX])->(RLock())
1890             (aAlias[nX])->(DbDelete())
1891         EndIf
1892         (aAlias[nX])->(DbSkip())
1893     End
1894 Next
1895 Return
1896
1897 Static Function FWXXPValid(nType)
1898 Local lAction := .T.
1899 Local cDriver := RddName()
1900 Local oModel := FWModelActive()
1901 Local aFields := {}
1902 Local oFields
1903 Local lReturn := .T.
1904
1905 #IFDEF WAXS
1906     cDriver := "DBFCDXAX"
1907 #ENDIF
1908 #IFDEF TOP
1909     cDriver := "TOPCONN"
1910 #ENDIF
1911
1912 If !("TOP" $ cDriver) .AND. File("\system\xxp.dbf")
1913     If Select("XXP") == 0
1914         FWOpenXXP()
1915         //Use &("XXP") Alias &("XXP") SHARED NEW Via cDriver

```

```

1916 EndIf
1917
1918 //cID := PadR(GetPvProfString("FwCommunication","ID","",GetAdv97()),19)
1919 cID := oModel:GetValue("XXPMaster","XXP_ID")
1920 XXP->(DbSetOrder(1))
1921 If XXP->(DbSeek(cID,.T.))
1922     If ALLTRIM(XXP->XXP_ID) == ALLTRIM(cID) .AND. !Empty(XXP->XXP_FATHER)
1923         lAction := .F.
1924     EndIf
1925 EndIf
1926 //XXP->(DbCloseArea())
1927 ElseIf "TOP" $ cDriver
1928     If Select("XXP") == 0
1929         FWOpenXXP()
1930     EndIf
1931 EndIf
1932 If oModel:GetOperation() == 4 .AND. nType == 1 //VALID
1933     If !lAction
1934         oField := oModel:GetModel("XXPMaster")
1935         aFields := oField:GetStruct():GetFields()
1936         If select("XA2") == 0
1937             FWOpenXA2()
1938         EndIf
1939         XA2->(DbSetOrder(1))
1940         If XA2->(DbSeek(oField:GetValue("XXP_PROFIL"),.T.))
1941             If XA2->XA2_MASTER > 0
1942                 lReturn := .F.
1943             Else
1944                 lReturn := .T.
1945             EndIf
1946         EndIf
1947     Else
1948         lFather := .T.
1949     EndIf
1950 ElseIf oModel:GetOperation() == 4 .AND. nType == 2 //bWHEN
1951     oField := oModel:GetModel("XXPMaster")
1952     aFields := oField:GetStruct():GetFields()
1953     If lAction
1954         aFields[aScan(aFields,{|x| x[3] == "XXP_PROFIL" } )][8] := {|| .F. }
1955         lReturn := .F.
1956     Else
1957         aFields[aScan(aFields,{|x| x[3] == "XXP_PROFIL" } )][8] := {|| .T. }
1958         lReturn := .T.
1959     EndIf
1960 EndIf
1961 Return lReturn
1962
1963 //-----
1964 // Valida o agendamento
1965 //-----
1966 Static Function VldSched(oFWUISchedulePersist,oSchedule)
1967 Local lRet := .F.

```

```

1968     oFWUISchedulePersist:Submit(.T., @oSchedule)
1969     If oSchedule != NIL
1970         lRet := .T.
1971     EndIf
1972 Return lRet
1973
1974 //-----
1975 // Configuração de agendamento de atualização
1976 //-----
1977 Static Function ConfigSched( uFWVOSchedule, cFunction, nModule , oOwner , cEnv)
1978     Local oFWUISchedulePersist := FWUISchedulePersist():new()
1979
1980     oFWUISchedulePersist:setButtonOkCancel(.F.)
1981     oFWUISchedulePersist:init( oOwner, .T., @uFWVOSchedule)
1982
1983     oFWUISchedulePersist:setFunctionCtrl( UPPER(ALLTRIM(cFunction)) )
1984     oFWUISchedulePersist:setModuleCtrl( nModule )
1985     oFWUISchedulePersist:setEnv(cEnv)
1986     oFWUISchedulePersist:setNoEndDate(.T.)
1987
1988 Return oFWUISchedulePersist
1989
1990 //-----
1991 /*{Protheus.doc} AuxLoad
1992 Efetua o Load das informações do Fields da conexão com o servidor auxiliar
1993
1994 */
1995 //-----
1996 Static Function AuxLoad(oField)
1997 Local oModel := oField:GetModel()
1998 Local oMaster := oModel:GetModel("XXPMMASTER")
1999 Local aFields := oField:GetStruct():GetFields()
2000 Local nI := 0
2001 Local aLoad := {}
2002 If !Empty(oMaster:getvalue("XXP_FATHER"))
2003     aADD(aLoad,{AllTrim(GetPvProfString("FwCommunication","auxip","","GetAdv97()")),AllTrim(GetPvProfString("FwCommunication","auxport","","GetAdv97()")),AllTrim(GetPvProfString(
        "FwCommunication","auxenv","","GetAdv97()))})
2004     aADD(aLoad,0)
2005 Else
2006     For nI:=1 TO LEN(aFields)
2007         aFields[nI][8] := {|| .F. }
2008     Next
2009 EndIf
2010 Return aLoad
2011
2012 //-----
2013 /*{Protheus.doc} FWCFGA050Commit
2014 Função responsável efetuar o commit da rotina
2015
2016 */
2017 //-----
2018 Static Function FWCFGA050Commit()

```

```

2019 Local oModel := FWModelActive()
2020 Local lRet := .F.
2021 Local nI
2022 Local lValid := .F.
2023 //Commit do MVC
2024
2025 If oModel:GetOperation() == 4
2026     If oModel:GetModel("AUXDETAIL"):IsModified()
2027         If lConnVld .AND. LEN(aSrvAux) > 2
2028             lValid := .T.
2029             FWMsgRun( Nil,{|| lAux := VldServers( aSrvAux[1], aSrvAux[2], aSrvAux[3], , , .T. )},/'cHeader'*/,STR0069)
2030         Else
2031             lValid := .F.
2032         EndIf
2033     Else
2034         lValid := .T.
2035     EndIf
2036     If lValid
2037         lRet := FWFormCommit(oModel)
2038     Else
2039         lRet := .F.
2040         oModel:SetErrorMessage(,,,,,STR0082)
2041     EndIf
2042 Else
2043     lRet := FWFormCommit(oModel)
2044 EndIf
2045 Return lRet
2046
2047 //-----
2048 /*/{Protheus.doc} MarkAllFunc
2049 Função responsável marcar ou desmarcar todas as funcionalidades
2050
2051 */
2052 //-----
2053 Static Function MarkAllFunc(oBrowse,aBrwFun)
2054 Local lMark := .T.
2055
2056 Default oBrowse := Nil
2057 Default aBrwFun := {}
2058
2059 //De acordo com o primeiro, marca todos ou desmarca todos
2060 If Len(aBrwFun) > 0
2061     lMark := !aBrwFun[1][1]
2062 EndIf
2063
2064 AEval(aBrwFun,{|x| x[1] := lMark})
2065
2066 oBrowse:Refresh(.T.)
2067
2068 Return .T.
2069
2070 //-----

```

```
2071 /*/{Protheus.doc} SaveUser
2072 Função responsável em salvar o usuario e senha a partir da função
2073 desenvolvida pelo framework
2074
2075 */
2076 //-----
2077 Static Function SaveUser(cUsuario, cSenha)
2078
2079     Local lRet      := .T.
2080     Local lCanUse    := ExistFunc("CanUsePDVAuth") .AND. ExistFunc("PDVSaveUsr") .AND. CanUsePDVAuth()
2081
2082     If lCanUse
2083         lRet := PDVSaveUsr(cUsuario, cSenha)
2084     EndIf
2085
2086 Return lRet
```

Framework/Message/STFMessage.PRW

543 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "FWCSS.CH"
5 #INCLUDE "STFMESSAGE.CH"
6
7
8 Static nNumMsg := SuperGetMv("MV_LJTLOG",,40) // Tamanho do Array que armazena as mensagens
9
10 Static aMessage := {} // Array de mensagens
11 Static aMsgBkp := Iif (nNumMsg == 40,Array(40),Array(nNumMsg)) // Array de mensagens de bkp para exibicao em tela quando necessario
12 Static lClientHtml := GetCliHTML()
13
14 //-----
15 /*{Protheus.doc} STFMessage
16 Recebe e armazena as mensagens que serao enviadas ao display no comando showmessage
17 @param cIdentify ID da funcao
18 @param cType Tipo de mensagem
19 @param cMessage Mensagem
20 @param bBlock Bloco de codigo
21 @return Nil
22 @obs
23 @sample
24 */
25 //-----
26 Function STFMessage( cIdentify , cType , cMessage , bBlock )
27
28 Local nPos := 0 //Posicao
29
30 Default cIdentify := ""
31 Default cType := ""
32 Default cMessage := ""
33 Default bBlock := NIL
34
35 If !Empty(cIdentify) .AND. !Empty(cType) .AND. !Empty(cMessage)
36
37     nPos := AscanMessage(cIdentify)
38
39     If nPos > 0
40         aMessage[nPos][3] += IIf(!IsBlind()),CRLF," ") + cMessage
41     Else
42         Add(aMessage,{UPPER(Alltrim(cIdentify)),AvaliableType(cType),cMessage, bBlock})
43     EndIf
44
45     // Guarda ultimas mensagens exibidas no sistema
46     //Adiciona Mensagem sempre na primeira posicao do array
47     //para manter sempre o mesmo tamanho, o ultimo elemento eh descartado
48
49
50     AINS(aMsgBkp , 1)
51     aMsgBkp[1] := {UPPER(Alltrim(cIdentify)),AvaliableType(cType),cMessage, Time()}
52

```

```

53
54 EndIf
55
56 Return NIL
57
58
59 //-----
60 /*{Protheus.doc} AvaliableType
61 avalia o tipo de mensagem recebida
62 @param   cType      Tipo de mensagem
63 @return  cType      Tipo de mensagem
64 @obs
65 @sample
66 */
67 //-----
68 Static Function AvaliableType(cType)
69
70 Default cType := ""
71
72 If UPPER(Alltrim(cType)) == "ALERT"
73     cType := "MSGALERT"
74 ElseIf UPPER(Alltrim(cType)) == "STOP"
75     cType := "MSGSTOP"
76 ElseIf UPPER(Alltrim(cType)) == "YESNO"
77     cType := "MSGYESNO"
78 ElseIf UPPER(Alltrim(cType)) == "NOYES"
79     cType := "MSGNOYES"
80 ElseIf UPPER(Alltrim(cType)) == "OK"
81     cType := "MSGINFO"
82 ElseIf UPPER(Alltrim(cType)) == "RUN"
83     cType := "MSGRUN"
84 ElseIf UPPER(Alltrim(cType)) == "FWMSGRUN"
85     cType := "FWMSGRUN"
86 ElseIf UPPER(Alltrim(cType)) == "POPUP"
87     cType := "POPUP"
88 ElseIf UPPER(Alltrim(cType)) == "STOPPOPUP"
89     cType := "STOPPOPUP"
90 Else
91     cType := ""
92 EndIf
93
94 Return cType
95
96
97 //-----
98 /*{Protheus.doc} STFGetMessage
99 Retorna as mensagens armazenadas para o ID informado por parametros
100 @param   cIdentify   ID da mensagem que deseja receber
101 @return  cMessage    retorna mensagens armazenadas para o ID informado
102 @obs
103 @sample
104 */

```

```

105 //-----
106 Function STFGetMessage(cIdentify)
107
108 Local nPos      := 0      //Posicao da mensagem no array
109 Local cMessage  := ""     //Mensagem
110
111 Default cIdentify := ""
112
113 nPos := AscanMessage(cIdentify)
114
115 If nPos > 0
116     cMessage := aMessage[nPos][3]
117 EndIf
118
119 Return cMessage
120
121
122 //-----
123 /*{Protheus.doc} STFSShowMessage
124 Exibe as mensagens do ID indicado
125 @param cIdentify      ID da mensagem que deseja exibir
126 @return uRet          retorna mensagens armazenadas para o ID informado
127 @obs
128 @sample
129 */
130 //-----
131 Function STFSShowMessage(cIdentify)
132
133 Local nPos      := 0
134 Local uRet      := NIL
135
136 Default cIdentify := ""
137
138 PARAMTYPE 0 Var cIdentify AS Character DEFAULT ""
139
140 nPos := AscanMessage(cIdentify)
141
142 If nPos > 0
143
144     //Exibe as mensagens dependendo do tipo de execucao atual
145     uRet := STIMessage(aMessage[nPos][3], aMessage[nPos][2], aMessage[nPos][4] , cIdentify )
146
147
148 EndIf
149
150 STFCleanMessage(cIdentify)
151
152 Return uRet
153
154
155 //-----
156 /*{Protheus.doc} STFCleanMessage

```

```

157 apaga as mensagens do ID indicado
158 @param  cIdentify      ID da mensagem que deseja exibir
159 @return  Nil
160 @obs
161 @sample
162 /*/
163 //-----
164 Function STFCleanMessage(cIdentify)
165
166 Local nPos := 0
167
168 Default cIdentify := ""
169
170 PARAMTYPE 0 Var cIdentify AS Character DEFAULT ""
171
172 If Empty(cIdentify)
173     aMessage := {}
174 Else
175     nPos := AscanMessage(cIdentify)
176     If nPos > 0
177         aDel(aMessage,nPos)
178         aSize(aMessage,Len(aMessage)-1)
179     EndIf
180 EndIf
181
182 Return Nil
183
184
185 //-----
186 /*/{Protheus.doc} STFCleanInterfaceMessage
187 Apaga as mensagens do interface atual
188 @return Nil
189 @obs
190 @sample
191 /*/
192 //-----
193 Function STFCleanInterfaceMessage()
194
195 STIMessage("")
196
197 Return Nil
198
199
200 //-----
201 /*/{Protheus.doc} AscanMessage
202 Procura as mensagens do ID indicado
203 @param  cIdentify      ID da mensagem que deseja exibir
204 @return  nRet          Posicao da mensagens no array de mensagens
205 @obs
206 @sample
207 /*/
208 //-----

```

```

209 Static Function AscanMessage(cIdentify)
210
211 Local nPos := 0
212 Local nRet := 0
213
214 Default cIdentify := ""
215
216 nPos := aScan(aMessage,{ |x| UPPER(Alltrim(x[1])) == UPPER(Alltrim(cIdentify)) })
217
218 If nPos > 0 .AND. !Empty(aMessage[nPos][2])
219     nRet := nPos
220 EndIf
221
222 Return nRet
223
224
225 //-----
226 /*{Protheus.doc} STIMessage
227 STIMessage Exibe as mensagens dependendo do tipo de execucao atual
228 @param cMsg      Mensagem a ser exibida
229 @param cType      Tipo da mensagem
230 @param bBlock      Bloco a ser a ser executado
231 @param cIdentify  ID da mensagem
232 @return uRet      retorno generico
233 @obs
234 @sample
235 */
236 //-----
237 Function STIMessage(cMsg, cType, bBlock , cIdentify )
238
239 Local uRet      := NIL
240 Local oMessage := Nil
241 Local nPos      := 0
242 Local lMobile   := STFCfg("lMobile", .F.)      //Smart Client Mobile
243
244 Default cMsg      := ""
245 Default cType      := ""
246 Default bBlock     := NIL
247 Default cIdentify  := ""
248
249 lMobile := ValType(lMobile) == "L" .AND. lMobile
250
251 If FindFunction("STIGetObjTela")
252     oMessage := STIGetObjTela()
253 EndIf
254
255
256 If ValType(oMessage) == "0"
257
258     If cType == "MSGRUN" .AND. ValType(bBlock) == "B"
259         If lMobile
260             nPos := AscanMessage(cIdentify)

```

```

261     Eval(aMessage[nPos][4])
262     Else
263         FWMsgRun(,bBlock, STR0004,cMsg)
264     EndIf
265     STFCleanInterfaceMessage()
266     ElseIf cType == "FWMSGRUN" .AND. ValType(bBlock) == "B"
267         FWMsgRun(,bBlock, "Aguarde", cMsg ) //"Aguarde"
268     ElseIf cType $ "MSGNOYES#MSGYESNO#MSGINFO"
269
270         //Versao Mobile exibe novas janelas de mensagens
271         If lMobile
272             Do Case
273                 Case cType == "MSGNOYES"
274                     cType := "FWAlertNoYes"
275
276                 Case cType == "MSGYESNO"
277                     cType := "FWAlertYesNo"
278
279                 Case cType == "MSGINFO"
280                     cType := "FWAlertInfo"
281
282             EndCase
283         EndIf
284
285         uRet := &(Upper(cType) + "(" + StrTran(StrTran(StrTran(StrTran(cMsg, CHR(10), " " ), CHR(13), ""),"',""), "' ' , "" ) + "'")
286     ElseIf AllTrim(cType) == "POPUP"
287         If lMobile
288             FWAlertInfo(cMsg)
289         Else
290             MsgAlert(cMsg)
291         EndIf
292     ElseIf AllTrim(cType) == "STOPPOPUP"
293         If lMobile
294             FWAlertError(cMsg)
295         Else
296             MsgStop(cMsg)
297         EndIf
298
299     Else
300         STISetMsg(Upper(cMsg))
301     EndIf
302
303
304 Else
305
306     //To Do Tratar mensagem yes/no e ok
307     If IsBlind()
308         If cType $ "MSGNOYES#MSGYESNO#MSGINFO#POPUP#STOPPOPUP"
309             uRet := .T.
310         EndIf
311         ConOut(cType + " | " + cMsg)
312         If ( cType == "MSGRUN" .OR. cType == "FWMSGRUN") .AND. ValType(bBlock) == "B"

```

```

313         FWMsgRun( ,bBlock ,"Aguarde", cMsg ) //"Aguarde"
314     EndIf
315 ElseIf AllTrim(cType) == "POPUP"
316     If lMobile
317         FWAlertInfo(cMsg)
318     Else
319         MsgAlert(cMsg)
320     EndIf
321 ElseIf AllTrim(cType) == "STOPPOPUP"
322     If lMobile
323         FWAlertError(cMsg)
324     Else
325         MsgStop(cMsg)
326     EndIf
327 ElseIf !Empty(cType) .AND. ( !( cType == "MSGRUN" .OR. cType == "FWMsGRUN") .OR. ValType(bBlock) <> "B")
328     uRet := &(Upper(cType) +"(' " + StrTran(StrTran(StrTran(cMsg, CHR(10), " " ), CHR(13), ""),"'", ""), ''', "" ) + "'")
329 ElseIf ( cType == "MSGRUN" .OR. cType == "FWMsGRUN") .AND. ValType(bBlock) == "B"
330     FWMsgRun( ,bBlock ,"Aguarde", cMsg ) //"Aguarde"
331 EndIf
332 EndIf
333
334 Return uRet
335
336
337 //-----
338 /*{Protheus.doc} STFLatestMsg()
339 Mostra as ultimas mensagens exibidas no sistema
340 @param lBorder      Mostra borda da janela
341 @param lShowEmail   Mostra opcao de email
342 @param lShowArq     Mostra opcao de Salvar em arquivo
343 @param lShowWind    Mostra janela com mensagem
344
345 @return cMSG        Retorna texto com as ultimas mensagens exibidas no sistema
346 @obs
347 @sample
348 */
349 //-----
350 Function STFLatestMsg( lBorder ,lShowEmail , lShowArq , lShowWind)
351
352 Local cMSG := ""          // Mensagem
353 Local nX   := 0          // Contador
354
355 Default lBorder      := .T.
356 Default lShowEmail   := .T.
357 Default lShowArq     := .T.
358 Default lShowWind    := .T.
359
360
361 For nX := 1 To Len(aMsgBkp)
362     If ValType( aMsgBkp[nX] ) == "A"
363         cMsg += Alltrim(Str(nX)) + (" - ")
364         cMsg += aMsgBkp[nX][4]

```

```

365      cMsg += " - " + aMsgBkp[nX][3]
366      if lClientHtml
367          cMsg += "<br><br>"
368      Else
369          cMsg += Chr(10)+Chr(13)
370      Endif
371
372  Endif
373 Next nX
374
375 if lClientHtml
376     cMsg:= "<font size=4>" + cMsg + "</font>"
377 Endif
378
379 If lShowWind
380     STPosMSG( "Histórico de mensagens", cMSG ,lBorder ,lShowEmail ,;
381             lShowArq) //"Histórico de mensagens"
382 EndIf
383
384
385 Return cMSG
386
387
388 //-----
389 /*{Protoheus.doc} STPosMSG
390 Exibe mensagem em tela com o padrao do TOTVS PDV
391 @param cTitulo      Titulo da janela
392 @param cMsg         Mensagem a ser exibida
393 @param lBorder      Mostra borda da janela
394 @param lShowEmail   Mostra opcao de email
395 @param lShowArq     Mostra opcao de Salvar em arquivo
396 @param lHtml        Formato do texto é html?
397 @param lFullScreen  Abre tela cheia?
398 @return Nil
399 @obs
400 @sample
401 */
402 //-----
403 Function STPosMSG( cTitulo      , cMsg , lBorder      ,lShowEmail , ;
404                  lShowArq , lHtml , lFullScreen)
405
406 Local oBtnOk      := Nil      //Obj botao sair
407 Local oBtnMail    := Nil      //Obj botao opcoes Copiar colar recortar
408 Local oLstMsg     := Nil      //Obj say mensagem
409 Local oDlg        := Nil      //Obj tela
410 Local oEdit       := Nil      //Objeto para texto apresentada na tela
411 Local nWidth      := 500      //Largura
412 Local nHeight     := 500      //Altura
413 Local nSEWidth    := 240      //Simple Editor Largura
414 Local nSEHeight   := 220      //Simple Editor Altura
415 Local aRes        := {}       //Recupera Resolução atual
416 Local nLineBtns   := 231      //Linha dos Botoes

```

```

417 Local nColBtOk      := 185      //Coluna do Botao Ok
418 Local nColBtEmail   := 95       //Coluna do Botao Email
419 Local nColBtArq     := 005      //Coluna do Botao Arquivo
420
421 Default cTitulo      := ""
422 Default cMsg         := ""
423 Default lBorder      := .T.
424 Default lShowEmail   := .T.
425 Default lShowArq    := .T.
426 Default lHtml        := .F.
427 Default lFullScreen := .F.
428
429 If lFullScreen
430     aRes      := GetScreenRes() // Recupera Resolução atual
431     nWidth    := aRes[1]        // Largura
432     nHeight   := aRes[2]        // Altura
433     nSEWidth  := (nWidth/2) * 0.96 // Largura
434     nSEHeight := (nHeight/2) * 0.88 // Altura
435 EndIf
436
437 DEFINE FONT oFont NAME "Courier New" SIZE 09,21
438
439 // Se exibir a borda da janela
440 If lBorder
441     DEFINE MSDIALOG oDlg TITLE cTitulo FROM 000, 000 TO nHeight, nWidth COLORS 0, 16777215 PIXEL
442 Else
443     DEFINE MSDIALOG oDlg TITLE cTitulo FROM 000, 000 TO nHeight, nWidth COLORS 0, 12632256 PIXEL STYLE nOR(WS_VISIBLE,WS_POPUP)
444 EndIf
445
446 oEdit := TSimpleEditor():New( 005      ,005      ,oDlg      ,nSEWidth      ,;
447                               nSEHeight ,      ,.T.      ,      ,;
448                               oFont      ,.T.      )
449
450 oEdit:TextFormat(IIF(lHtml,1,2))//1=Html 2=Texto simples
451 oEdit:Load(cMsg)
452
453 nLineBtns := (oEdit:nClientHeight/2) + 10 //Posicao da linha dos Botoes
454
455 If lFullScreen
456     nColBtOk := (oEdit:nClientHeight)-75 //Coluna do Botao Ok
457     nColBtEmail := (oEdit:nClientHeight/2)-25 //Coluna do Botao Email
458 EndIf
459
460 oBtnOk := TButton():New( nLineBtns,nColBtOk, STR0005 ,oDlg,{|| oDlg:End() }, 60,15,,,F.,.T.,.F.,,.F.,,.F. )// "Ok"
461 oBtnOk:SetCSS( POSCSS (GetClassName(oBtnOk) , CSS_BTN_FOCAL ))
462
463 If lShowEmail
464     oBtnMail := TButton():New( nLineBtns,nColBtEmail, STR0006 ,oDlg,{|| STFSendMail(cMsg,cTitulo,.T.) },60,15,,,F.,.T.,.F.,,.F.,,.F. )// "Email"
465     oBtnMail:SetCSS( POSCSS (GetClassName(oBtnMail) , CSS_BTN_FOCAL ))
466 EndIf
467
468 If lShowArq

```

```

469     oBtnSave := TButton():New( nLineBtns,005, STR0002 ,oDlg,{|| STFGrvArq(cMsg) },60,15,,,F.,.T.,.F.,.F.,.F.,.F. )// "Arquivo..."
470     oBtnSave:SetCSS( POSCSS (GetClassName(oBtnSave) , CSS_BTN_FOCAL ))
471 EndIf
472
473 SetKey(VK_F10, Nil )
474
475 ACTIVATE MSDIALOG oDlg CENTERED
476
477 SetKey(VK_F10, { || STFLatestMsg() } )
478
479 Return Nil
480
481 //-----
482 /*{Protheus.doc} STDGrvArq
483 Gera e grava o Arquivo texto com as Mensagens de Log
484
485 @return Nil
486 @obs
487 @sample
488 */
489 //-----
490 Function STFGrvArq(cMsg)
491
492 Local aArea := GetArea()           // Area atual
493 Local nHandle := ""               // Handle do arquivo texto
494 Local aParametros := {}           // Array que armazena os parametros MV_LJ
495 Local cCaixa := STDNumCash()       // Código do Caixa logado
496 Local cStation := STFGetStation("CODIGO") // Estacao atual
497 Local cHora := Substr(Time(),1,2) + SUBSTR(TIME(), 4, 2) // Hora formatada sem dois pontos
498 Local cData := Substr(Dtoc(dDataBase),1,2) + SUBSTR(Dtoc(dDataBase), 4, 2) + SUBSTR(Dtoc(dDataBase), 7, 4) // Data formatada sem as barras
499 Local nI := 0                     // Contador
500 Local cParametro := ""             // Parâmetro recebidos do Array para envio no FCreate
501 Local cLocal := SuperGetMV("MV_LJLLOG",,"") // Local em que será gravado o arquivo
502
503 Default cMsg := ""                 // Mensagens a serem exibidas
504
505 DbSelectArea("SX6")
506
507 DbGoTop()
508 While SX6->(!EoF())
509     If "MV_LJ" $ SX6->X6_VAR
510         Aadd (aParametros, {SX6->X6_VAR , SX6->X6_CONTEUD})
511     Endif
512 SX6->(DbSkip())
513 End
514
515 If Empty(cLocal)
516     nHandle := FCreate(cCaixa + cStation + cData + cHora + '.txt',0) // Grava na System
517 Else
518     nHandle := FCreate(cLocal + cCaixa + cStation + cData + cHora + '.txt',0) // grava no local determinado via parâmetro
519 EndIf
520

```

```
521 cParametro := cMsg
522 cParametro += Chr(10)+Chr(13) + Chr(10)+Chr(13)
523
524 For nI := 1 To Len(aParametros)
525
526     If Empty(cParametro)
527         cParametro := aParametros[nI][1] + " " + AllTrim(aParametros[nI][2]) + Chr(10)+Chr(13)
528     Else
529         cParametro += aParametros[nI][1] + " " + AllTrim(aParametros[nI][2]) + Chr(10)+Chr(13)
530     EndIf
531
532 Next nI
533
534
535 FWRITE(nHandle, cParametro ) // Grava arquivo texto
536
537 MsgAlert("Arquivo gerado com sucesso") //"Arquivo gerado com sucesso"
538
539 FClose(nHandle)
540
541 RestArea(aArea)
542
543 Return Nil
```

Framework/Message/STFYesNo.prw

107 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'PROTHEUS.CH'
2 #INCLUDE "POSCSS.CH"
3 #INCLUDE "STPOS.CH"
4 #INCLUDE "STFYESNO.CH"
5
6 //-----
7 /*{Protheus.doc} STFYesNo
8 Tela de confirmacao de processos (Sim/Nao)
9
10 @param      cMsg          Mensagem da tela
11 @param      cTitleYes     Titulo do botao de confirmacao
12 @param      cTitleNo      Titulo do botao de nao confirmacao
13 @param      bActionYes    Acao do botao de confirmacao
14 @param      bActionNo     Acao do botao de nao confirmacao
15 @param      lButYesDef    Foco do botao
16 @return
17 @obs
18 @sample
19 */
20 //-----
21 Function STFYesNo(cMsg,cTitleYes,cTitleNo,bActionYes,bActionNo,lButYesDef)
22
23 Local oPanelMVC      := STIGetPanel() //Painel principal
24 Local oLblMsg        := Nil          //Label da mensagem
25 Local oPanMsg        := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) //Criacao do painel da mensagem
26
27 Default cMsg          := ''
28 Default cTitleYes     := ''
29 Default cTitleNo      := ''
30 Default bActionYes    := {}||}
31 Default bActionNo     := {}||}
32 Default lButYesDef    := .F.
33
34 oLblMsg := TSay():New(POSVERT_CAB,POSHOR_1,{||cMsg},oPanMsg,,,,,.T.,,,)
35 oLblMsg:SetCSS( POSCSS(GetClassName(oLblMsg), CSS_BREADCUMB ) )
36
37 //Button Yes
38 oButYes := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,UPPER(cTitleYes),oPanMsg,bActionYes, ;
39                          LARGBTN,ALTURABTN,,,,.T.)
40 oButYes:SetCSS( POSCSS (GetClassName(oButYes), CSS_BTN_FOCAL ))
41
42 //Button No
43 oButNo := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,UPPER(cTitleNo),oPanMsg,bActionNo,;
44                       LARGBTN,ALTURABTN,,,,.T.)
45 oButNo:SetCSS( POSCSS (GetClassName(oButNo), CSS_BTN_FOCAL ))
46
47 If lButYesDef
48     oButYes:SetFocus()
49 Else
50     oButNo:SetFocus()
51 EndIf
52

```

```

53 Return oPanMsg
54
55
56 //-----
57 /*{Protheus.doc} STFMMsgYesNo
58 Tela de confirmacao de processos (Sim/Nao)
59
60 @param      cMsg      Mensagem da tela
61 @param      lButYesFocos  Se .T. Foco do botao Yes se .F. foca no Botão No
62 @return
63 @obs
64 @sample
65 */
66 //-----
67 Function STFMMsgYesNo( cMsg , lButYesFocos )
68
69 Local oBtnYes      := Nil      //Obj botao sair
70 Local oBtnNo       := Nil      //Obj botao opcoes Copiar colar recortar
71 Local oLstMsg      := Nil      //Obj say mensagem
72 Local oDlg         := Nil      //Obj tela
73 Local oEdit        := Nil      //Objeto para texto apresentada na tela
74 Local lRet         := .F.      //Retorno da funcao
75 Local lPermitExit  := .F.      //Valida se permite sair apenas se clicar nos botoes Sim ou Não
76
77 Default cMsg       := ""
78 Default lButYesFocos := .F.
79
80
81 DEFINE FONT oFont NAME "Courier New" SIZE 09,20
82
83 DEFINE MSDIALOG oDlg TITLE "" FROM 000, 000 TO 300, 500 COLORS 0, 16777215 PIXEL STYLE nOr(WS_VISIBLE,WS_POPUP) //Grafico Radar
84
85 oEdit := TSimpleEditor():New( 000 ,000 ,oDlg ,oDlg:nWidth ,;
86                               oDlg:nHeight , ,.T. , ,;
87                               oFont ,.T. )
88
89 oEdit:TextFormat(2)
90 oEdit:Load(cMsg)
91 oEdit:Align := CONTROL_ALIGN_ALLCLIENT
92
93 oBtnYes := TButton():New( 275/2 ,300/2 , STR0001 ,oDlg ,{|| lRet := .T. , lPermitExit:= .T. , oDlg:End() },45,20,,,F.,.T.,.F.,.F.,.F.,.F. )// "Sim"
94 oBtnYes:SetCSS( POSCSS (GetClassName(oBtnYes) , CSS_BTN_FOCAL ))
95
96 oBtnNo := TButton():New( 275/2 ,400/2 , STR0002 ,oDlg ,{|| lRet := .F. , lPermitExit:= .T. , oDlg:End() },45,20,,,F.,.T.,.F.,.F.,.F.,.F. )// "Não"
97 oBtnNo:SetCSS( POSCSS (GetClassName(oBtnNo) , CSS_BTN_NORMAL ))
98
99 If lButYesFocos
100   oBtnYes:SetFocus()
101 Else
102   oBtnNo:SetFocus()
103 EndIf
104

```

```
105 ACTIVATE MSDIALOG oDlg CENTERED VALID lPermitExit
106
107 Return lRet
```

Framework/MessageManager/STFWCMessage.prw

229 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 //ÚAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
4 //³0 Protheus necessita ter ao menos uma funcao para que o      ³
5 //³fonte seja exibido na inspecao de fontes do RPO.              ³
6 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
7 Function STFWCMessage() ; Return
8
9 //-----
10 /*{Protheus.doc} STFWCMessage
11 Classe que representa uma mensagem utilizada pela classe STFWCMessageManager.
12
13 @param
14 @see    Veja tambem a classe STFWCMessageManager
15 @return Self
16 @todo
17 @obs    Herda da classe FWSerialize
18 @sample
19 */
20 //-----
21
22 Class STFWCMessage From FWSerialize
23     Data cName
24     Data cMessage
25     Data cCallStack
26     Data nType
27     Data oInnerMessage
28
29     Method STFWCMessage()
30     Method HasMessage()
31     Method HasError()
32     Method HasWarning()
33     Method HasInformation()
34     Method ToString()
35 EndClass
36
37 //-----
38 /*{Protheus.doc} STFWCMessage
39 Metodo construtor da classe STFWCMessage.
40
41 @param  cName          Nome identificador da mensagem
42 @param  nType          Tipo da mensagem, sendo: 1 - Erro | 2 - Aviso | 3 - Informacao
43 @param  cMessage       A mensagem
44 @param  oInnerMessage  Objeto STFWCMessage interno,ou a mensagem que gerou essa.
45
46 @see
47 @return Self
48 @todo
49 @obs    Classe STFWCMessage
50 @sample
51 */
52 //-----

```

```

53
54 Method STFWMMessage( cName, nType, cMessage, oInnerMessage) Class STFWMMessage
55     Local nCount      := 1           // Contador de for
56     Local aCallStack  := {}          // Array que armazenara a pilha de chamada
57     Local aType       := {}          // Tipos de retorno
58     Local aFile       := {}          // Array que armazenara os arquivos
59     Local aLine       := {}          // Array que armazenara as linhas da mensagem
60     Local aDate       := {}          // Array que armazenara as datas
61     Local aTime       := {}          // Array que armazenara as horas
62
63     Self:cName        := cName
64     Self:cMessage     := cMessage
65     Self:oInnerMessage := oInnerMessage
66     Self:nType        := nType
67     Self:cCallStack   := ""
68
69     If ValType(cName) != "U"
70         // Pega o callstack
71         While !Empty(ProcName(nCount))
72             aAdd( aCallStack, ProcName(nCount) )
73             nCount++
74         End
75
76         For nCount := 1 To Len(aCallStack)
77             GetFuncArray( aCallStack[nCount], @aType, @aFile, @aLine, @aDate, @aTime )
78
79             Self:cCallStack += "    on " + aCallStack[nCount] + "(" + If(Len(aFile)>0, aFile[1], "Unknown") + ") " + If(Len(aFile)>0,DToc(aDate[1]),"Unknown") + " line : " +
AllTrim(Str(ProcLine(nCount+1))) + If(Len(aCallStack)==nCount,"",Chr(13) + Chr(10))
80             nCount++
81         End
82     EndIf
83 Return Self
84
85 //-----
86 /*/{Protheus.doc} HasMessage
87 Se eh uma mensagem e/ou se tem alguma mensagem interna com o nome passado pelo parametro.
88
89 @param    cMessageName      Nome identificador da mensagem
90 @see
91 @return   lHasMessage: .T. se há alguma mensagem ou a mensagem questionada.
92 @todo
93 @obs      Classe STFWMMessage
94 @sample
95 /*/
96 //-----
97
98 Method HasMessage( cMessageName ) Class STFWMMessage
99     Local lHasMessage := .T.          // Mensagem de retorno
100
101     If cMessageName != Nil
102         lHasMessage := .F.
103         If Lower(Self:cName) == Lower(cMessageName)

```

```

104         lHasMessage := .T.
105     Else
106         If Self:oInnerMessage != Nil
107             lHasMessage := Self:oInnerMessage:HasMessage( cMessageName )
108         EndIf
109     EndIf
110 EndIf
111 Return lHasMessage
112
113 //-----
114 /*/{Protheus.doc} HasError
115 Se eh uma mensagem do tipo erro e/ou se tem alguma mensagem interna do tipo erro com o nome passado
116 pelo parametro.
117
118 @param  cMessageName      Nome da mensagem a ser identificada. (Opcional)
119 @see
120 @return lHasMessage: .T. se ha alguma mensagem ou a mensagem questionada.
121 @todo
122 @obs    Classe STFWCMessage
123 @sample
124 */
125 //-----
126
127 Method HasError( cMessageName ) Class STFWCMessage
128     Local lHasMessage := .T.          // Mensagem de retorno
129
130     If cMessageName != Nil
131         lHasMessage := .F.
132         If Lower(Self:cName) == Lower(cMessageName) .And. Self:nType == 1
133             lHasMessage := .T.
134         Else
135             If Self:oInnerMessage != Nil
136                 lHasMessage := Self:oInnerMessage:HasError( cMessageName )
137             EndIf
138         EndIf
139     EndIf
140 Return lHasMessage
141
142 //-----
143 /*/{Protheus.doc} HasWarning
144 Se eh uma mensagem do tipo erro e/ou se tem alguma mensagem interna do tipo aviso com o nome passado
145 pelo parametro.
146
147 @param  cMessageName      Nome da mensagem a ser identificada. (Opcional)
148 @see
149 @return lHasMessage: .T. se ha alguma mensagem ou a mensagem questionada.
150 @todo
151 @obs    Classe STFWCMessage
152 @sample
153 */
154 //-----
155

```

```

156 Method HasWarning( cMessageName ) Class STFWCMessage
157     Local lHasMessage := .T.           // Mensagem de retorno
158
159     If cMessageName != Nil
160         lHasMessage := .F.
161         If Lower(Self:cName) == Lower(cMessageName) .And. Self:nType == 2
162             lHasMessage := .T.
163         Else
164             If Self:oInnerMessage != Nil
165                 lHasMessage := Self:oInnerMessage:HasWarning( cMessageName )
166             EndIf
167         EndIf
168     EndIf
169 Return lHasMessage
170
171 //-----
172 /*/{Protheus.doc} HasInformation
173 Se eh uma mensagem do tipo erro e/ou se tem alguma mensagem interna do tipo informacao com o nome
174 passado pelo parametro.
175
176 @param    cMessageName      Nome da mensagem a ser identificada. (Opcional)
177 @see
178 @return   lHasMessage: .T. se ha alguma mensagem ou a mensagem questionada.
179 @todo
180 @obs      Classe STFWCMessage
181 @sample
182 */
183 //-----
184
185 Method HasInformation( cMessageName ) Class STFWCMessage
186     Local lHasMessage := .T.           // Mensagem de retorno
187
188     If cMessageName != Nil
189         lHasMessage := .F.
190         If Lower(Self:cName) == Lower(cMessageName) .And. Self:nType == 3
191             lHasMessage := .T.
192         Else
193             If Self:oInnerMessage != Nil
194                 lHasMessage := Self:oInnerMessage:HasWarning( cMessageName )
195             EndIf
196         EndIf
197     EndIf
198 Return lHasMessage
199
200 //-----
201 /*/{Protheus.doc} ToString
202 Converte a mensagem para um texto de exibicao.
203
204 @param
205 @see
206 @return   cText: Texto de exibicao.
207 @todo

```

```
208 @obs      Classe STFWCMessage
209 @sample
210 /*/
211 //-----
212
213 Method ToString() Class STFWCMessage
214
215     Local cText := ""           // Variavel de texto de exibicao.
216
217     cText += "Message: " + Self:cName
218     If Self:cMessage != Nil .Or. !Empty(Self:cMessage)
219         cText += Chr(13) + Chr(10) + Self:cMessage
220     EndIf
221     If Self:cCallStack != Nil .Or. !Empty(Self:cCallStack)
222         cText += Chr(13) + Chr(10) + Self:cCallStack
223     EndIf
224
225     // Se existir mensagem interna, pega o texto dela
226     If Self:oInnerMessage != Nil
227         cText += Chr(13) + Chr(10) + Self:oInnerMessage:ToString()
228     EndIf
229 Return cText
```

Framework/MessageManager/STFWCMessageManager.prw

341 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "LOJA0049.CH"
3
4 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
5 //³0 Protheus necessita ter ao menos uma funcao para que o      ³
6 //³fonte seja exibido na inspecao de fontes do RPO.              ³
7 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
8 Function StFwCMsgMe() ; Return
9
10 //-----
11 /*/{Protheus.doc} STFWCMessageManager
12 Gerenciador de troca de mensagens, utilizada para a troca de mensagens entre funcoes.
13
14 @param
15 @see    Veja tambem a classe STFWCMessageManager
16 @return Self
17 @todo
18 @obs    Herda da classe FWSerialize
19 @sample
20
21 Static Function TstLJMM()
22     Local oLJCMessageManager := GetLJCMessageManager()
23     Public __cInternet
24
25     __cInternet := "AUTOMATICO"
26
27     If oLJCMessageManager:HasMessage()
28         ConOut("Falhou")
29     Else
30         ConOut("Passou")
31     EndIf
32
33     oLJCMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroA", 1, "Ocorreu o erro no TCP" ) )
34     If oLJCMessageManager:HasMessage()
35         ConOut("Passou")
36     Else
37         ConOut("Falhou")
38     EndIf
39
40     oLJCMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroB", 1, "Não foi possivel efetuar o download." ) )
41     If oLJCMessageManager:HasMessage( "ErroB" )
42         ConOut("Passou")
43     Else
44         ConOut("Falhou")
45     EndIf
46
47     oLJCMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroC", 2, "Não foi possivel efetuar carga" ) )
48     If oLJCMessageManager:HasWarning()
49         ConOut("Passou")
50     Else
51         ConOut("Falhou")
52     EndIf

```

```

53
54     If oLJCMessageManager:HasWarning( "ErroC" )
55         ConOut("Passou")
56     Else
57         ConOut("Falhou")
58     EndIf
59
60     If oLJCMessageManager:HasInformation( "ErroC" )
61         ConOut("Falhou")
62     Else
63         ConOut("Passou")
64     EndIf
65
66     oLJCMessageManager:Show( "Há uma mensagem" )
67
68     ConOut( oLJCMessageManager:Serialize() )
69
70     oLJCMessageManager:Clear()
71     If oLJCMessageManager:HasMessage()
72         ConOut("Falhou")
73     Else
74         ConOut("Passou")
75     EndIf
76 Return
77
78 /*/
79 //-----
80
81 Class STFWCMessageManager From FWSerialize
82     Data oMessage
83
84     Method STFWCMessageManager()
85     Method HasMessage()
86     Method HasError()
87     Method HasWarning()
88     Method HasInformation()
89     Method ThrowMessage()
90     Method Clear()
91     Method Show()
92 EndClass
93
94 //-----
95 /*/{Protheus.doc} STFWCMessageManager
96 Metodo construtor da classe STFWCMessageManager.
97
98 @param
99 @see
100 @return Self
101 @todo
102 @obs     Classe STFWCMessageManager
103 @sample
104 /*/

```

```

105 //-----
106
107 Method STFWCMessageManager() Class STFWCMessageManager
108
109 Return Self
110
111 //-----
112 /*{Protheus.doc} HasMessage
113 Se eh uma mensagem e/ou se tem alguma mensagem interna com o nome passado pelo parametro.
114
115
116 @param cMessageName Nome da mensagem a ser identificada. (Opcional).
117 @see
118 @return lHasMessage: .T. se há alguma mensagem ou a mensagem questionada.
119 @todo
120 @obs Classe STFWCMessageManager
121 @sample
122 */
123 //-----
124
125 Method HasMessage( cMessageName ) Class STFWCMessageManager
126 Local lHasMessage := .F.
127
128 If Self:oMessage != Nil
129 If cMessageName == Nil
130 lHasMessage := .T.
131 Else
132 lHasMessage := Self:oMessage:HasMessage( cMessageName )
133 EndIf
134 EndIf
135 Return lHasMessage
136
137 //-----
138 /*{Protheus.doc} HasError
139 Se eh uma mensagem do tipo erro e/ou se tem alguma mensagem interna do tipo erro com o nome passado
140 pelo parametro.
141
142 @param cMessageName Nome da mensagem a ser identificada. (Opcional)
143 @see
144 @return lHasMessage: .T. se ha alguma mensagem ou a mensagem questionada.
145 @todo
146 @obs Classe STFWCMessageManager
147 @sample
148 */
149 //-----
150
151 Method HasError( cMessageName ) Class STFWCMessageManager
152 Local lHasMessage := .F.
153
154 If Self:oMessage != Nil
155 If cMessageName == Nil
156 lHasMessage := .T.

```

```

157         Else
158             lHasMessage := Self:oMessage:HasError( cMessageName )
159         EndIf
160     EndIf
161 Return lHasMessage
162
163 //-----
164 /*{Protheus.doc} HasWarning
165 Se eh uma mensagem do tipo erro e/ou se tem alguma mensagem interna do tipo aviso com o nome passado
166 pelo parametro.
167
168 @param    cMessageName      Nome da mensagem a ser identificada. (Opcional)
169 @see
170 @return   lHasMessage: .T. se ha alguma mensagem ou a mensagem questionada.
171 @todo
172 @obs      Classe STFWCMessageManager
173 @sample
174 */
175 //-----
176
177 Method HasWarning( cMessageName ) Class STFWCMessageManager
178     Local lHasMessage := .F.
179
180     If Self:oMessage != Nil
181         If cMessageName == Nil
182             lHasMessage := .T.
183         Else
184             lHasMessage := Self:oMessage:HasWarning( cMessageName )
185         EndIf
186     EndIf
187 Return lHasMessage
188
189 //-----
190 /*{Protheus.doc} HasInformation
191 Se eh uma mensagem do tipo erro e/ou se tem alguma mensagem interna do tipo informacao com o nome
192 passado pelo parametro.
193
194 @param    cMessageName      Nome da mensagem a ser identificada. (Opcional)
195 @see
196 @return   lHasMessage: .T. se ha alguma mensagem ou a mensagem questionada.
197 @todo
198 @obs      Classe STFWCMessageManager
199 @sample
200 */
201 //-----
202
203 Method HasInformation( cMessageName ) Class STFWCMessageManager
204     Local lHasMessage := .F.
205
206     If Self:oMessage != Nil
207         If cMessageName == Nil
208             lHasMessage := .T.

```

```

209         Else
210             lHasMessage := Self:oMessage:HasInformation( cMessageName )
211         EndIf
212     EndIf
213 Return lHasMessage
214
215 //-----
216 /*{Protheus.doc} ThrowMessage
217 Grava uma mensagem no gerenciador de mensagens.
218
219 @param  oMessage          Objeto LJCMMessage com a mensagem.
220 @see
221 @return Self
222 @todo
223 @obs    Classe STFWCMessageManager
224 @sample
225 */
226 //-----
227
228 Method ThrowMessage( oMessage ) Class STFWCMessageManager
229     oMessage:oInnerMessage := Self:oMessage
230     Self:oMessage := oMessage
231 Return
232
233 //-----
234 /*{Protheus.doc} Clear
235 Limpa a cadeia de mensagens existente no gerenciador de mensagem.
236
237 @param
238 @see
239 @return Self
240 @todo
241 @obs    Classe STFWCMessageManager
242 @sample
243 */
244 //-----
245
246 Method Clear() Class STFWCMessageManager
247     Self:oMessage := Nil
248 Return
249
250 //-----
251 /*{Protheus.doc} Show
252 Exibe a mensagem e suas mensagens internas.
253
254 @param  cMessage: Texto de exibição principal.
255 @see
256 @return Self
257 @todo
258 @obs    Classe STFWCMessageManager
259 @sample
260 */

```

```

261 //-----
262
263 Method Show( cMessage ) Class STFWCMessageManager
264     Local oDlgII           := Nil
265     Local oFntTit          := Nil
266     Local oFntMsg          := Nil
267     Local oBmp             := Nil
268     Local oMsgDet          := Nil
269     Local lTelaDetalhe     := .F.
270     Local lExibeBotaoDetalhe := .F.
271
272     If IsBlind()
273         ConOut( STR0001 + ": " + cMessage ) // "Mensagem"
274         If Self:oMessage != Nil
275             ConOut( STR0002 + ": " + Self:oMessage:ToString() ) // "Detalhes"
276         EndIf
277     Else
278         If Self:HasMessage() .And. Self:oMessage != Nil
279             lExibeBotaoDetalhe := .T.
280         EndIf
281
282         DEFINE MSDIALOG oDlgII TITLE STR0001 FROM 0,0 TO 130,600 PIXEL // "Mensagem"
283
284         DEFINE FONT oFntTit NAME "Arial" SIZE 6,16 BOLD
285         DEFINE FONT oFntMsg NAME "Arial" SIZE 5,15
286
287         @05,2 TO 45,300 PROMPT STR0003 PIXEL // "Informação"
288         @11,4 GET cMessage FONT oFntMsg MULTILINE NOBORDER READONLY HSCROLL SIZE 245,30 PIXEL
289
290         @50,200 BUTTON "OK" PIXEL ACTION oDlgII:End()
291
292         If lExibeBotaoDetalhe .And. Self:oMessage != Nil
293             @50,230 BUTTON STR0002 PIXEL ACTION If( !lTelaDetalhe,; // "Detalhes"
294                 (oDlgII:ReadClientCoors(.T.),oDlgII:Move(oDlgII:nTop,oDlgII:nLeft,oDlgII:nWidth,oDlgII:nHeight+165,,.T.),lTelaDetalhe:=.T.),;
295                 (oDlgII:ReadClientCoors(.T.),oDlgII:Move(oDlgII:nTop,oDlgII:nLeft,oDlgII:nWidth,oDlgII:nHeight-165,,.T.),lTelaDetalhe:=.F.))
296             @ 67,2 TO 140,300 PROMPT STR0004 PIXEL // "Detalhes da informação:"
297             @ 73,4 GET oMsgDet VAR Self:oMessage:ToString() FONT oFntMsg MULTILINE NOBORDER READONLY HSCROLL SIZE 245,65 PIXEL
298         EndIf
299
300         ACTIVATE MSDIALOG oDlgII CENTERED
301     EndIf
302 Return
303
304 //-----
305 /*{Protheus.doc} GetSTFWCMessageManager
306 Pega o gerenciador de mensagens global.
307
308 @param
309 @see
310 @return Self
311 @todo
312 @obs      Funcao Get

```

```
313 @sample
314 /*/
315 //-----
316
317 Function GetSTFWCMessageManager()
318     Public _oSTFWCMessageManager
319
320     If _oSTFWCMessageManager == Nil
321         _oSTFWCMessageManager := STFWCMessageManager():New()
322     EndIf
323 Return _oSTFWCMessageManager
324
325 //-----
326 /*/{Protheus.doc} SetSTFWCMessageManager
327 Configura o gerenciador de mensagens global.
328
329 @param
330 @see
331 @return Self
332 @todo
333 @obs      Funcao Set
334 @sample
335 /*/
336 //-----
337
338 Function SetSTFWCMessageManager( oSTFWCMessageManager )
339     Public _oSTFWCMessageManager
340     _oSTFWCMessageManager := oSTFWCMessageManager
341 Return
```

Framework/MessageManager/STFWCTestMessageManager.prw

103 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 Class STFWCTestMessageManager From FWDefaultTestCase
4     Method STFWCTestMessageManager()
5
6     Method TestHasNoMessage()
7     Method TestHasMessage()
8     Method TestHasSMessage()
9     Method TestHasWarning()
10    Method TestHasSWarning()
11    Method TestHasNoSWarning()
12    Method TestMultiThrow()
13    Method TestClear()
14    Method TestShow()
15 EndClass
16
17 Method STFWCTestMessageManager() Class STFWCTestMessageManager
18     _Super:FWDefaultTestCase()
19     Self:AddTestMethod("TestHasNoMessage")
20     Self:AddTestMethod("TestHasSMessage")
21     Self:AddTestMethod("TestHasWarning")
22     Self:AddTestMethod("TestHasSWarning")
23     Self:AddTestMethod("TestHasNoSWarning")
24     Self:AddTestMethod("TestMultiThrow")
25     Self:AddTestMethod("TestClear")
26     //Self:AddTestMethod("TestShow")
27 Return
28
29 Method TestHasNoMessage() Class STFWCTestMessageManager
30     Local oMessageManager := STFWCMessageManager():STFWCMessageManager()
31     Local oResult := FWTestResult():FWTestResult()
32
33     oResult:AssertFalse( oMessageManager:HasMessage() )
34 Return oResult
35
36 Method TestHasMessage() Class STFWCTestMessageManager
37     Local oMessageManager := STFWCMessageManager():STFWCMessageManager()
38     Local oResult := FWTestResult():FWTestResult()
39
40     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroA", 1, "Ocorreu o erro no TCP" ) )
41     oResult:AssertTrue( oMessageManager:HasMessage() )
42 Return oResult
43
44 Method TestHasSMessage() Class STFWCTestMessageManager
45     Local oMessageManager := STFWCMessageManager():STFWCMessageManager()
46     Local oResult := FWTestResult():FWTestResult()
47
48     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroB", 1, "Não foi possível efetuar o download." ) )
49     oResult:AssertTrue( oMessageManager:HasMessage("ErroB") )
50 Return oResult
51
52 Method TestHasWarning() Class STFWCTestMessageManager

```

```

53     Local oMessageManager      := STFWCMessageManager():STFWCMessageManager()
54     Local oResult              := FWTestResult():FWTestResult()
55
56     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroC", 2, "Não foi possível efetuar carga" ) )
57     oResult:AssertTrue( oMessageManager:HasWarning() )
58 Return oResult
59
60 Method TestHasSwarning() Class STFWCTestMessageManager
61     Local oMessageManager      := STFWCMessageManager():STFWCMessageManager()
62     Local oResult              := FWTestResult():FWTestResult()
63
64     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroC", 2, "Não foi possível efetuar carga" ) )
65     oResult:AssertTrue( oMessageManager:HasWarning("ErroC") )
66 Return oResult
67
68 Method TestHasNoSwarning() Class STFWCTestMessageManager
69     Local oMessageManager      := STFWCMessageManager():STFWCMessageManager()
70     Local oResult              := FWTestResult():FWTestResult()
71
72     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroC", 2, "Não foi possível efetuar carga" ) )
73     oResult:AssertFalse( oMessageManager:HasWarning("ErroD") )
74 Return oResult
75
76 Method TestMultiThrow() Class STFWCTestMessageManager
77     Local oMessageManager      := STFWCMessageManager():STFWCMessageManager()
78     Local oResult              := FWTestResult():FWTestResult()
79
80     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroC", 2, "Não foi possível efetuar carga" ) )
81     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroD", 2, "Não foi possível efetuar carga" ) )
82     oResult:AssertTrue( oMessageManager:HasWarning("ErroC") )
83 Return oResult
84
85 Method TestClear() Class STFWCTestMessageManager
86     Local oMessageManager      := STFWCMessageManager():STFWCMessageManager()
87     Local oResult              := FWTestResult():FWTestResult()
88
89     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroC", 2, "Não foi possível efetuar carga" ) )
90     oMessageManager:Clear()
91     oResult:AssertFalse( oMessageManager:HasWarning("ErroC") )
92 Return oResult
93
94 Method TestShow() Class STFWCTestMessageManager
95     Local oMessageManager      := STFWCMessageManager():STFWCMessageManager()
96     Local oResult              := FWTestResult():FWTestResult()
97
98     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroC", 1, "Deu problema no pacote TCP." ) )
99     oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroD", 1, "Deu problema na comunicação" ) )
100    oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroE", 1, "Download não foi efetuado." ) )
101    oMessageManager:ThrowMessage( STFWCMessage():STFWCMessage( "ErroF", 1, "A carga não subiu." ) )
102    oMessageManager:Show("Ih, aconteceu alguma coisa.")
103 Return oResult

```